



We are on a mission to address the digital skills gap for 10 Million+ young professionals, train and empower them to forge a career path into future tech

TestNG Framework

MAR 2026



Contents

- Introduction and Unit Testing
- TestNG Annotations and its Types
- TestNG Priority
- TestNG Dependency
- TestNG Groups
- TestNG Xml file and its configuration
- TestNG parameters
- TestNG Assertions
- TestNG Suite & Parallel Execution
- TestNG DataProviders
- TestNG ITest Listeners
- TestNG Reporting

Introduction to Unit Testing



Introduction

- TestNG is a testing framework inspired from JUnit and NUnit but introducing some new functionalities that make it more powerful and easier to use.
- A famous programmer named as “Cedric Beust” developed TestNG. It is distribute under Apache Software License and is easily available to download.
- testNG is free and open source.

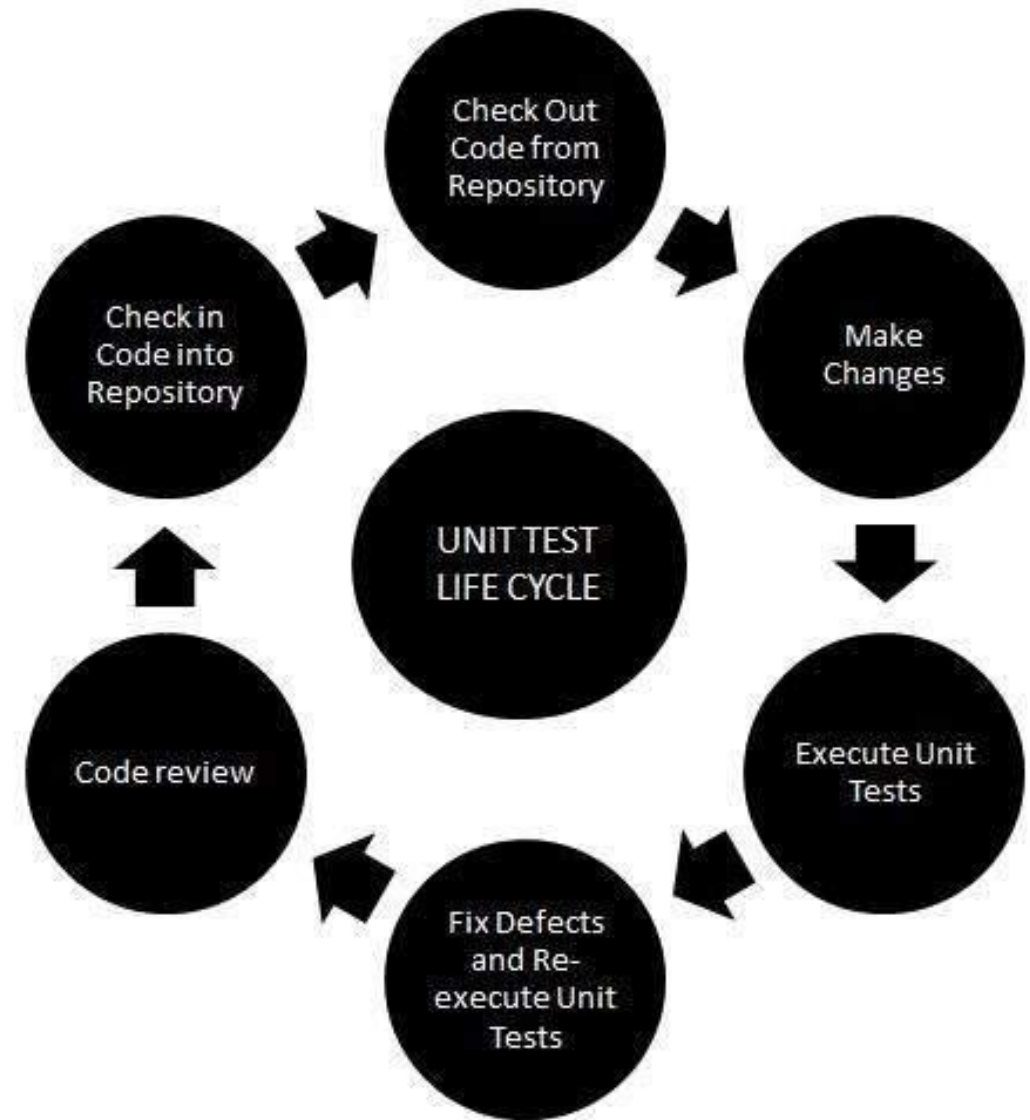
[<https://javadoc.io/doc/org.testng/testng/latest/org/testng/package-summary.html>]

- TestNG requires JDK 7 or higher.
- NG stands Next Generation.
- We can use testNG only with java programming language.
- Developers use testNG for automating Unit Tests.
- Testers use testNG for automating UI, API and mobile Tests.

Introduction and Unit Testing

Unit Testing

- Unit Testing is a type of software testing where individual units or components of a software are tested
- The purpose is to validate that each unit of the software code performs as expected
- Unit Testing is done during the development (coding phase) of an application by the developers.
- Unit Tests isolate a section of code and verify its correctness.



Unit Testing and its Advantages

- A unit may be an individual function, method, procedure, module or object.

Advantages of Unit Testing :

- Reduces defects in the newly developed features or reduces bugs when changing the existing functionality.
- Reduces cost of testing as defects are captured in very early phase.
- Improves design and allows better refactoring of code.
- Unit Test, when integrated with build gives the quality of the build as well.

Importance of Testing Frameworks in Automation

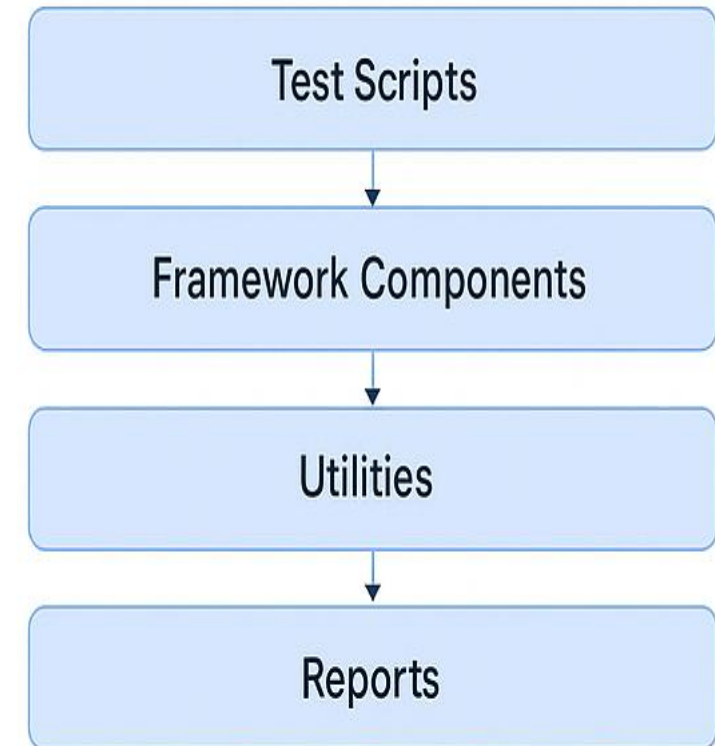


Introduction to Testing Framework

- A testing framework is a set of guidelines, rules, and reusable components that define how test automation should be designed, created, and executed.
- It provides a structured and standardized approach to automate test cases efficiently.

1. Reusability of Code:

- Frameworks promote modular and reusable code by organizing test scripts, utilities, and test data separately.
- Example: Common functions (like login or setup) can be reused across multiple tests, reducing duplication.



Introduction to Testing Framework

2. Improved Test Maintenance

- Frameworks separate test logic from the application logic.
- When an application changes, only the affected part of the framework (not all scripts) needs to be updated — improving maintainability.

3. Enhanced Test Reporting

- Most frameworks include or integrate with reporting tools (like Allure, Extent Reports, or JUnit Reports) that provide detailed test execution results, helping in quick defect analysis and decision-making.

4. Better Test Coverage and Scalability

- Frameworks allow tests to be structured systematically — supporting large test suites, multiple environments, and different browsers or devices.
- Example: Selenium with TestNG or PyTest supports parallel execution to speed up testing.

Introduction to Testing Framework

6. Integration with CI/CD Tools

- Modern frameworks integrate easily with tools like Jenkins, GitLab CI, or Azure DevOps, enabling continuous testing as part of the DevOps pipeline.

7. Data-Driven and Keyword-Driven Testing

- Frameworks support data-driven (separating test data from logic) and keyword-driven (using predefined keywords for test actions) approaches, improving flexibility and reducing human error.

8. Reduced Test Execution Time

- Parallel execution, optimized code, and modular design allow faster test runs — improving productivity and enabling quicker feedback to developers.

9. Better Collaboration and Version Control

- Frameworks support integration with version control systems (like Git), allowing multiple testers to work collaboratively on the same project.

Introduction to Testing Framework

Example Frameworks

- Selenium + TestNG / JUnit (Java)
- PyTest / Robot Framework (Python)
- Cypress / Jest (JavaScript)
- Cucumber (BDD-based Framework)



Overview of TestNG and Junit Frameworks



Key Features of TestNG



- **Annotation-based** testing (e.g., @BeforeSuite, @AfterTest, @Test)
- **Supports parallel execution** of tests to save time
- **Flexible test configuration** using testng.xml
- **Supports dependency testing** (e.g., run a test only if another test passes)
- **Data-driven testing** using @DataProvider
- **Generates detailed HTML and XML reports**
- **Integration-friendly** with Selenium, Maven, Jenkins, etc.

Introduction to JUnit



- JUnit is one of the most widely used unit testing frameworks for Java.
- It provides simple annotations and assertions for testing small units of code — typically methods or classes — in isolation.
- JUnit has evolved over time, and JUnit 5 offers modularity, extensibility, and better integration with modern build tools.

Key Features of JUnit:

- Lightweight and easy to use for unit-level testing
- Uses annotations like `@BeforeAll`, `@BeforeEach`, `@Test`, `@AfterEach`, `@AfterAll`
- Supports assertions to validate test outcomes
- Can run via IDEs, Maven, or Gradle easily
- JUnit 5 supports tagging and conditional test execution
- Limited support for parallel testing and parameterization (compared to TestNG)

TestNG vs JUnit

TestNG vs JUnit

Feature	TestNG	JUnit (mainly JUnit 5)
Full Form	Test Next Generation	Java Unit Testing Framework
Origin	Inspired by JUnit and NUnit	Original Java testing framework
Annotations	@BeforeSuite, @BeforeTest, @Test, @DataProvider, etc.	@BeforeAll, @BeforeEach, @Test, @ParameterizedTest, etc.
Configuration File	Supports XML configuration (testng.xml)	No XML config; uses code/annotations only
Parallel Execution	Fully supported	Limited support in JUnit 5
Dependency Testing	Supported via dependsOnMethods, dependsOnGroups	Not directly supported
Grouping of Tests	Supported (groups attribute)	Supported via @Tag annotation
Parameterization	Supported using @DataProvider	Supported via @ParameterizedTest but less flexible
Reporting	Built-in HTML and XML reports	Basic reports; external tools needed
Suite Execution Control	Easily managed via XML	Managed programmatically or via build tool
Integration with Tools	Excellent with Selenium, Jenkins, Maven	Excellent with Maven, Gradle, IDEs
Ease of Use	Slightly more complex (due to advanced features)	Simpler for small-scale unit testing
Best Suited For	Functional, Integration, and End-to-End tests	Unit and Component-level tests

Quiz



1) What is the main purpose of using a testing framework in automation?

a) To execute test cases manually

b) To design, organize, and execute automated tests efficiently

c) To record user actions for debugging

d) To compile Java programs

Answer : Option b)

Quiz



2) Which of the following is an advantage of using testing frameworks?

a) Reduced test coverage

b) Increased manual effort

c) Improved test maintenance and reusability

d) Slower execution speed

Answer : Option c)

Quiz



3) You are automating 150 login and checkout test cases. Your manager suggests using a testing framework instead of writing standalone Selenium scripts. What would be the most significant benefit in this case?

a) Increased execution time due to framework overhead

b) No need for test data management

c) Easier reusability and centralized test maintenance

d) Reduced need for reports

Answer : Option c)

Quiz



4) You are using JUnit but want to create dependent test cases (e.g., "Run Dashboard test only if Login test passes"). Which framework would best suit this need and why?

a) JUnit, because it uses tagging

b) TestNG, because it supports dependency-based execution

c) JUnit, because it runs tests alphabetically

d) Both frameworks support dependencies equally

Answer : Option b)

Quiz



5) You are comparing JUnit 5 and TestNG for a cross-browser Selenium framework. Why might TestNG be the preferred choice in this case?

a) TestNG does not support parallel testing

b) TestNG integrates easily with CI/CD and supports parallel execution

c) JUnit can only test APIs

d) JUnit cannot be used with Java 8+

Answer : Option b)

Quiz



6) Which of the following features is unique to TestNG compared to JUnit?

a) Annotation-based testing

b) Assertion-based validation

c) Test execution from IDE

d) Data-driven testing using `@DataProvider`

Answer : Option d)

Installing TestNG plugin in Eclipse



Installing TestNG plugin in Eclipse

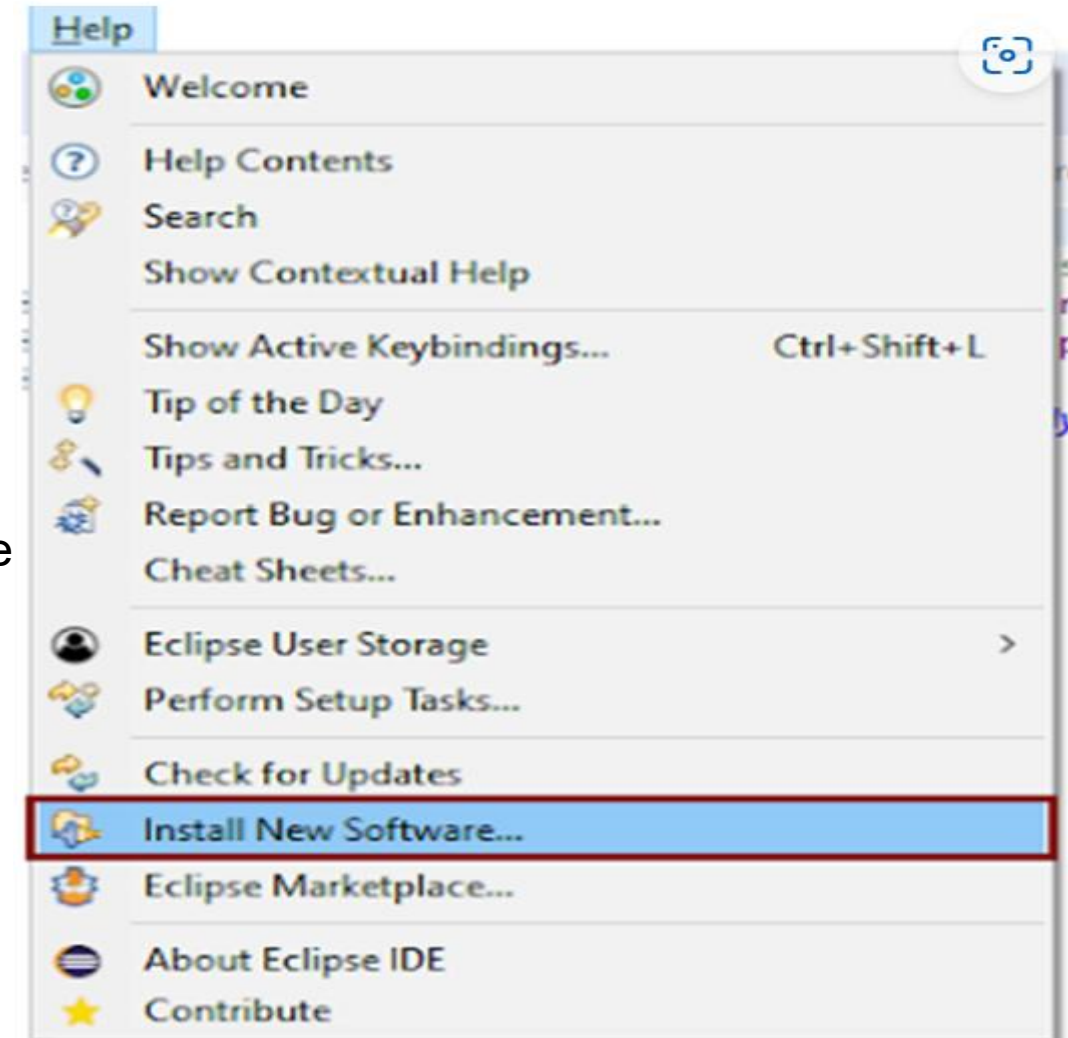
Download and Installing

- **Pre-Requisite:**

1. Java 7 and above.
2. Eclipse or any Java IDE

- **Install/Setup testNG:**

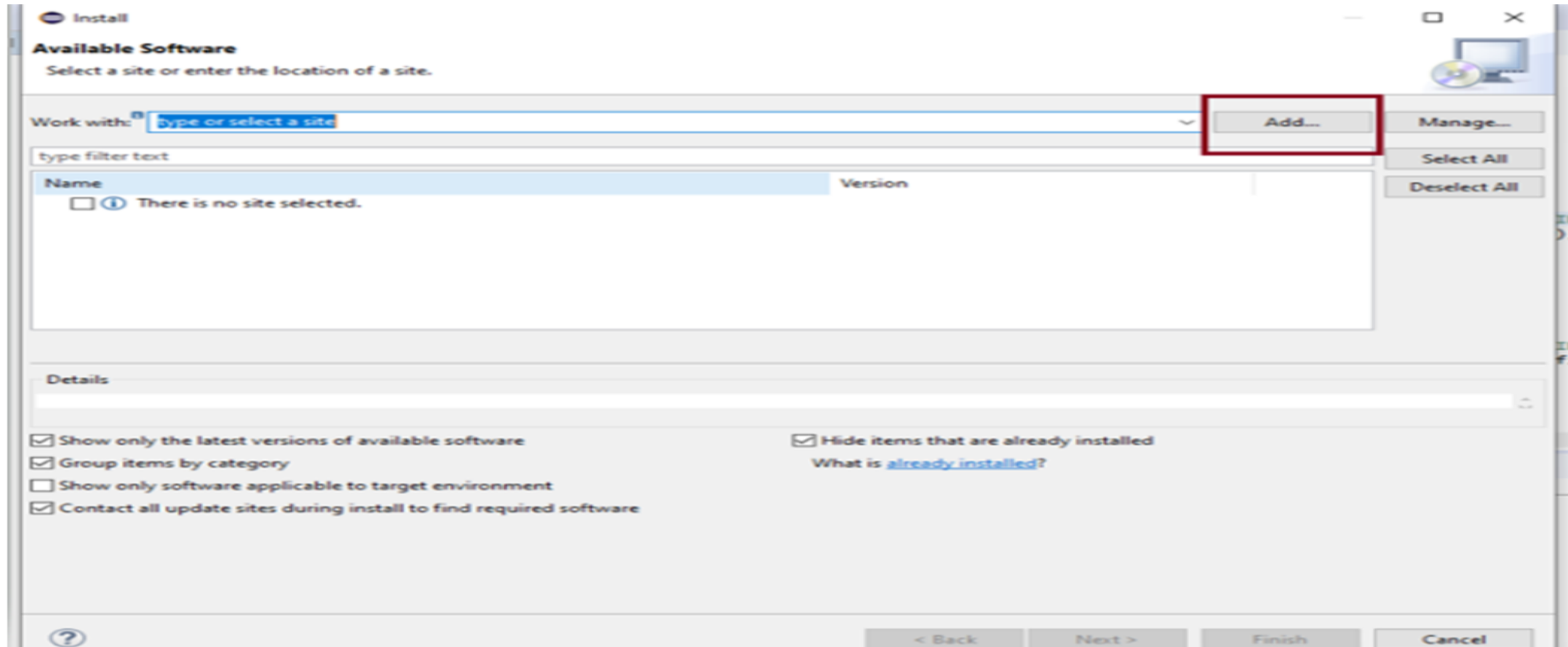
- 1) Launch Eclipse and go to the “Help”
option present at the top and select –“Install New Software



Installing TestNG plugin in Eclipse

Download and Installing

2) A dialog box will appear, click the “Add” button. Launch Eclipse and go to the “Help”



Installing TestNG plugin in Eclipse

Download and Installing

3) A new dialog box will appear. Mention the Name as TestNG and the location as “ **<https://testng.org/testng-eclipse-update-site>**; and click the Add button.

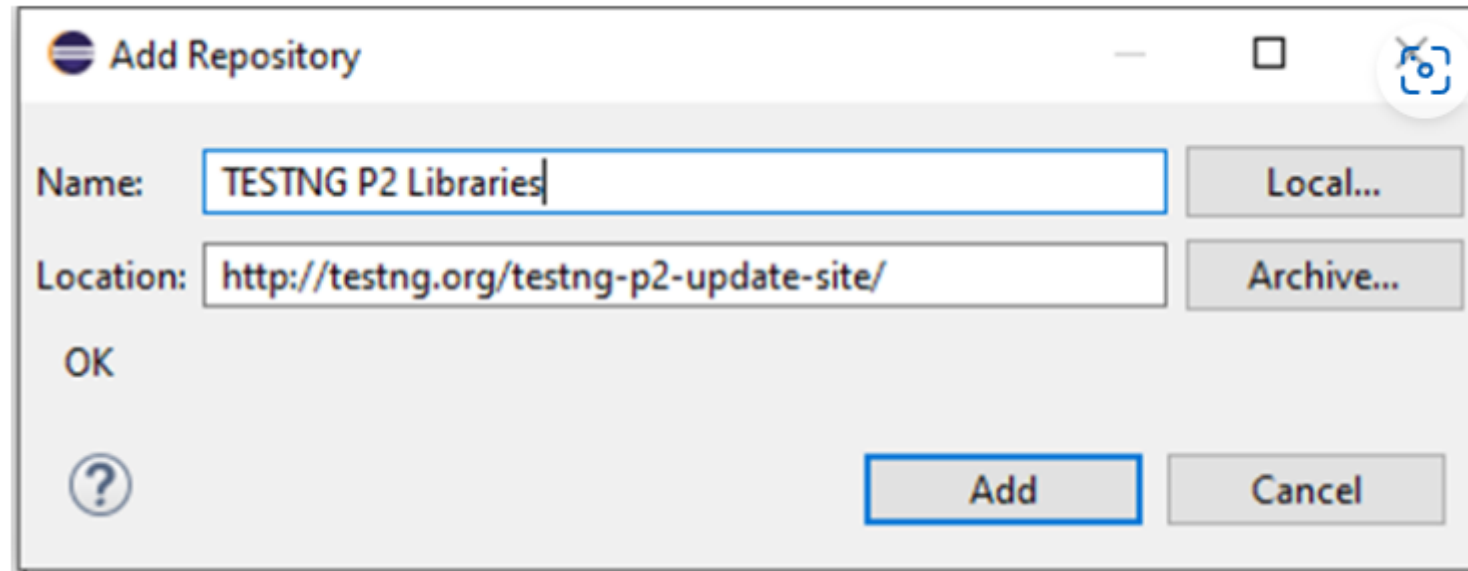
To Choose the location site go to the following website **<https://testng.org/doc/download.html>** , copy and paste the URL.

Install from update site

- Select Help / Install New Software...
- Enter the update site URL in "Work with:" field:
 - Update site for release: <https://testng.org/testng-eclipse-update-site>.

Installing TestNG plugin in Eclipse

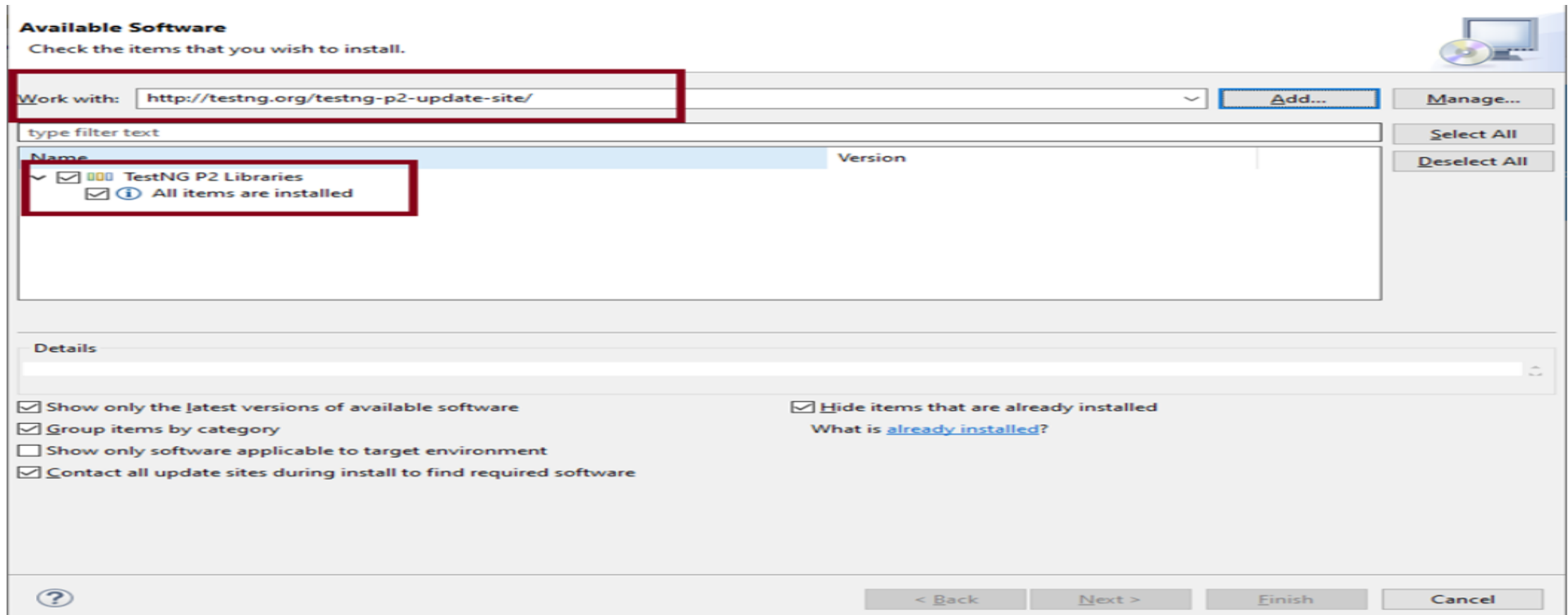
Download and Installing



Installing TestNG plugin in Eclipse

Download and Installing

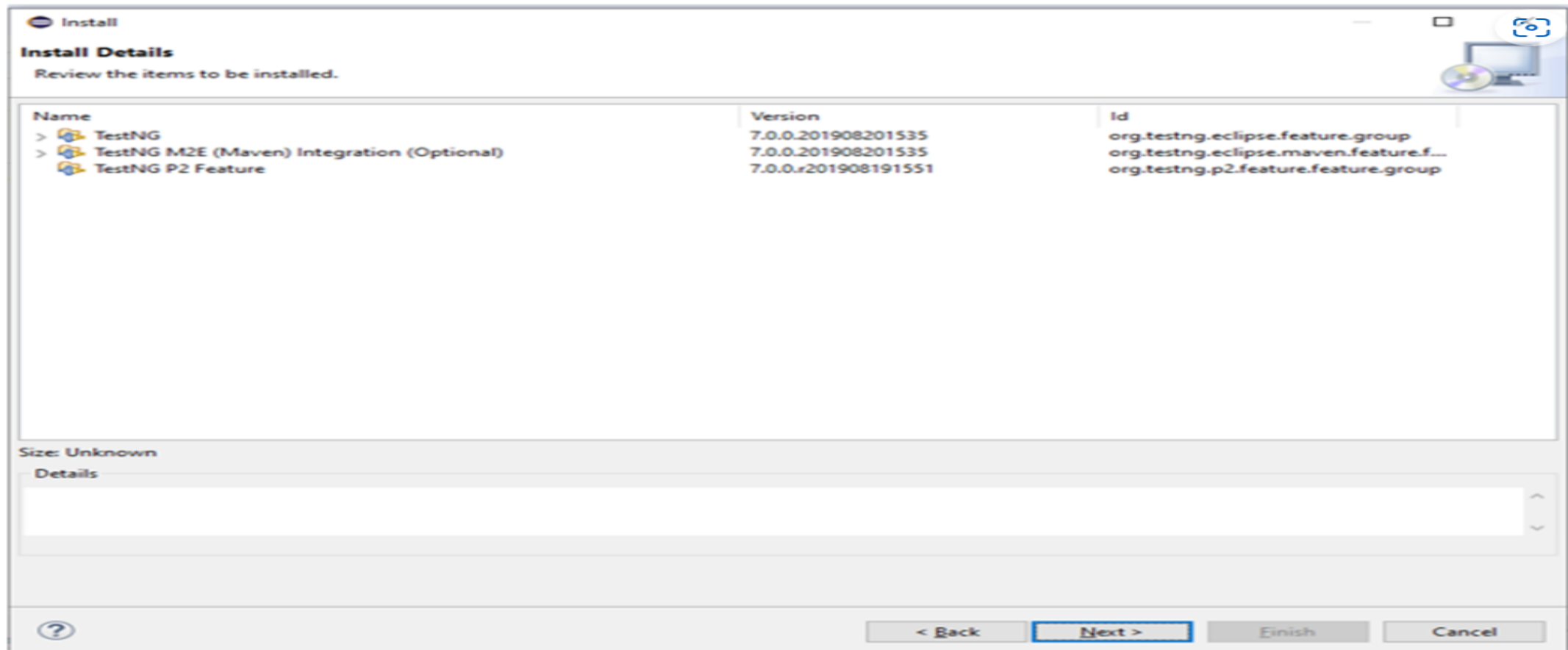
4) This time we will see TestNG is added to Install dialog box.



Installing TestNG plugin in Eclipse

Download and Installing

5) Accept the terms and conditions and then click the “Finish” button.



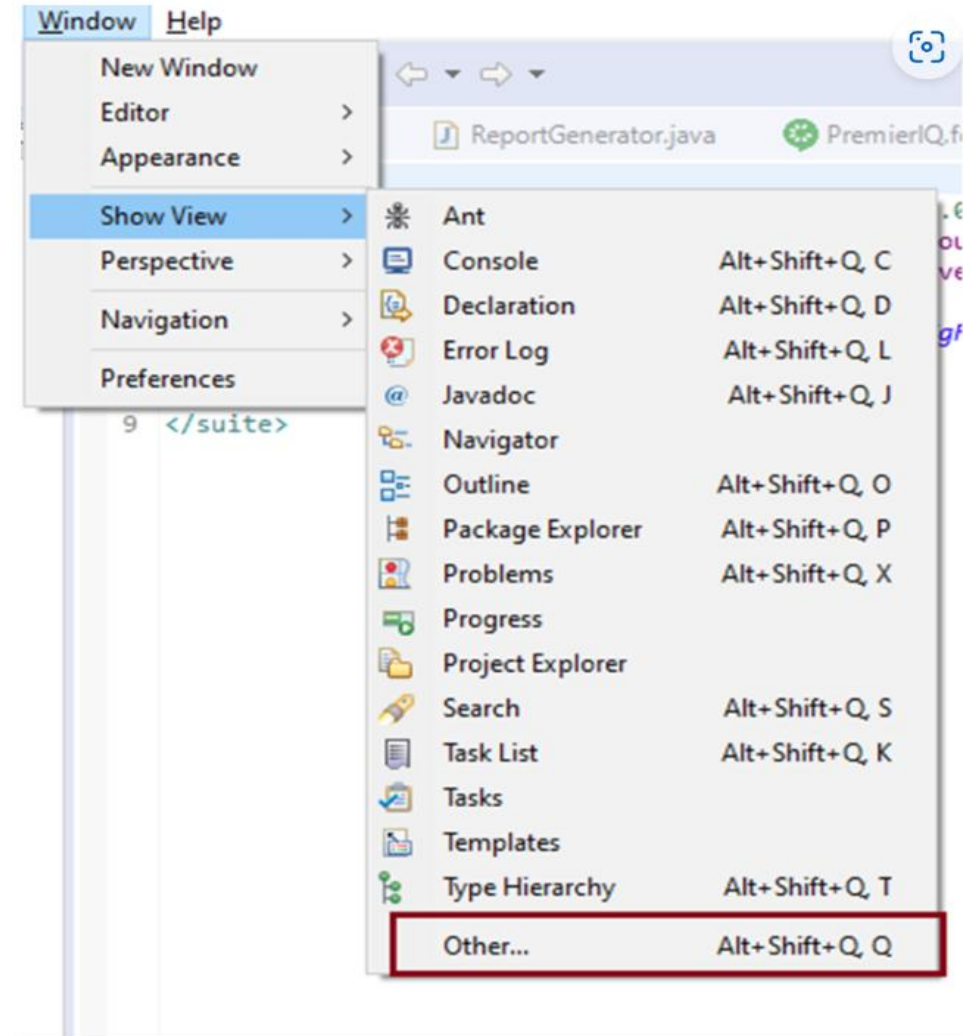
Installing TestNG plugin in Eclipse

Download and Installing

6) Once the installation is completed, you will get a message to **Restart the Eclipse**.

Select **Restart** the Eclipse

7) To verify if TestNG is installed successfully or not, go to **Window**, select **Show View**, and then **Other**.

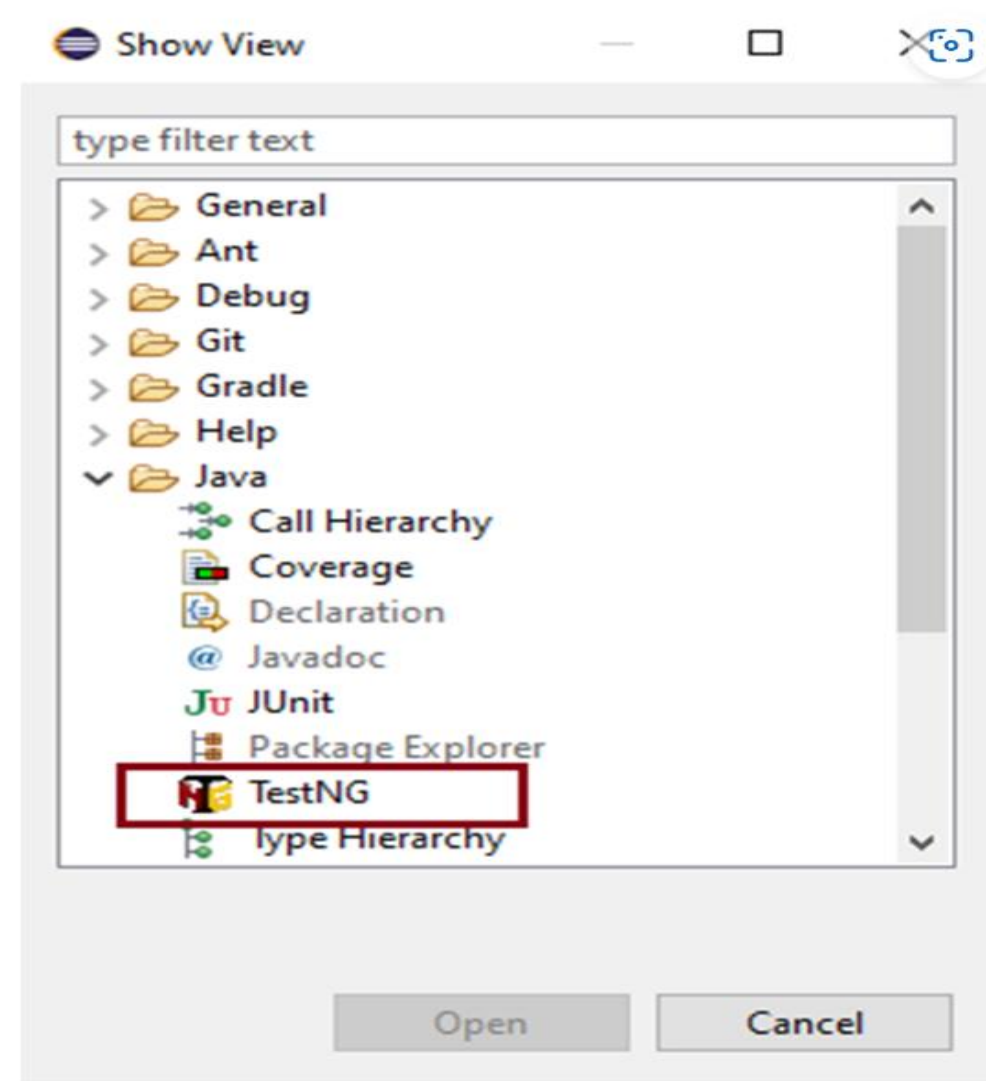


Installing TestNG plugin in Eclipse

Download and Installing

8) **Select Java and see, within the Java folder, you will see TestNG.**

This shows that TestNG is successfully installed on the machine.



Creating TestNG Class

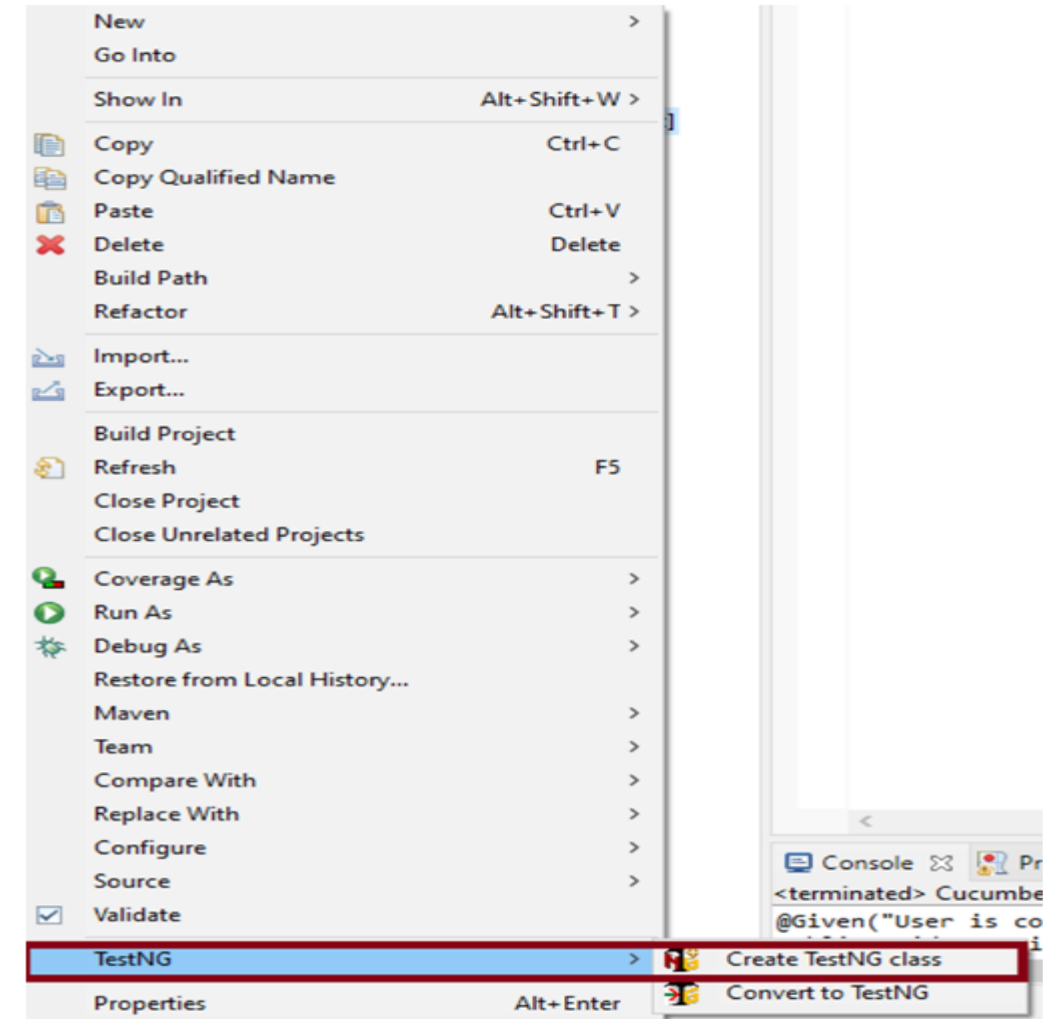


Creating TestNG Class

Steps to follow to create a TestNG class

1) Create a new TestNG class.

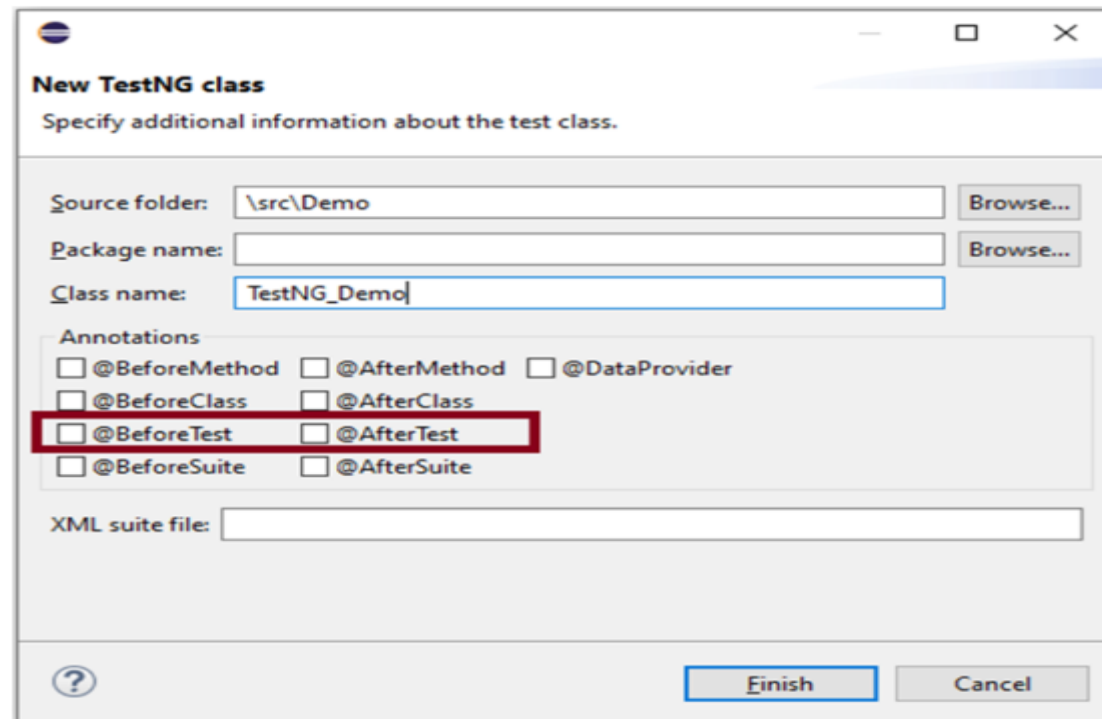
Right-click on the Folder where you want to create the TestNG class. Select TestNG and then Create the TestNG class as shown in the below image.



Creating TestNG Class

Steps to follow to create a TestNG class

2) In the below image, we can see that the Source folder is the name of the folder we want to create the class, and we can mention the name of the class in the Class name. Under annotations, I have checked **@BeforeTest** and **@AfterTest** and click the **Finish** button.



New TestNG class
Specify additional information about the test class.

Source folder: \src\Demo Browse...

Package name: Browse...

Class name: TestNG_Demo

Annotations

☐ @BeforeMethod ☐ @AfterMethod ☐ @DataProvider

☐ @BeforeClass ☐ @AfterClass

☒ @BeforeTest ☒ @AfterTest

☐ @BeforeSuite ☐ @AfterSuite

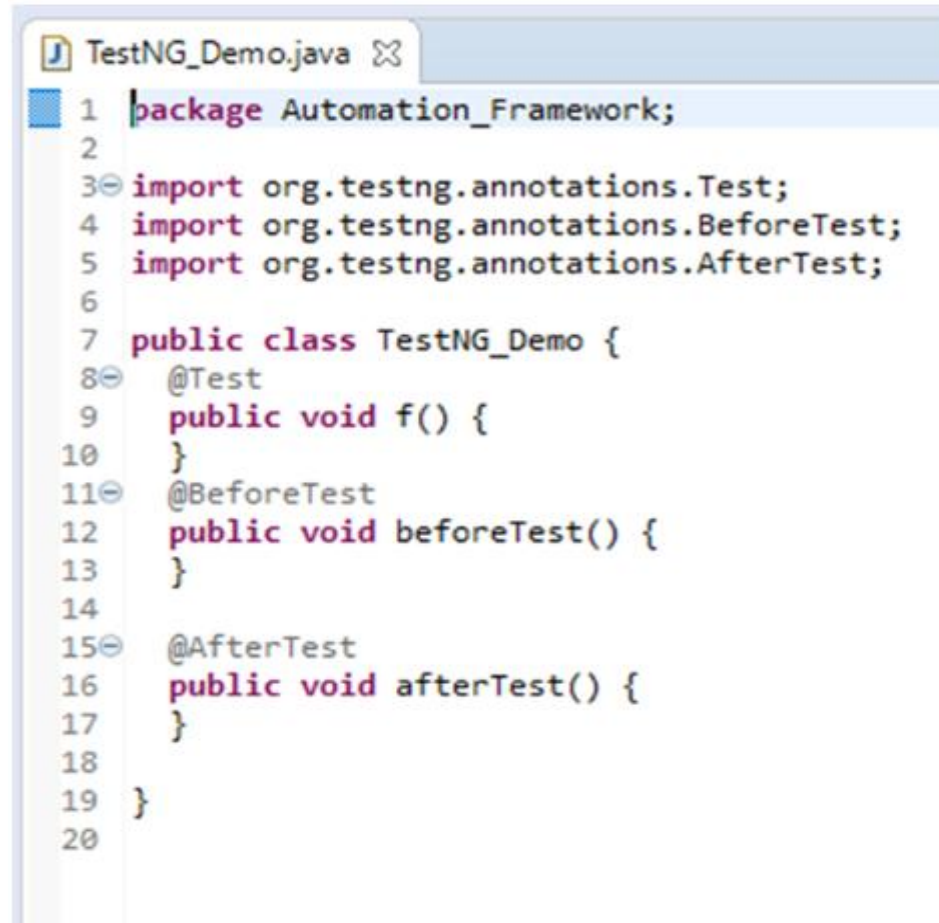
XML suite file:

? Finish Cancel

Creating TestNG Class

Steps to follow to create a TestNG class

3) We can see that the structure of the new TestNG class looks like as shown below.



```
1 package Automation_Framework;
2
3 import org.testng.annotations.Test;
4 import org.testng.annotations.BeforeTest;
5 import org.testng.annotations.AfterTest;
6
7 public class TestNG_Demo {
8     @Test
9     public void f() {
10    }
11     @BeforeTest
12     public void beforeTest() {
13    }
14
15     @AfterTest
16     public void afterTest() {
17    }
18
19 }
20
```


Creating TestNG Class

Steps to follow to create a TestNG class

4) In the below example, we want to navigate to an Amazon page and search for Hard Drive.

@BeforeTest : Launch google chrome and direct it to the Base URL

@Test : Search for HardDrive

@AfterTest : Close Chrome browser

Creating TestNG Class

Sample testNG Program

```
package com.testng;
import org.testng.annotations.Test;
import org.testng.annotations.BeforeTest;
import java.time.Duration;
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.chrome.ChromeOptions;
import org.testng.annotations.AfterTest;
public class testNGDemo {
public WebDriver driver;
    @BeforeTest
    public void beforeTest() {
        ChromeOptions options = new ChromeOptions();
        driver = new ChromeDriver(options);
        driver.manage().window().maximize();
        driver.manage().timeouts().implicitlyWait(Duration.ofSeconds(3));
        driver.get("https://www.amazon.com/");
    }
}
```

Creating TestNG Class

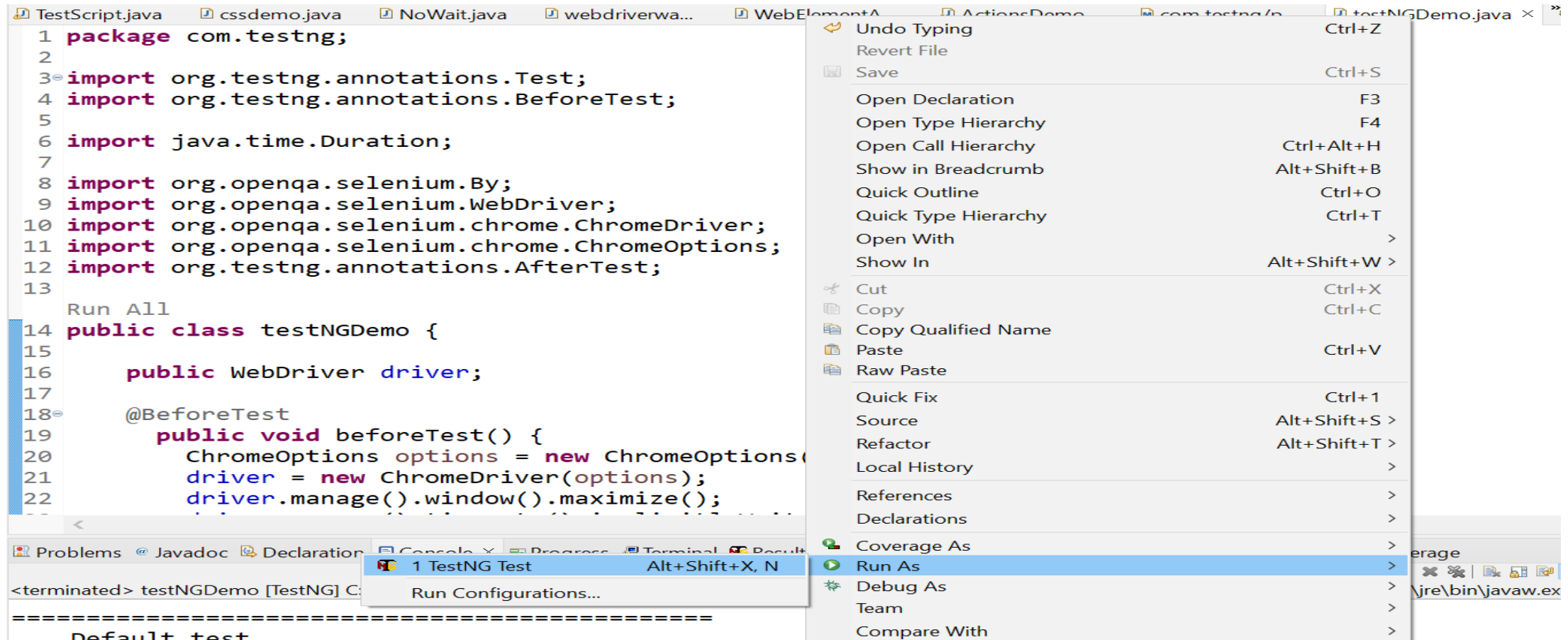
Sample testNG Program

```
@Test
public void validation() {
driver.findElement(By.xpath("//*[@id='twotabsearchtextbox']")).sendKeys("hard drive");
//XPath for search button
driver.findElement(By.id("nav-search-submit-button")).click();
}
@AfterTest
public void afterTest() {
driver.quit();
}
}
```

Creating TestNG Class

Steps to follow to create a TestNG class

5) To execute this program, we need to Right-click and select Run as – TestNG Test.



The screenshot shows an IDE with a Java file named `testNGDemo.java`. The code defines a package `com.testng`, imports necessary TestNG and Selenium classes, and contains a `testNGDemo` class with a `beforeTest` method. A right-click context menu is open over the code, showing various actions. The `Run As` option is highlighted, and a submenu is visible showing `1 TestNG Test` as the selected option. The bottom status bar indicates the test is running.

```

1 package com.testng;
2
3 import org.testng.annotations.Test;
4 import org.testng.annotations.BeforeTest;
5
6 import java.time.Duration;
7
8 import org.openqa.selenium.By;
9 import org.openqa.selenium.WebDriver;
10 import org.openqa.selenium.chrome.ChromeDriver;
11 import org.openqa.selenium.chrome.ChromeOptions;
12 import org.testng.annotations.AfterTest;
13
14 Run All
15 public class testNGDemo {
16     public WebDriver driver;
17
18     @BeforeTest
19     public void beforeTest() {
20         ChromeOptions options = new ChromeOptions();
21         driver = new ChromeDriver(options);
22         driver.manage().window().maximize();
23     }
24 }

```

Context Menu Options:

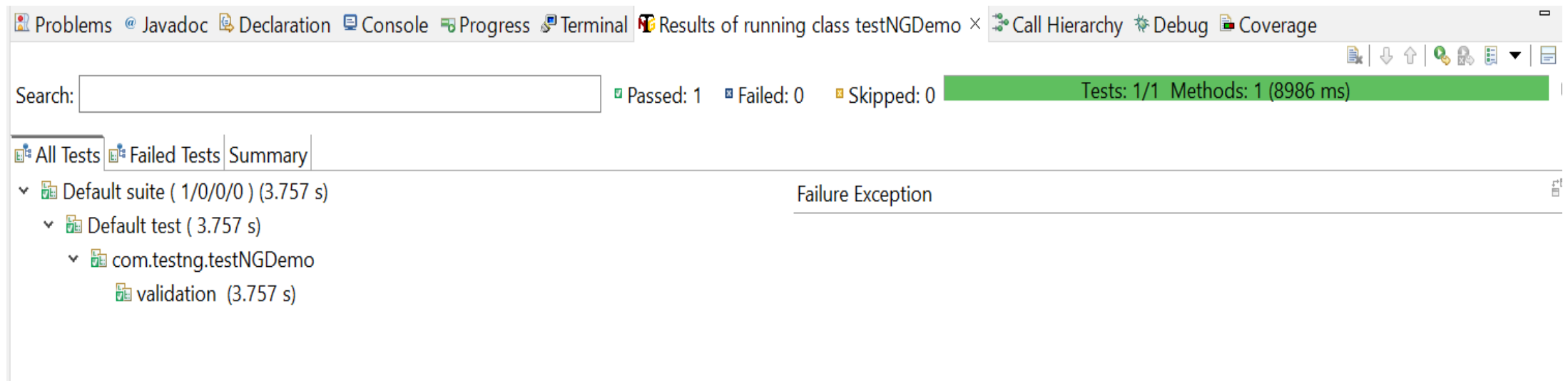
- Undo Typing (Ctrl+Z)
- Revert File
- Save (Ctrl+S)
- Open Declaration (F3)
- Open Type Hierarchy (F4)
- Open Call Hierarchy (Ctrl+Alt+H)
- Show in Breadcrumb (Alt+Shift+B)
- Quick Outline (Ctrl+O)
- Quick Type Hierarchy (Ctrl+T)
- Open With (>)
- Show In (Alt+Shift+W >)
- Cut (Ctrl+X)
- Copy (Ctrl+C)
- Copy Qualified Name
- Paste (Ctrl+V)
- Raw Paste
- Quick Fix (Ctrl+1)
- Source (Alt+Shift+S >)
- Refactor (Alt+Shift+T >)
- Local History (>)
- References (>)
- Declarations (>)
- Coverage As (>)
- Run As (>)**
 - 1 TestNG Test (Alt+Shift+X, N)**
 - Run Configurations...
- Debug As (>)
- Team (>)
- Compare With (>)

Bottom Status Bar: <terminated> testNGDemo [TestNG] C: \jre\bin\javaw.exe

Creating TestNG Class

Steps to follow to create a TestNG class

6) The result will look like something shown below. Here, we can see that Test Case Passed is 1, Failed 0, and Skipped 0.



The screenshot displays the TestNG results window in an IDE. The top bar shows the test results summary: **Passed: 1**, **Failed: 0**, **Skipped: 0**, and **Tests: 1/1 Methods: 1 (8986 ms)**. Below this, the test hierarchy is shown in a tree view:

- Default suite (1/0/0/0) (3.757 s)
 - Default test (3.757 s)
 - com.testng.testNGDemo
 - validation (3.757 s)

The right pane shows the **Failure Exception** for the failed test case.

Creating TestNG Class

Steps to follow to create a TestNG class

7) As we know that TestNG also produce HTML Reports. To access the report, go to the Eclipse folder, and you can see a folder with name test-output inside the Project where we have created TestNG class.

TestNG Annotations



TestNG Annotations

TestNG Annotations

- Annotations from testNG are mainly used to control the flow of executions of tests.
- TestNG Annotations are used to control the next method to be executed in the test script.
- TestNG annotations are defined before every method in the test code.
- In case any method is not prefixed with annotations, it will be ignored and not be executed as part of the test code.

Hierarchy of the TestNG Annotations

@BeforeSuite

@BeforeTest

@BeforeClass

@BeforeMethod

@Test

@AfterMethod

@AfterClass

@AfterTest

@AfterSuite

Types of TestNG Annotations

Types of TestNG Annotations

- **@Test** – This marks a class or a method as a part of the test.
- **@BeforeSuite** – The @BeforeSuite method in TestNG runs before the execution of all other test methods. It will run only once before all tests in the suite have to run.
- **@AfterSuite** – The @AfterSuite method in TestNG runs after the execution of all other test methods. It will run only once after all tests in the suite have run.
- **@BeforeTest** – The @BeforeTest method in TestNG runs before the execution of first @Test method. It runs only once per class.
- **@AfterTest** – The @AfterTest method in TestNG executes after the execution of all the test methods that are inside that folder.

Types of TestNG Annotations

Types of TestNG Annotations

- **@BeforeClass** – The @BeforeClass method in TestNG will run before the first method invokes of the current class.
- **@AfterClass** – The @AfterClass method in TestNG will execute after all the test methods of the current class execute.
- **@BeforeMethod** – The @BeforeMethod method in TestNG will execute before each test method.
- **@AfterMethod** – The @AfterMethod method in TestNG will run after each test method is executed.

Types of TestNG Annotations

Types of TestNG Annotations

- **@BeforeGroups** – The @BeforeGroups method in TestNG run before the test cases of that group execute. It executes just once.
- **@AfterGroups** – The @AfterGroups method in TestNG run after the test cases of that group execute. It executes only once.
- **@Parameters** – This annotation is used to pass parameters to test methods.

Types of TestNG Annotations

Types of TestNG Annotations

- **@DataProvider** – If we use @DataProvider annotation for any method that means you are using that method as a data supplier. The configuration of @DataProvider annotated method must be like it always return Object[][] which we can use in @Test annotated method. The @Test method that wants to receive data from this DataProvider needs to use a dataProvider name equals to the name of this annotation.
- **@Factory** – Marks a method as a factory that returns objects that will be used by TestNG as Test classes. The method must return Object[].
- **@Listeners**– This annotation is used with test class. It helps in writing logs and results.

Types of TestNG Annotations

@Test Annotation

- **@Test** is the core annotation in TestNG — it marks a method as a test case that will be executed by the TestNG framework. Without it, TestNG won't recognize your method as a test.
- **Syntax:**

```
@Test  
  
public void testMethodName() {  
  
    // test steps  
  
}
```

Types of TestNG Annotations

@Test Annotation

Purpose of @Test: The @Test annotation is used to:

- Define individual test methods.
- Control execution order using attributes (like priority, dependsOnMethods).
- Assign tests to groups (e.g., Smoke, Regression).
- Handle data-driven testing through @DataProvider.
- Enable or disable tests easily (enabled attribute).
- Specify timeouts, retry logic, expected exceptions, etc.

Types of TestNG Annotations

@Test Annotation

Attribute	Purpose	Example
priority	Defines order of execution (lowest runs first).	<code>@Test (priority=1)</code>
groups	Groups related tests for selective execution.	<code>@Test (groups={ "smoke" })</code>
dependsOnMethods	Runs test only if another test passes.	<code>@Test (dependsOnMethods="loginTest")</code>
dataProvider	Runs the test multiple times with different data.	<code>@Test (dataProvider="loginData")</code>
enabled	Skips or includes a test.	<code>@Test (enabled=false)</code>
invocationCount	Runs same test multiple times.	<code>@Test (invocationCount=3)</code>
timeOut	Fails test if it exceeds specified time (in ms).	<code>@Test (timeOut=5000)</code>
expectedExceptions	Marks test as passed if an exception occurs.	<code>@Test (expectedExceptions=NoSuchElementException.class)</code>

Types of TestNG Annotations

TestNG Annotations — what they do & when to use them

@BeforeSuite / @AfterSuite

- Run once before/after the entire test suite.
- Use to: start/stop grid, create test reports folder, global test data load, or initialize DB connections.

Example: Demo use (Demoblaze):

- @BeforeSuite: create results directory, initialize remote WebDriver grid (if using).
- @AfterSuite: send consolidated report email (optional) or teardown grid.

Types of TestNG Annotations

TestNG annotations — what they do & when to use them

@BeforeTest / @AfterTest

- Run before/after <test> tag in testng.xml. Useful for per-environment setup (e.g., set base URL, browser type).
- Use to: set test-level config or environment parameters.

Demo use: Read browser parameter from testng.xml and set it here.

@BeforeClass / @AfterClass

- Run once before/after all test methods in a given test class.
- Use to: create page objects, open a stable browser session for tests in that class.

Demo use (LoginTests.java): Open browser, navigate to Demoblaze once per class.

Types of TestNG Annotations

TestNG annotations — what they do & when to use them

@BeforeMethod / @AfterMethod

- Run before/after every @Test method.
- Use for per-test setup/cleanup: ensure each test starts from a clean state (e.g., log out existing user, clear cart, reset DB).

Demo use:

- @BeforeMethod: ensure we are on Home page and logged out.
- @AfterMethod: take screenshot on failure (or run cleanup like removing items from cart).

Sample TestNG Annotations Program

```
import org.testng.annotations.*;

public class AnnotationsExample {

    @BeforeTest
    public void init(){
        System.out.println("----- Before Test ----- ");
        //open the browser
    }

    @AfterTest
    public void quit(){
        System.out.println("----- After Test ----- ");
        //close the browser
    }

    @BeforeMethod
    public void beforeMethod() {
        System.out.println("##### Before Method ##### ");
        //Initialize the Report
    }
}
```

TestNG and its Types

Sample TestNG Annotations Program

```
@AfterMethod
public void afterMethod() {
    System.out.println("##### After Method #####");
    //Finalize the Report
}

@BeforeClass
public void beforeClass() {
    System.out.println("***** Before Class *****");
}

@AfterClass
public void afterClass() {
    System.out.println("***** After Class *****");
}

@BeforeSuite
public void beforeSuite() {
    System.out.println("This will execute Before Suite");
}
```

TestNG and its Types

Sample TestNG Annotations Program

```
@AfterSuite
public void afterSuite() {
    System.out.println("This will execute After Suite");
}

@Test
public void loginTest(){
    System.out.println("Login the application ");
}

@Test
public void logout(){
    System.out.println("Logout the application ");
}
}
```

Types of TestNG Annotations

Sample TestNG Annotations Program - output

```

Problems Javadoc Declaration Console × Progress Terminal Results of running class testNGAnnotation Call Hierarchy Debug Coverage
<terminated> testNGAnnotation [TestNG] C:\Users\Arun Kumar\.p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64_17.0.5.v20221102-0933\jre\bin\javaw.exe (21-Oct-2023, 8:52:01 pm – 8:52:03 pm) [pid: 18208]
SLF4J: Defaulting to no-operation (NOP) logger implementation
SLF4J: See http://www.slf4j.org/codes.html#StaticLoggerBinder for further details.
This will execute Before Suite
----- Before Test -----
***** Before Class *****
##### Before Method #####
Login the application
##### After Method #####
##### Before Method #####
Logout the application
##### After Method #####
***** After Class *****
----- After Test -----
PASSED: com.testng.testNGAnnotation.logout
PASSED: com.testng.testNGAnnotation.loginTest

=====
    Default test
    Tests run: 2, Failures: 0, Skips: 0
=====

This will execute After Suite

=====
Default suite
Total tests run: 2, Passes: 2, Failures: 0, Skips: 0
=====

```

Types of TestNG Annotations

Sample TestNG Annotations Program - Execution Process

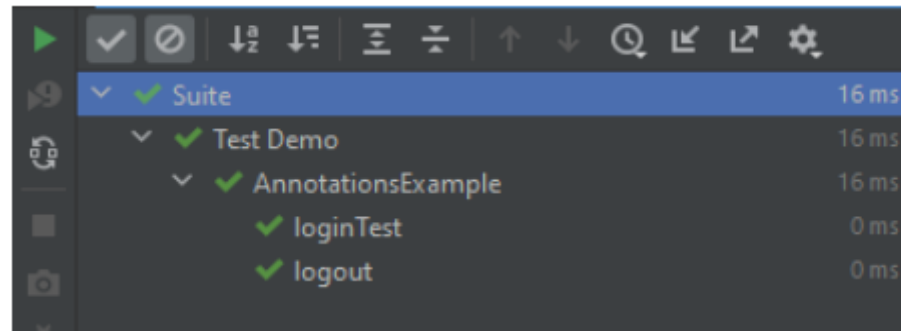
1. First of all, ***beforeSuite()*** method is executed only once.
2. Lastly, the ***afterSuite()*** method executes only once.
3. ***@BeforeMethod*** and ***@AfterMethod*** are executed twice, one for Test Case 1 and again for Test Case 2.
4. Methods ***beforeTest()***, ***beforeClass()***, ***afterClass()***, and ***afterTest()*** methods are executed only once.
5. To execute this program, we need to Right click and select ***Run as -> TestNG or Run testng.xml***.

Types of TestNG Annotations

Sample TestNG Annotations Program - Execution Process

1. First of all, ***beforeSuite()*** method is executed only once.
2. Lastly, the ***afterSuite()*** method executes only once.
3. ***@BeforeMethod*** and ***@AfterMethod*** are executed twice, one for Test Case 1 and again for Test Case 2.
4. Methods ***beforeTest()***, ***beforeClass()***, ***afterClass()***, and ***afterTest()*** methods are executed only once.
5. To execute this program, we need to Right click and select ***Run as -> TestNG or Run testng.xml***.

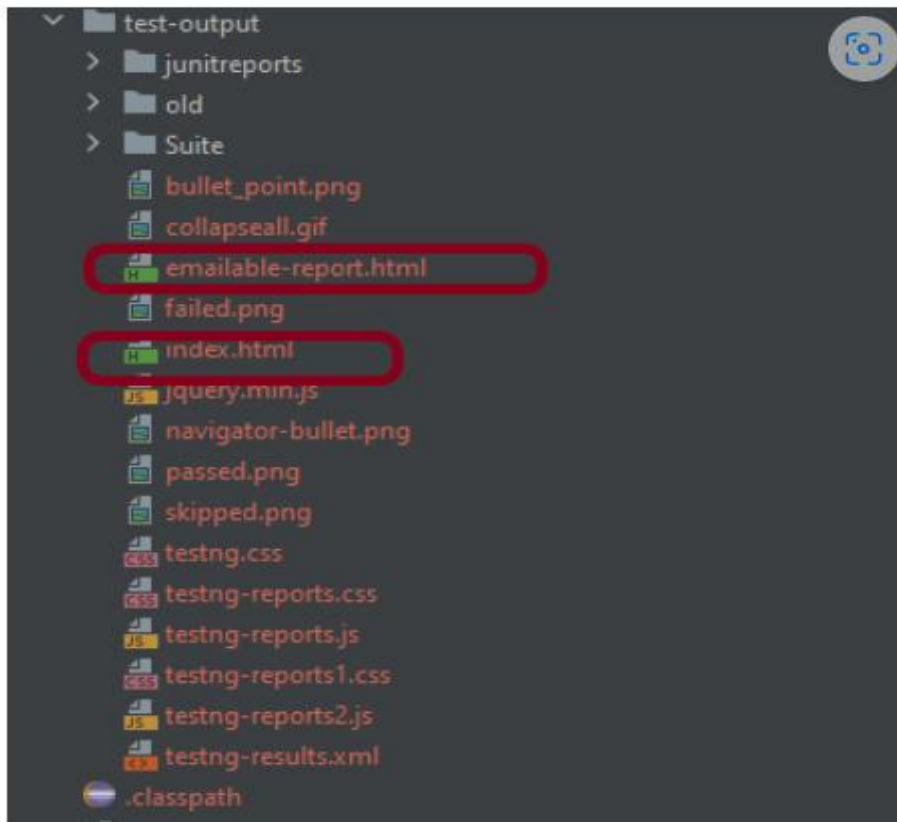
The test execution result will look like something shown below



Types of TestNG Annotations

Sample TestNG Annotations Program - Execution Process

TestNG generates various type of reports under test-output folder.



Types of TestNG Annotations

Sample TestNG Annotations Program - Execution Process

Open '**emailable-report.html**', as this is a html report open it with browser. It will look like something below.

Test	# Passed	# Skipped	# Retried	# Failed	Time (ms)	Included Groups	Excluded Groups
Suite							
Test Demo	2	0	0	0	38		

Class	Method	Start	Time (ms)
Suite			
Test Demo — passed			
TestNG_Demo.AnnotationsExample	loginTest	1648039202301	0
	logout	1648039202305	0

Test Demo

TestNG_Demo.AnnotationsExample#loginTest

[back to summary](#)

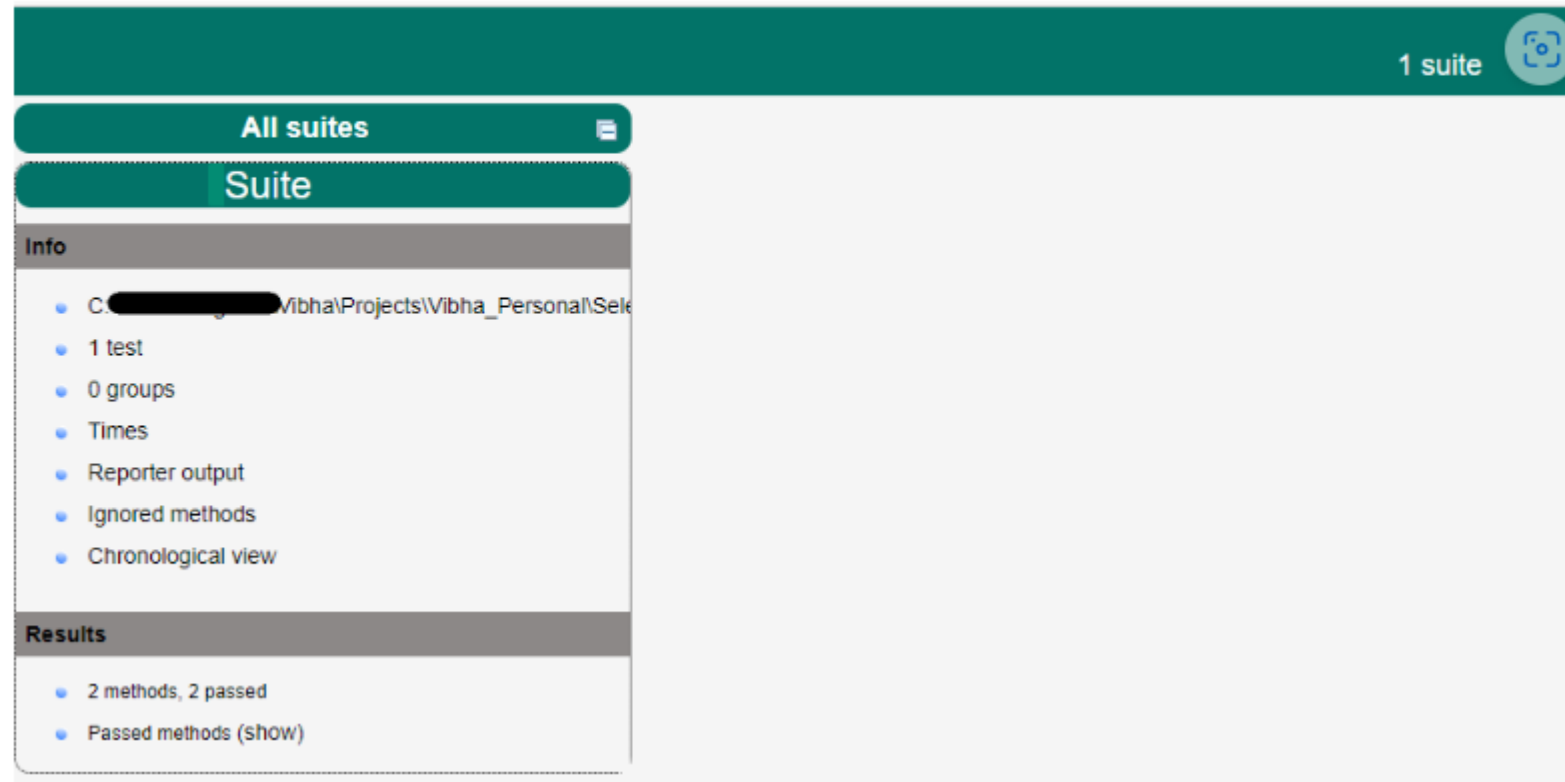
TestNG_Demo.AnnotationsExample#logout

[back to summary](#)

Types of TestNG Annotations

Sample TestNG Annotations Program - Execution Process

TestNG also produce "***index.html***" report and it resides under test-output folder.



Quiz



1) You are testing the login functionality of demoblaze.com using TestNG. You need the browser to launch before every test case and close after each test. Which annotations would best achieve this?

a) `@BeforeSuite` and `@AfterSuite`

b) `@BeforeTest` and `@AfterTest`

c) `@BeforeMethod` and `@AfterMethod`

d) `@BeforeClass` and `@AfterClass`

Answer : Option c)

Quiz



2) Your testng.xml file has multiple <test> tags for different environments (QA, Staging). You need to run environment setup before each <test>. Which annotation is suitable?

a) @BeforeSuite

b) @BeforeTest

c) @BeforeMethod

d) @BeforeClass

Answer : Option b)

Quiz



3) You are testing multiple user roles in demoblaze.com. To pass login credentials dynamically from Excel, which TestNG feature helps?

a) @Factory

b) @Parameters

c) @DataProvider

d) @BeforeTest

Answer : Option c)

Quiz



4) Your automation suite includes 10 test classes for different modules (Login, Cart, Payment, etc.). Each class uses the same browser instance to reduce overhead. You also need the browser to close only once after all modules finish execution. Where should the setup and teardown methods be placed?

a) Setup in `@BeforeClass` and teardown in `@AfterClass`

b) Setup in `@BeforeMethod` and teardown in `@AfterMethod`

c) Setup in `@BeforeTest` and teardown in `@AfterTest`

d) Setup in `@BeforeSuite` and teardown in `@AfterSuite`

Answer : Option d)

TestNG Test Priority, Disabling, Enabling and Grouping Tests



TestNG Priority Execution

- Prioritization in TestNG is a way to provide a sequence to the methods so that they do not run out of order
- Providing priority to these test cases helps us managing our tests' execution.
- The test methods with no priority assigned have a default priority equal to 0. It means if we define no priorities, all the test methods will be assigned priority 0, and a similar priority case will apply

```
public class FirstTestNGFile {
```

```
    @Test(priority = 3)
    public void c_test() {
        Assert.fail();
    }
```

the 2nd least priority value so
this will be executed 2nd

```
    @Test(priority = 0)
    public void a_test() {
        Assert.assertTrue(true);
    }
```

this has the lowest priority value
so this will be executed first

```
    @Test(priority = 7)
    public void b_test() {
        throw new SkipException("Skipping b_test...");
    }
```

largest priority value so this will
be executed last

TestNG Test Priority, Disabling, Enabling and Grouping Tests

TestNG Priority Execution (Contd.)

The following is the syntax for allocating a priority to a test case method.

```
@Test (priority = 1)
public void func(){
    //test code
}
```

Here the test method function has a priority of 1.

TestNG Priority Execution (Contd.)

Points to remember regarding priority in TestNG:

- 1) Definition of Priority in TestNG test methods can only be the @Test methods.
- 2) Lower the priority number; higher is the priority of the test case method.
- 3) Priority in TestNG contains only integer value.
- 4) The value can be negative, zero, or positive.
- 5) If a tester defines a priority in decimal in TestNG, it needs to convert first to Integer (through typecasting).
- 6) One method is allowed to have only one priority in TestNG.
- 7) Priority cannot pass through the XML files.

@enabled Annotation

- Imagine there are 100 test cases in a Regression Test Suite. We need to execute 99 test cases in a release and do not want to execute any particular test case. But we do not want to delete that test case from the Test Suite also. In this case, TestNG has a feature that allows skipping a particular test case by setting the parameters to.

@Enabled Annotation

```
1 | @Test(enabled = false)
```

To use two or more parameters in a single annotation, separate them with a comma:

```
1 | @Test(priority = 3, enabled = false)
```

To Run the TestNG program, Right-click on the program, and select **Run As TestNG Test**.

TestNG Test Priority, Disabling, Enabling and Grouping Tests

@ignore Annotation

- There is another way of skipping the tests by using the @Ignore annotation.

1. **TestNG lets you ignore all the @Test methods**
2. **In a class (or)**
3. **In a particular package (or)**
4. **In a package and all of its child packages**

```
1.      @Ignore
        @Test
        public void ThirdTest() {

            System.out.println("This is Test Case 3");

        }
```

@ignore Annotation

2. In a class (or)

```
import org.testng.annotations.Ignore;  
import org.testng.annotations.Test;  
  
@Ignore  
public class PriorityDemo {  
  
    @Test  
    public void FirstTest() {  
  
        System.out.println("This is Test Case 1");  
  
    }  
}
```

dependsOnMethods - Explanation

- Using 'dependsOnMethod' attribute within @Test annotation, we can specify the name of the parent test method on which the test should be dependent.

- Some points

```
@Test(dependsOnMethods = "FirstTest")
```

- On execution of the whole test class or test suite, the parentTest method will run before the dependentTest.
- If we run the dependentTest alone even then the parentTest method will also run that too before the dependentTest.
- In case, the parentTest method gets failed then the dependentTest method will not run and will be marked as skipped..

dependsOnMethods Example

```
public class dependson1 {  
    WebDriver driver;  
    @Test  
    public void f() {  
        WebDriverManager.chromedriver().setup();  
        driver=new ChromeDriver();  
        driver.get("https://google.com");  
    }  
    @Test(dependsOnMethods = {"f"})  
    public void f1() {  
        driver.findElement(By.name("q1")).sendKeys("Depends on",Keys.ENTER);  
    }  
  
    @Test(dependsOnMethods = {"f","f1"})  
    public void f2() {  
        driver.quit();  
    }  
}
```

Testng Groups

- Groups in TestNG denotes the process of grouping different tests together into a straightforward group and running these tests together by just running the group in a single command.
- It does not even matter if they belong to different classes.
- As an example, let's say you have a hundred tests of a class A and in it ten methods of front-end design, ten methods of functional tests, and so on.
- You probably like to run all the front-end tests together in a batch.
- And you want all of them to be in a single test suite. With the help of grouping, you can easily overcome this situation by including all the front-end tests into one group.

TestNG Groups Example - An example XML file for Groups formatting

```
<!DOCTYPE suite SYSTEM "https://testng.org/testng-1.0.dtd" >
<suite name="Test-Suite" >
  <test name="TQA" >
    <groups>
      <run>
        <include name = "demo"></include>
        <include name = "demo1"></include>
      </run>
    </groups>
    <classes>
      <class name="TestNG" />
      <class name="TestNG1" />
    </classes>
  </test>
</suite>
```

The XML Suite file executes only the tests which are in Group name *demo, demo1*.

TestNG Groups Example

```
public class TestNG {  
    WebDriver driver = new ChromeDriver();  
    driver.manage().timeouts().implicitlyWait(1, TimeUnit.SECONDS);  
  
    @BeforeTest  
    public void starting_point(){  
        System.out.println("This is the starting point of the test");  
        driver.get("https://demoqa.com/");  
    }  
  
    @Test(groups = {"demo" })  
    public void checkTitle() {  
        String testTitle = "DEMOQA";  
        String originalTitle = driver.getTitle();  
        Assert.assertEquals(originalTitle, testTitle);  
    }  
}
```

TestNG Groups Example

```
@Test(groups = { "demo1" })
public void click_element() {
    driver.findElement(By.xpath("//img[@alt='Selenium Online Training']")).click();
    System.out.println("Selenium Training Page is displayed");
    String page_title = "Tools QA - Selenium Training"
    String page_originalTitle = driver.getTitle();
    Assert.assertEquals(page_originalTitle, page_title);
}
}
```

dependsOnGroup

- The dependsOnGroup attribute is used to make a test method dependent on a collection of tests belonging to a particular group. Thus ensuring that the dependent test will run after all the tests of the parent group are executed.
- Some points to remember about dependsOnMethod-
- On execution of the whole test class or test suite, the tests belonging to the parent group will run before the dependentTest.
- If we run the dependentTest alone even then all the tests belonging to the parent group will also run that too before the dependentTest.
- In case, anyone or all of the tests of the parent group method gets failed then the dependentTest method will not run and will be marked as skipped..

dependsOnGroups Example

```
public class TutorialExample {  
    @Test(groups = "groupA")  
    public void testMethod1ForGroupA() {  
        System.out.println("Running test method1 of groupA");  
    }  
  
    @Test(groups = "groupA")  
    public void testMethod2ForGroupA() {  
        System.out.println("Running test method2 of groupA");  
        Assert.fail();  
    }  
  
    @Test(groups = "groupB")  
    public void testMethod1ForGroupB() {  
        System.out.println("Running test method1 of groupB");  
    }  
}
```

dependsOnGroups Example

```
@Test(groups = "groupB")
    public void testMethod2ForGroupB() {
        System.out.println("Running test method2 of groupB");
    }

@Test(dependsOnGroups = "groupA")
    public void dependentTestOnGroupA() {
        System.out.println("Running the dependent test");
    }
}
```


Quiz



1) You want to execute a checkout test only after the login test has passed. Which TestNG feature should you use?

a) Priority attribute

b) dependsOnMethods attribute

c) DataProvider annotation

d) Groups attribute

Answer : Option b)

Quiz



2) A tester has marked 3 tests with priority values as 2, 1, and -1. In what sequence will TestNG execute them?

a) $1 \rightarrow 2 \rightarrow -1$

b) $-1 \rightarrow 1 \rightarrow 2$

c) $2 \rightarrow 1 \rightarrow -1$

d) Randomly

Answer : Option b)

Quiz



3) During a regression run, one test case repeatedly fails due to a known defect. You want to skip it temporarily without deleting it. Which annotation attribute would you use?

a) `@Test(enabled = false)`

b) `@Test(priority = 0)`

c) `@Ignore`

d) `@Skip`

Answer : Option a)

Quiz



4) You have 50 @Test methods, but you only want to run those belonging to "Smoke" testing. How would you achieve this?

a) Using testng.xml to define `<groups name="Smoke">`

b) Using @BeforeTest annotation

c) Using dependsOnGroups attribute

d) Commenting out unwanted tests

Answer : Option a)

Quiz



5) You have two test classes: LoginTests and CheckoutTests. You want to ensure LoginTests run before CheckoutTests. What's the best approach?

a) Add dependsOnMethods in CheckoutTests

b) Set lower priority in LoginTests

c) Define class order in testng.xml

d) Use @BeforeSuite in CheckoutTests

Answer : Option c)

Quiz



6) You created a test for “Add to Cart” and forgot to annotate it with @Test. What will happen during execution?

a) It will run before all other tests

b) It will run last

c) It will not be considered as test by TestNG

d) It will throw compile-time error

Answer : Option c)

TestNG Assertions



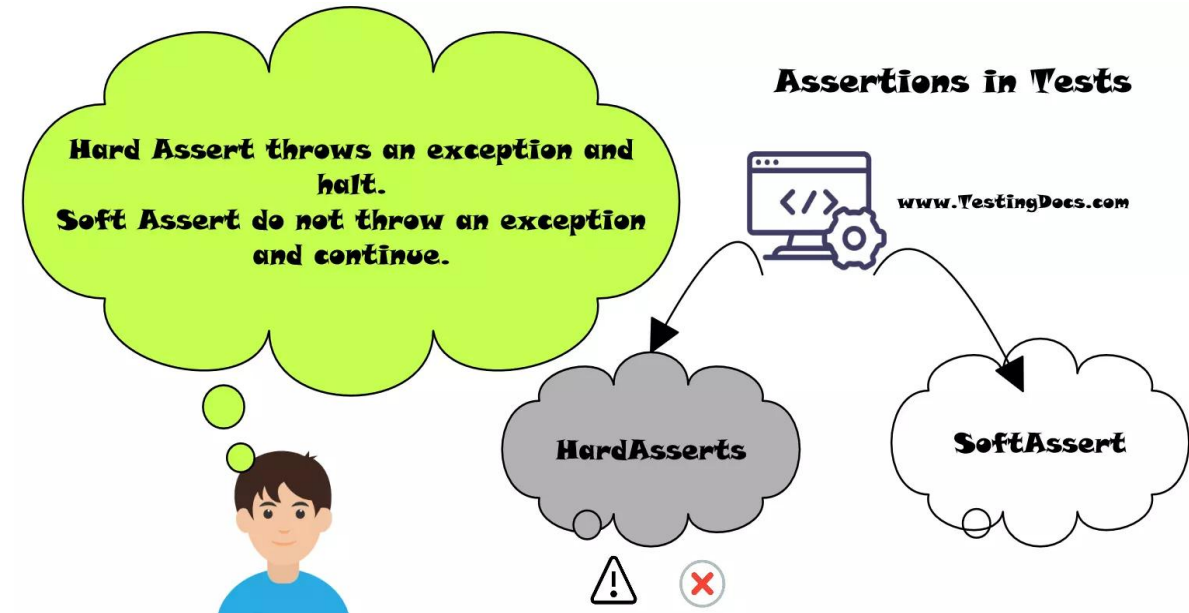
TestNG Assertions

TestNG Assertions

- TestNG asserts are the most frequently used methods in TestNG and are so common that it is hard to find a TestNG code without the asserts.
- TestNG asserts the tester decides whether the test was successful or not, along with the exceptions.

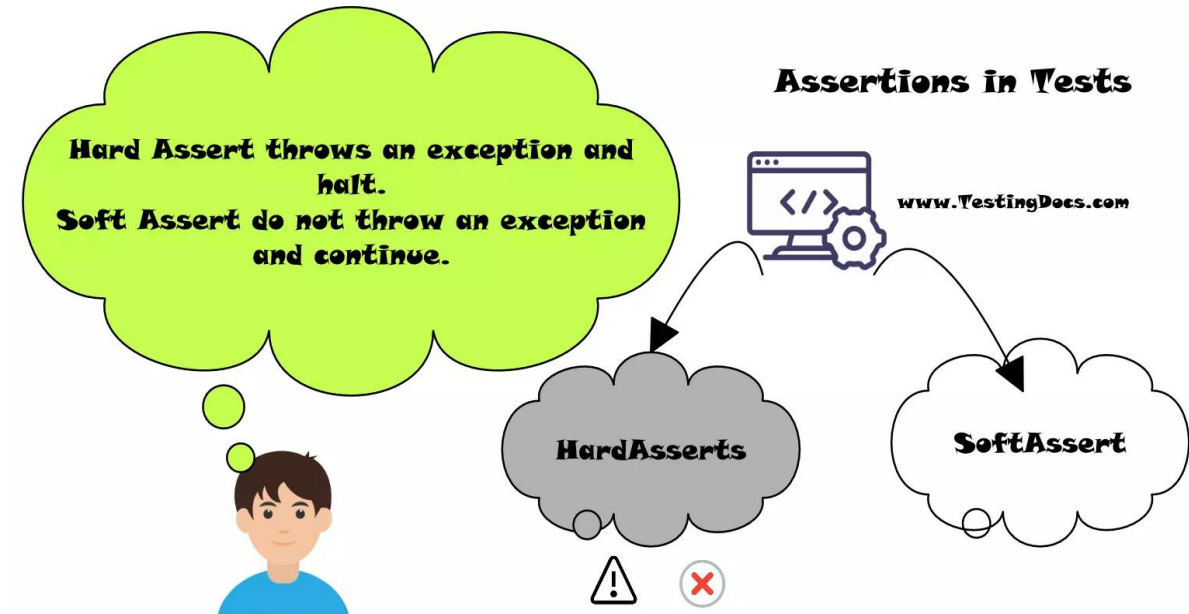
Syntax :

Assert.Method(actual, expected)



TestNG Assertions

- Assertions in TestNG are a way to verify that the expected result and the actual result matches or not in the test case.
- A test is considered successful **ONLY** if it is completed without throwing any exception.
- An example of assertion can be logging into the website, checking the title of the webpage, verifying the functionality of an input box that takes only integers, etc.



TestNG Assertions (Contd.)

TestNG Assertions Example:

Example:

```
String expectedTitle = "Facebook";  
String originalTitle = driver.getTitle();  
Assert.assertEquals(originalTitle, expectedTitle);
```

Types of TestNG Assertions

Commonly used assertions in TestNG:

1. **Assert.assertEquals(String actual, String expected)** – It takes two string arguments and checks whether both strings are equal or not. If they are not, it will fail the test and an AssertionError is thrown.
2. **Assert.assertEquals(String actual, String expected, String message)** – It takes two string arguments and checks whether both strings are equal or not. If they are not, it will fail the test and an AssertionError is thrown with message provided as argument.
3. **Assert.assertEquals(boolean actual, boolean expected)** – It takes two Boolean arguments and checks whether both are equal or not. If they are not, it will fail the test and an AssertionError is thrown.

Types of TestNG Assertions

Commonly used assertions in TestNG:

- 4. `Assert.assertEquals(java.util.Collection actual, java.util.Collection expected, java.lang.String message)`** – It takes two collection objects and verifies both collections contain the same elements and with the same order. If they are not, it will fail the test and an `AssertionError` is thrown.
- 5. `Assert.assertTrue(condition)`** – It takes one boolean argument and checks that a condition is true or not. If not, it will fail the test and an `AssertionError` is thrown.
- 6. `Assert.assertTrue(condition, message)`** – It takes one boolean argument and checks that a condition is true or not. If not, it will fail the test and an `AssertionError` is thrown with message provided as argument.

Types of TestNG Assertions

Commonly used assertions in TestNG:

7. Assert.assertFalse(condition) – It takes one boolean argument and checks that a condition is false or not. If not, it will fail the test and an AssertionError is thrown.

8. Assert.assertFalse(condition, message) – It takes one boolean argument and checks that a condition is false or not. If not, it will fail the test and an AssertionError is thrown with message provided as argument.

9. Assert.assertSame(String actual, String expected) – It asserts that two objects refer to the same object. If they do not, an AssertionError is thrown.

10. Assert.assertNotSame(String actual, String expected) – It asserts that two objects do not refer to the same object. If they do, an AssertionError is thrown.

Sample testNG Assertions Program

```
@Test
    public void testAssertions() {
        // test data
        String str1 = new String("TestNG");
        String str2 = new String("TestNG");
        String str3 = null;
        String str4 = "TestNG";
        String str5 = "TestNG";
        String str6 = new String("Not_TestNG");

        int val1 = 5;
        int val2 = 6;

        // Check that two objects are equal
        Assert.assertEquals(str1, str2);
        System.out.println("Equals Assertion is successful");

        // Check that two objects are not equal
        Assert.assertNotEquals(str1, str6);
        System.out.println("NotEquals Assertion is successful");

        // Check that a condition is true
        Assert.assertTrue(val1 < val2);
        System.out.println("True Assertion is successful");
    }
```

Sample testNG Assertions Program

```
// Check that a condition is false
Assert.assertFalse(val1 > val2);
System.out.println("False Assertion is successful");

// Check that an object isn't null
Assert.assertNotNull(str1);
System.out.println("Not Null Assertion is successful");

// Check that an object is null
Assert.assertNull(str3);

// Check if two object references point to the same object
Assert.assertSame(str4, str5);
System.out.println("Same Assertion is successful");

// Check if two object references not point to the same object
Assert.assertNotSame(str1, str3);
System.out.println("Not Same Assertion is successful");
    }
}
```

TestNG and its Types

Sample testNG Assertions Program

```
50  
51 Output  
52 Equals Assertion is successful  
53 NotEquals Assertion is successful  
54 True Assertion is successful  
55 False Assertion is successful  
56 Not Null Assertion is successful  
57 Same Assertion is successful  
58 Not Same Assertion is successful
```

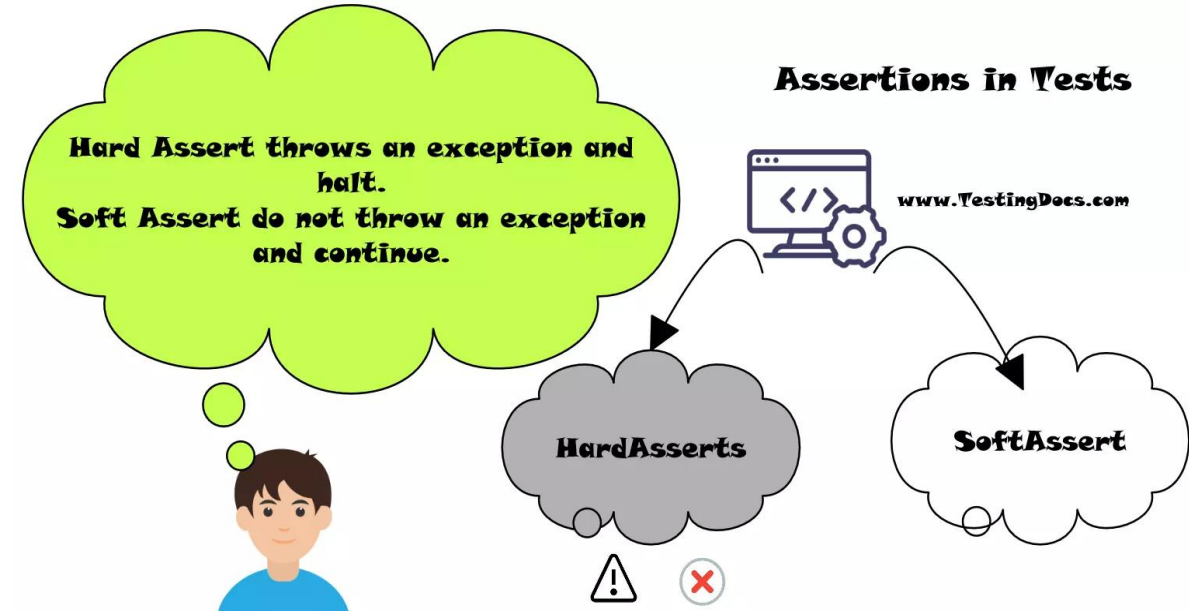

Hard Assert, Soft Assert and AssertAll



TestNG Assertions

Hard Assert and Soft Assert

- Hard Assertion throws `AssertionError` immediately when an Assert Condition fails and moves to the next `@Test` method
- Suppose, there are 2 assertions in a Test and the first assertion fails, then `HardAssertion` does not execute the second Assertion Condition and declares the test as failed.



Hard Assert and Soft Assert

- Soft Assert does not throw an exception when an Assert Condition fails, and continues with the next step after the Assert Condition.
- Soft assert does not include by default in TestNG. For this, you need to include the below package :

org.testng.asserts.SoftAssert;

- The first step is to create an instance of SoftAssert class.

```
SoftAssert softAssertion = new SoftAssert();
```

- After this, we can use this softAssert variable instead of hard assert.

```
softAssertion.assertEquals(actualTitle, expectedTitle, "Incorrect page title");
```

TestNG Assertions

AssertAll

- If there is any exception, and you want to throw it, then you need to use `assertAll()` method as a last statement in the `@Test` and test suite again to continue with the next `@Test` as it is.

```
softAssertion.assertAll();
```


Understanding TestNG XML file structure



Understanding TestNG XML file structure

testng.xml

- testng.xml is the configuration file for the TestNG framework.
- It defines how your tests should run — which tests, in what order, under what conditions, and with what parameters.
- Think of it as the master plan for TestNG execution — like a project manager that tells “who runs what.”
- It is NOT mandatory for simple tests, but essential for large projects or CI/CD pipelines.

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE suite SYSTEM "http://testng.org/testng-1.0.dtd">
<suite name="Cycle1">
  <test name="QAblepoint">
    <classes>
      <class name="Test1">
        <methods>
          <exclude name="Login.*"/>
        </methods>
      </class>
    </classes>
  </test>
</suite>
```

Purpose of testng.xml

testng.xml is the suite descriptor used by TestNG to:

- group and organize tests (suites, tests, classes)
- pass parameters to tests
- select groups to run (include/exclude)
- control parallelism and threads
- order classes/methods explicitly
- attach listeners and reporters

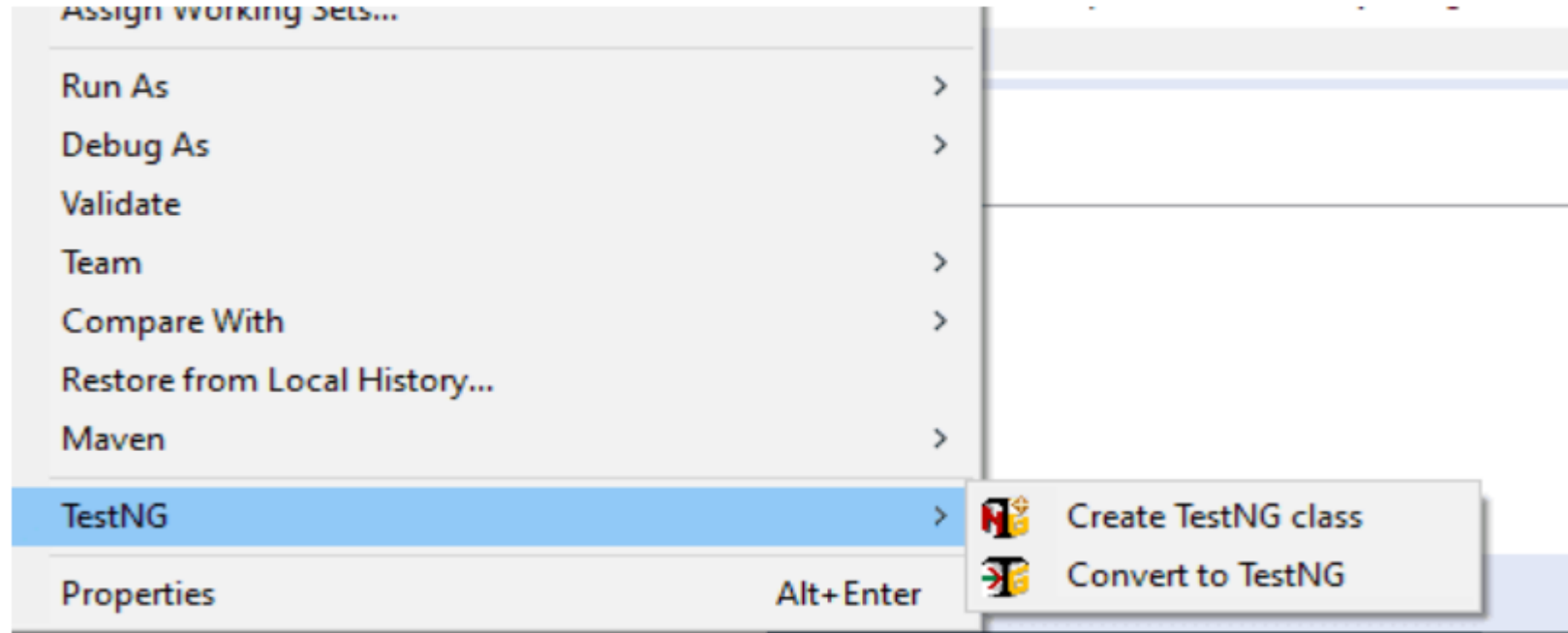


How to Create a TestNG.xml File in Eclipse?

There are two methods to create a TestNG.xml file:

Method 1:

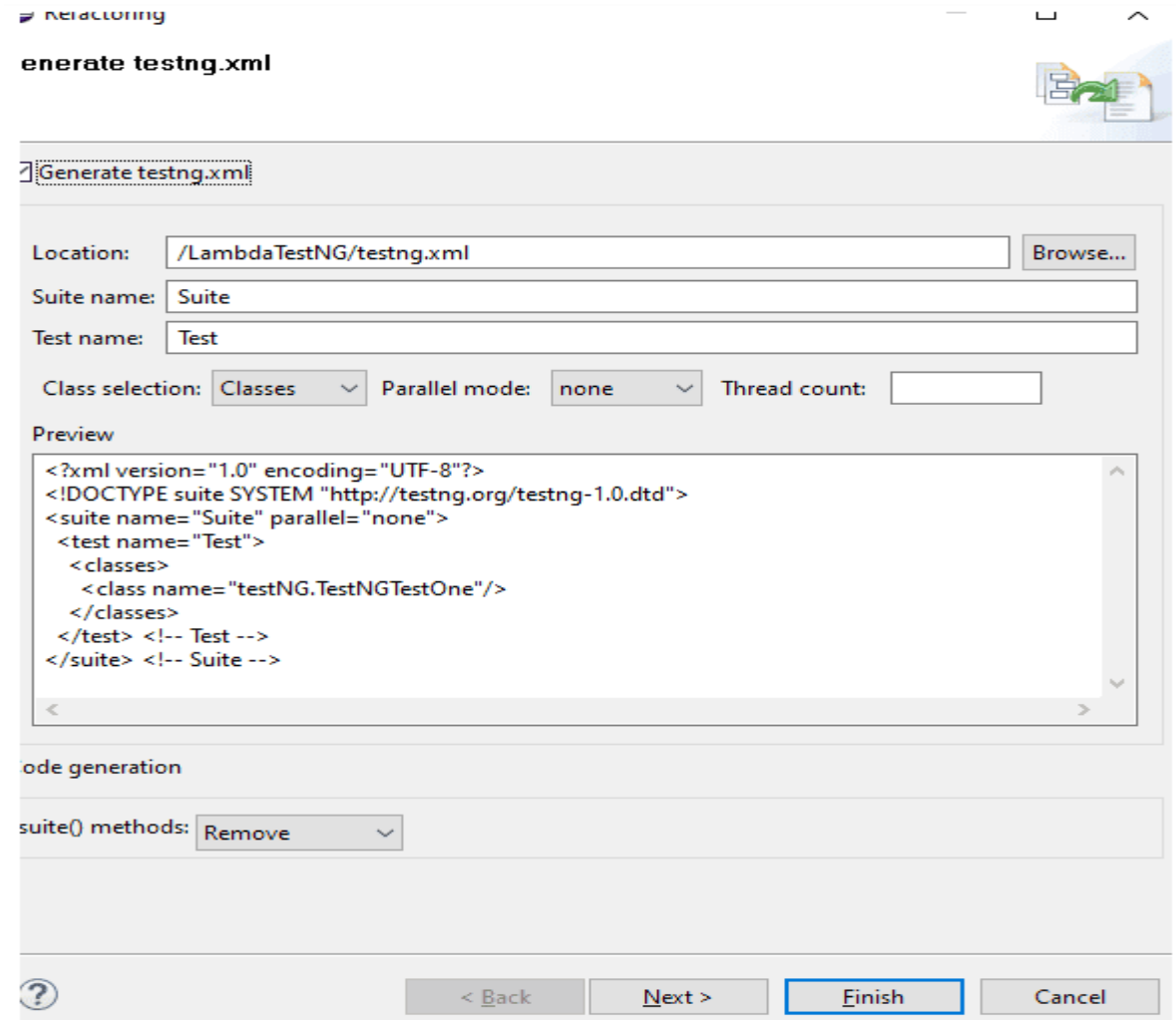
Step1: Right-click on the project folder, and towards the bottom of the list, select TestNG and then Convert to TestNG



How to Create a TestNG.xml File in Eclipse?

Method 1:

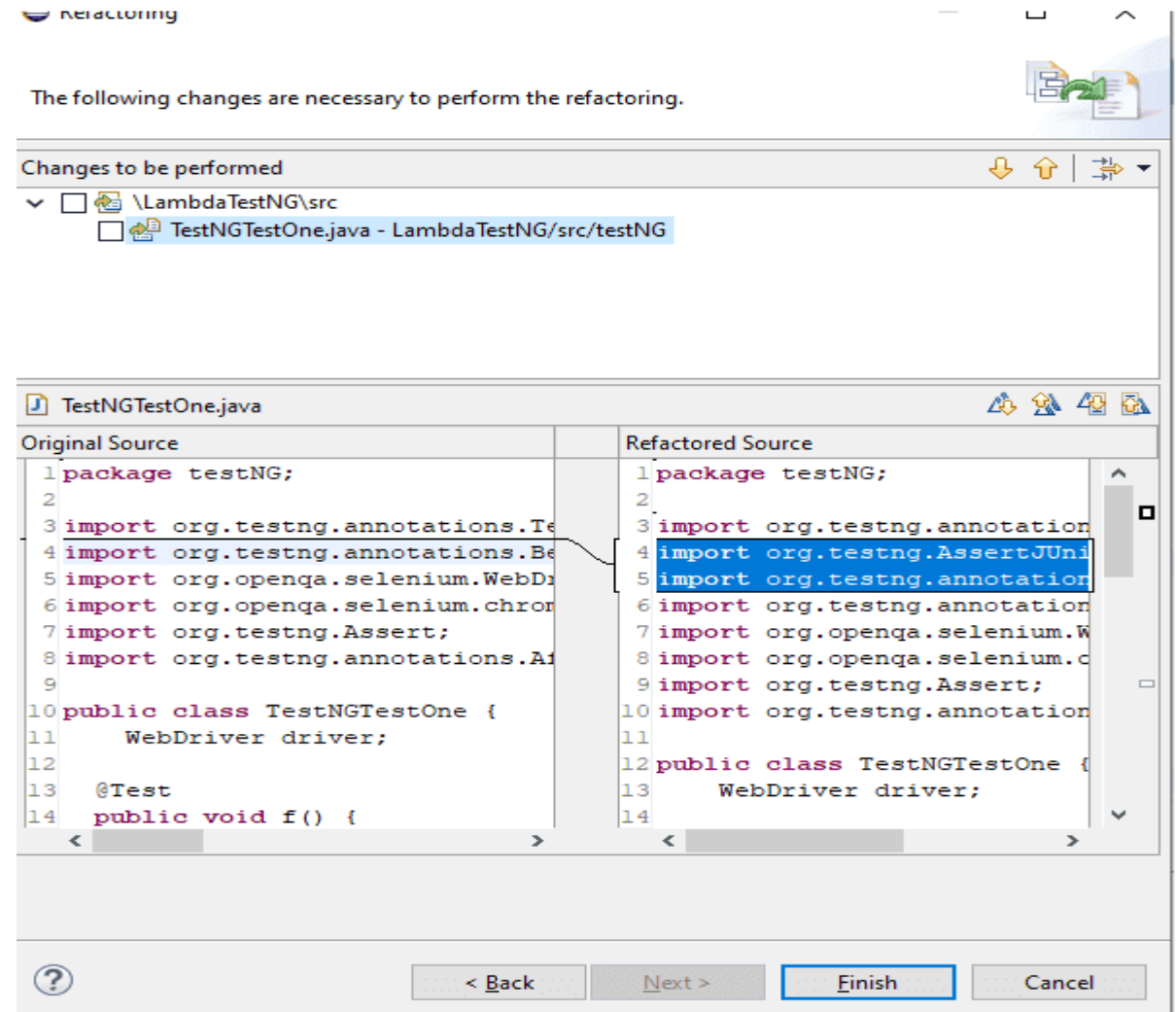
Step2: A screen like the one below would appear. Click on Next.



How to Create a TestNG.xml File in Eclipse?

Method 1:

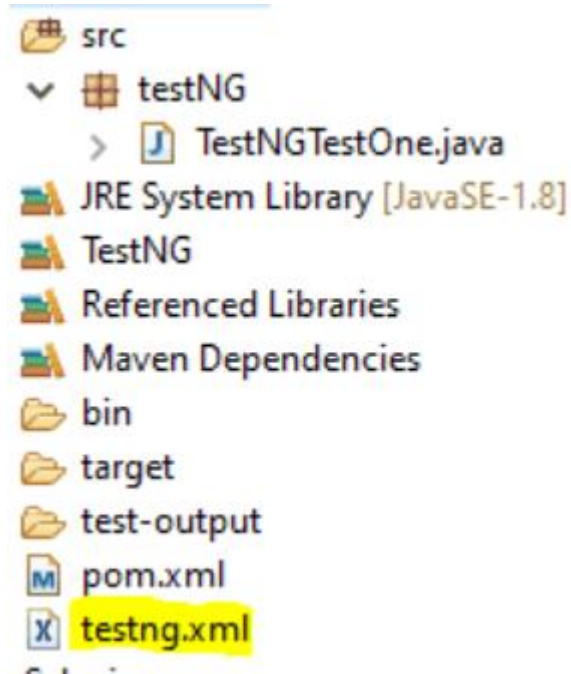
Step3: The next window that pops up will have the refactored source code after you click Finish.



How to Create a TestNG.xml File in Eclipse?

Method 1:

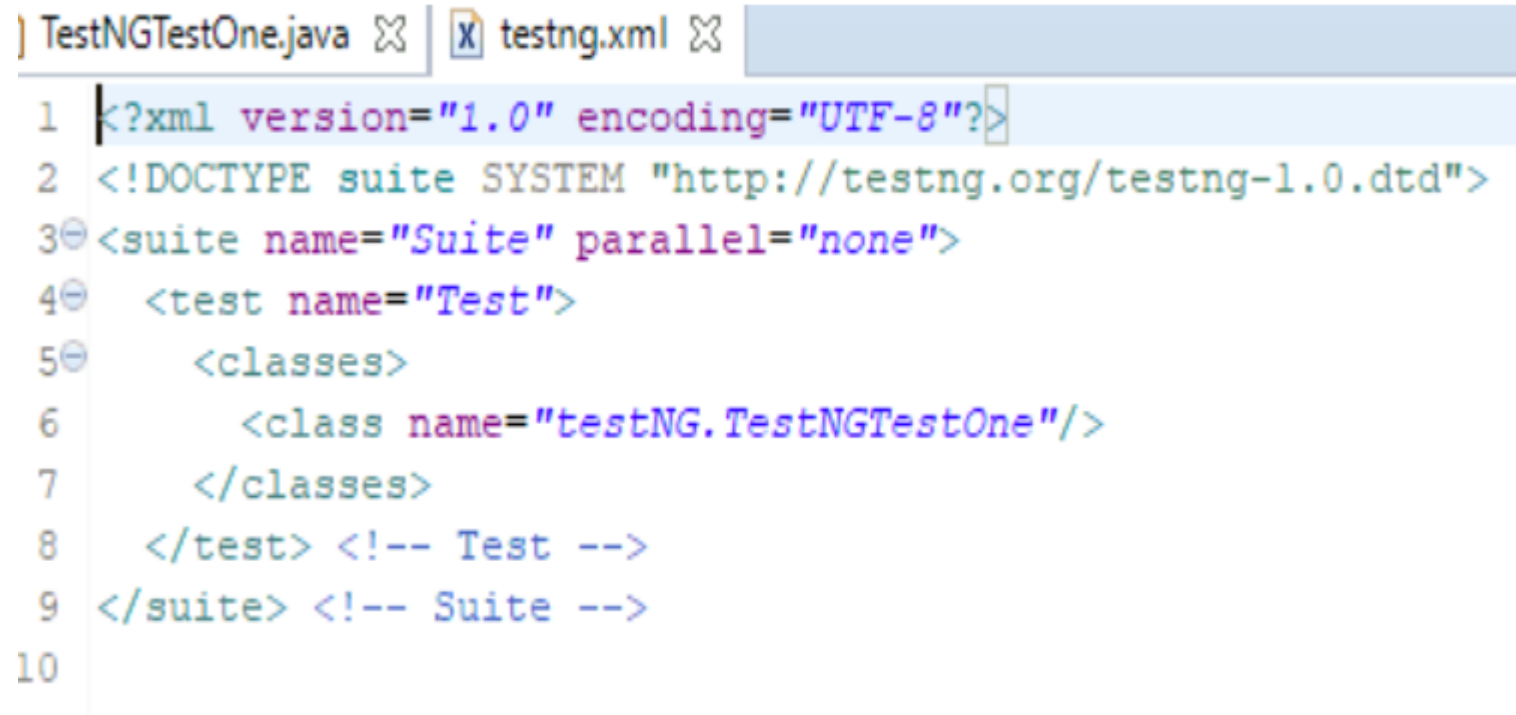
Step4: A new addition to your project directory will be displayed, named as testng.xml.



How to Create a TestNG.xml File in Eclipse?

Method 1:

Step5: Double-click on testng.xml to open the XML file, which looks like the one below.

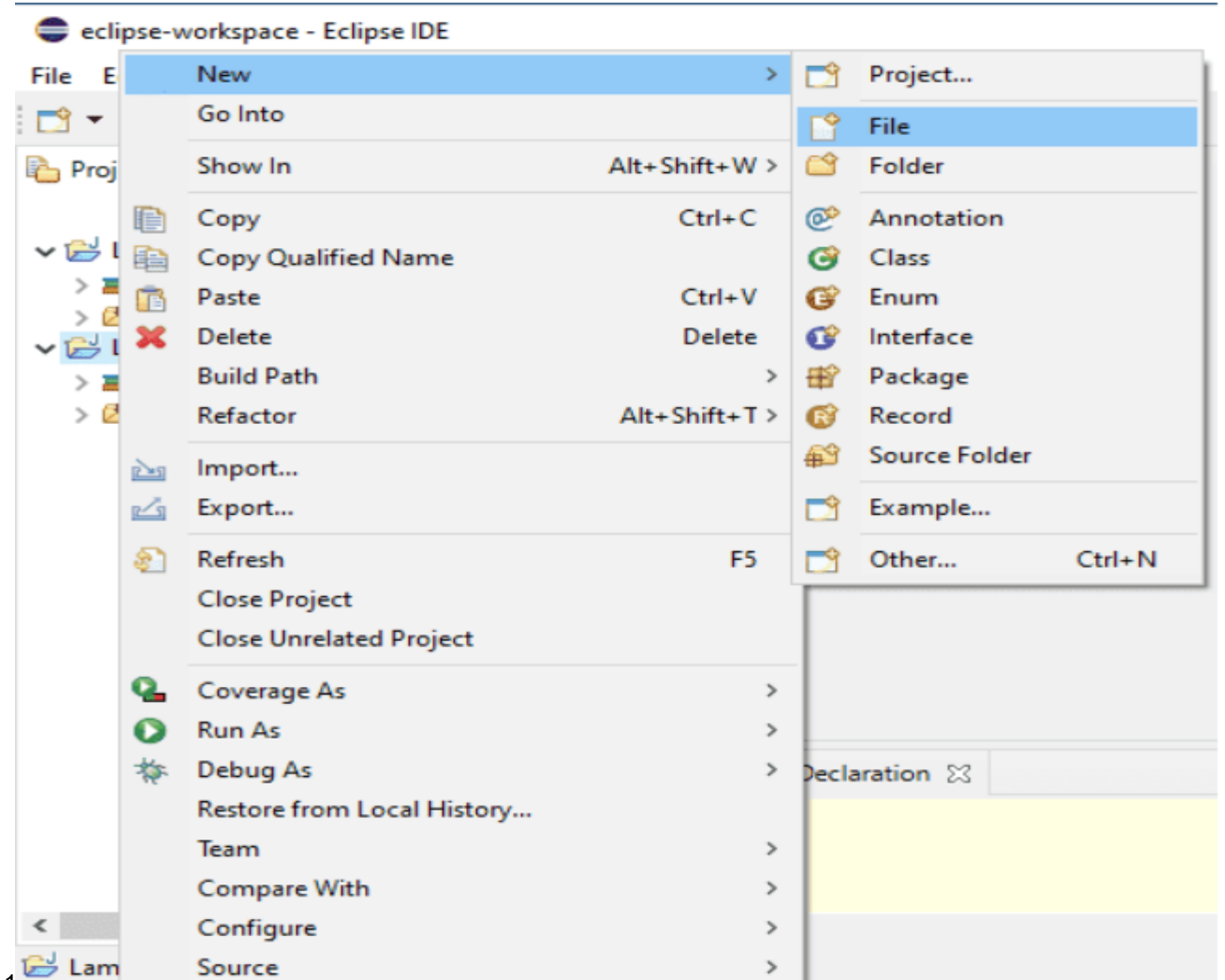


```
TestNGTestOne.java  testng.xml
1 <?xml version="1.0" encoding="UTF-8"?>
2 <!DOCTYPE suite SYSTEM "http://testng.org/testng-1.0.dtd">
3 <suite name="Suite" parallel="none">
4   <test name="Test">
5     <classes>
6       <class name="testNG.TestNGTestOne"/>
7     </classes>
8   </test> <!-- Test -->
9 </suite> <!-- Suite -->
10
```

How to Create a TestNG.xml File in Eclipse?

Method 2:

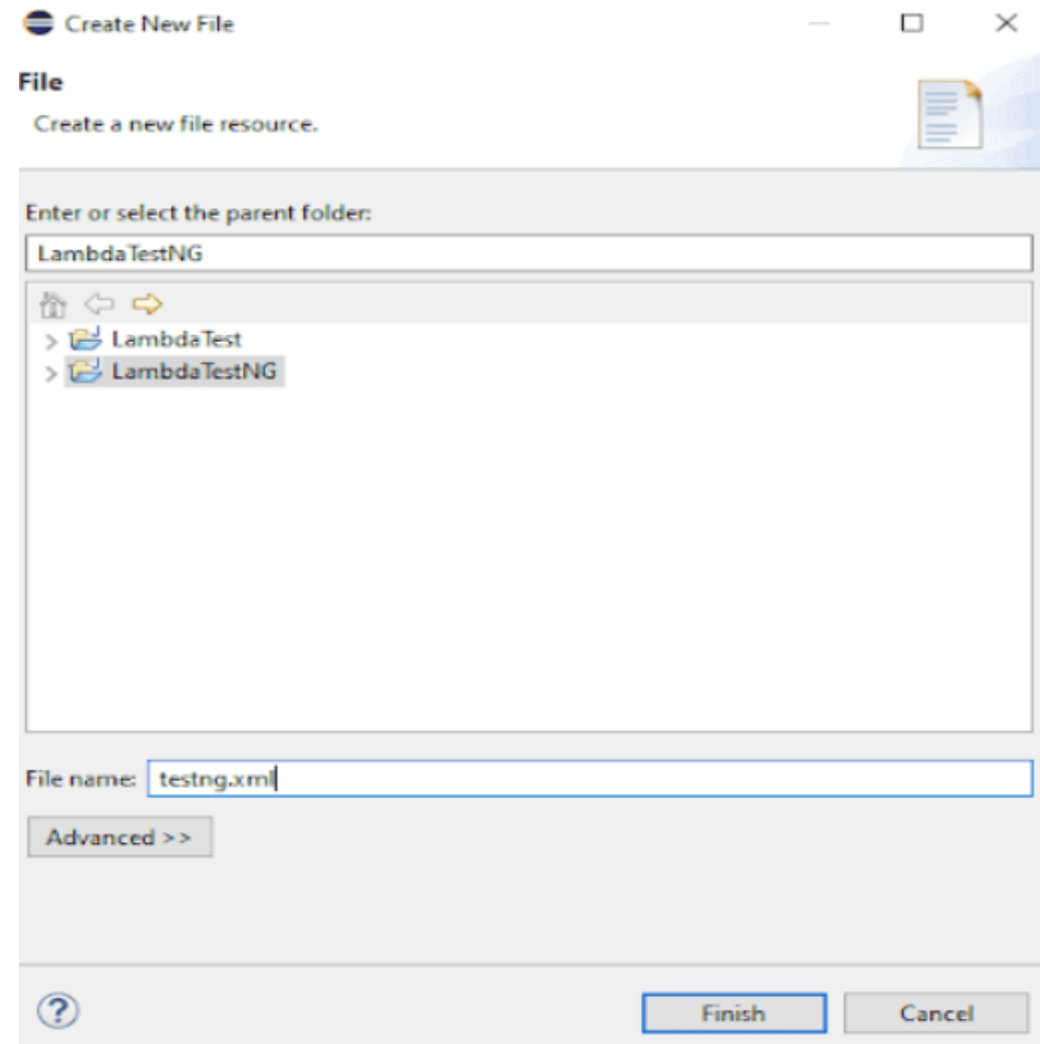
Step1: Right-click on New, and select File..



How to Create a TestNG.xml File in Eclipse?

Method 2:

Step2: Enter the following file name testng.xml, and then click on Finish.



testng.xml - **<suite>** attributes

```
<suite name="AppSuite"  
  verbose="1"  
  parallel="tests"  
  thread-count="4"  
  preserve-order="true">
```

- name — suite name.
- verbose — logging verbosity (0..10).
- parallel — controls parallelism. Valid values:
 - tests → run each <test> in parallel
 - classes → run test classes in parallel
 - methods → run test methods in parallel
 - instances → instances of test classes in parallel and - none (default)
- thread-count — number of threads to use when parallel is set.
- preserve-order — if true, TestNG will respect the order in which <test> and <class> appear.

testng.xml - Parameters (suite/test level)

```
<suite name="Suite1">
  <parameter name="baseUrl" value="https://demoblaze.com"/>
  <test name="FunctionalTests">
    <parameter name="browser" value="chrome"/>
    ...
  </test>
</suite>
```

- Use @Parameters in your code to read these values:

```
@Parameters({"baseUrl", "browser"})
@BeforeClass
public void setup(String url, String browser) { ... }
```

NOTE: Test-level parameters override suite-level parameters with the same name for that test.

testng.xml - Running groups (include/exclude)

```
<test name="Sanity">
  <groups>
    <run>
      <include name="smoke"/>
      <exclude name="slow"/>
    </run>
  </groups>
  <classes> ... </classes>
</test>
```

- groups + run lets you include/exclude groups.
- You can select multiple group names.
- dependsOnGroups (in code) makes tests dependent on entire groups.

testng.xml - Controlling class & method order

```
<test name="OrderTest">  
  <classes>  
    <class name="tests.LoginTests" />  
    <class name="tests.ProductTests" />  
    <class name="tests.CheckoutTests" />  
  </classes>  
</test>
```

- If preserve-order="true", classes run in this order.

testng.xml - Controlling class & method order

```
<class name="tests.CheckoutTests">
```

```
  <methods>
```

```
    <include name="validateCart" />
```

```
    <include name="placeOrder" />
```

```
    <exclude name="flakyTest" />
```

```
  </methods>
```

```
</class>
```

- Using <methods> you define exactly which methods to run and in what order.
- This is the most reliable way to enforce a strict method execution order from XML.

priority (TestNG @Test) vs XML ordering — how execution is determined

- **@Test(priority = N)** is method-level and assigns execution priority inside a test class.
 - Lower numeric value → higher priority (runs earlier).
 - Default priority is 0 if not specified.
- **XML ordering** (class and method includes) explicitly controls order when you must enforce a sequence across classes or methods.

General rules / practical guidance:

- If you need guaranteed order within a class, use priority or dependsOnMethods (for dependency) and avoid relying on unspecified behavior.
- If you need exact order independent of code, use <methods><include> ordering in testng.xml.
- If tests have dependsOnMethods / dependsOnGroups, dependencies override naive ordering: dependent methods run only if dependencies succeed.

priority (TestNG @Test) vs XML ordering — how execution is determined

General rules / practical guidance:

- If both priority and XML <methods> are used, <methods> ordering in XML takes precedence for methods in that class (because it instructs TestNG exactly which methods to run and the order).

Important: Do not assume stable order when priorities are equal or unspecified — behavior can vary; be explicit.

testng.xml - listeners, suites, and reporters

```
<suite name="MySuite">  
  <listeners>  
    <listener class-name="utils.TestListener"/>  
  </listeners>  
  ...  
</suite>
```

- Implement `ITestListener`, `ISuiteListener`, etc. in your listener class.
- Reporters can be attached similarly or via plugins (Allure, ExtentReports).

testng.xml - Parallel execution example

```
<suite name="ParallelSuite" parallel="classes" thread-count="4" preserve-order="true">
  <test name="ChromeTests">
    <parameter name="browser" value="chrome"/>
    <classes>
      <class name="tests.LoginTests"/>
      <class name="tests.ProductTests"/>
      <class name="tests.CheckoutTests"/>
    </classes>
  </test>
</suite>
```

- When parallel="classes", each class instance runs in its own thread.
- Ensure WebDriver instances are thread-safe (create per-thread driver in @BeforeClass or using ThreadLocal<WebDriver>).

testng.xml - Parallel execution example

```
<suite name="ParallelSuite" parallel="classes" thread-count="4" preserve-order="true">
  <test name="ChromeTests">
    <parameter name="browser" value="chrome"/>
    <classes>
      <class name="tests.LoginTests"/>
      <class name="tests.ProductTests"/>
      <class name="tests.CheckoutTests"/>
    </classes>
  </test>
</suite>
```

- When parallel="classes", each class instance runs in its own thread.
- Ensure WebDriver instances are thread-safe (create per-thread driver in @BeforeClass or using ThreadLocal<WebDriver>).

Understanding TestNG XML file structure

testng.xml (comprehensive)

```
<!DOCTYPE suite SYSTEM "https://testng.org/testng-1.0.dtd" >
<suite name="DemoblazeSuite"
  verbose="2"
  parallel="tests"
  thread-count="3"
  preserve-order="true">

  <!-- Suite-level parameter -->
  <parameter name="baseUrl" value="https://demoblaze.com"/>

  <!-- Listeners -->
  <listeners>
    <listener class-name="utils.TestListener"/>
  </listeners>
```

Understanding TestNG XML file structure

testng.xml (comprehensive)

```
<!-- Test 1: Functional Smoke (runs in parallel with other tests) -->
<test name="FunctionalSmoke" preserve-order="true">
  <parameter name="browser" value="chrome"/>
  <groups>
    <run>
      <include name="smoke"/>
    </run>
  </groups>
  <classes>
    <class name="tests.LoginTests" />
    <class name="tests.ProductTests">
      <!-- Enforce method order inside ProductTests -->
      <methods>
        <include name="verifyCategories" />
        <include name="addToCart" />
      </methods>
    </class>
  </classes>
</test>
```

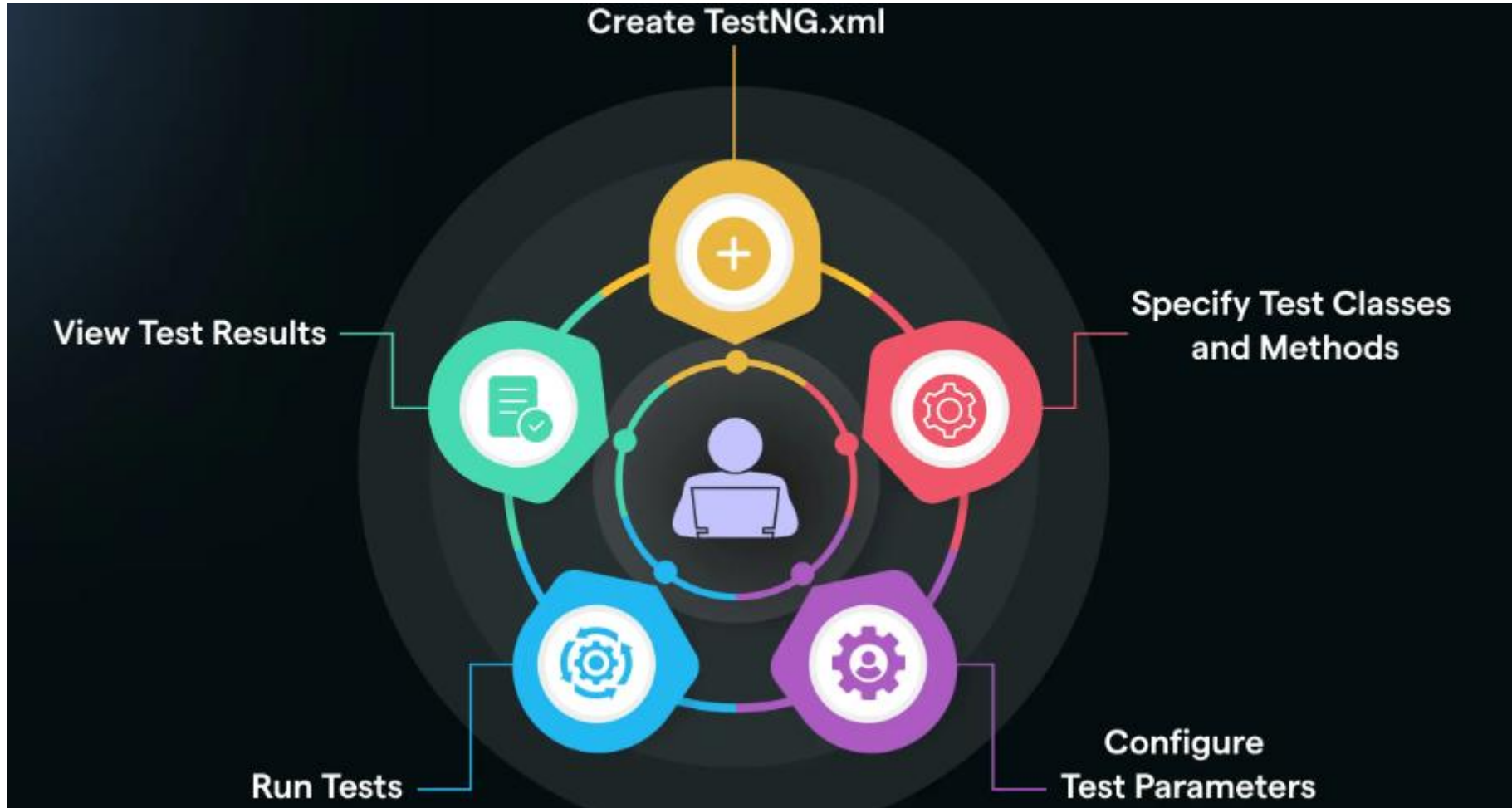
Understanding TestNG XML file structure

testng.xml (comprehensive)

```
<!-- Test 2: Regression (different environment) -->
<test name="Regression" preserve-order="true">
  <parameter name="browser" value="firefox"/>
  <classes>
    <class name="tests.CheckoutTests" />
    <class name="tests.PaymentTests" />
  </classes>
</test>

</suite>
```

Example Run Using TestNG.xml



Example Run Using TestNG.xml

- 1. Create TestNG.xml:** Begin by creating a TestNG.xml file where you define your test suite, test classes, methods, parameters, and other settings.
- 2. Specify Test Classes and Methods:** In TestNG.xml, specify the test classes and methods you want to include in your test execution. You can also define test groups if needed.
- 3. Configure Test Parameters:** Set any necessary test parameters or configurations in TestNG.xml, such as browser type, environment settings, or test data.
- 4. Run Tests:** Once TestNG.xml is configured, run your tests using TestNG. You can run tests from the command line, IDE (Integrated Development Environment) like IntelliJ IDEA or Eclipse, or through build tools like Maven.

Example Run Using TestNG.xml

5. View Test Results: After test execution, view the test results generated by TestNG. You'll see detailed reports indicating the status of each test case, including pass/fail status, execution time, and any errors or exceptions encountered.

How to create and run TestNG.xml of a TestNG class

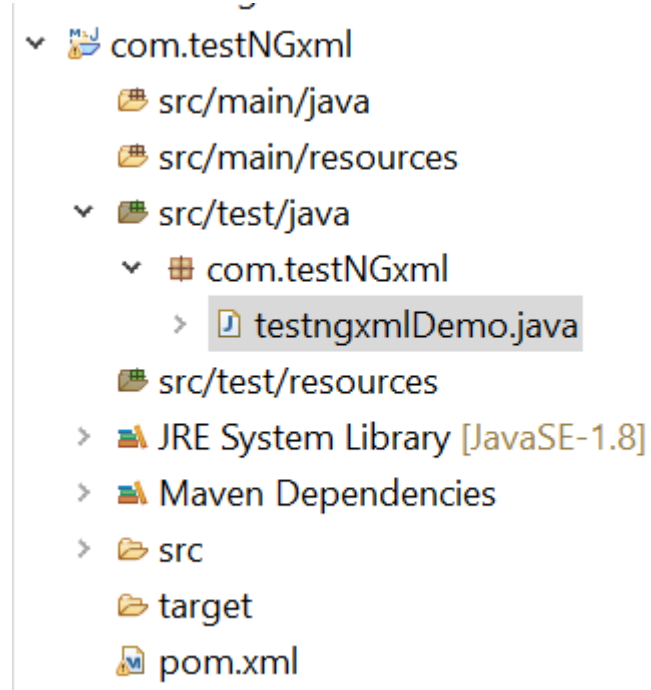


testNG.xml

Run using testNG.xml and testNG class

- There are 2 ways to run TestNG tests – using Run As TestNG tests and from testng.xml.

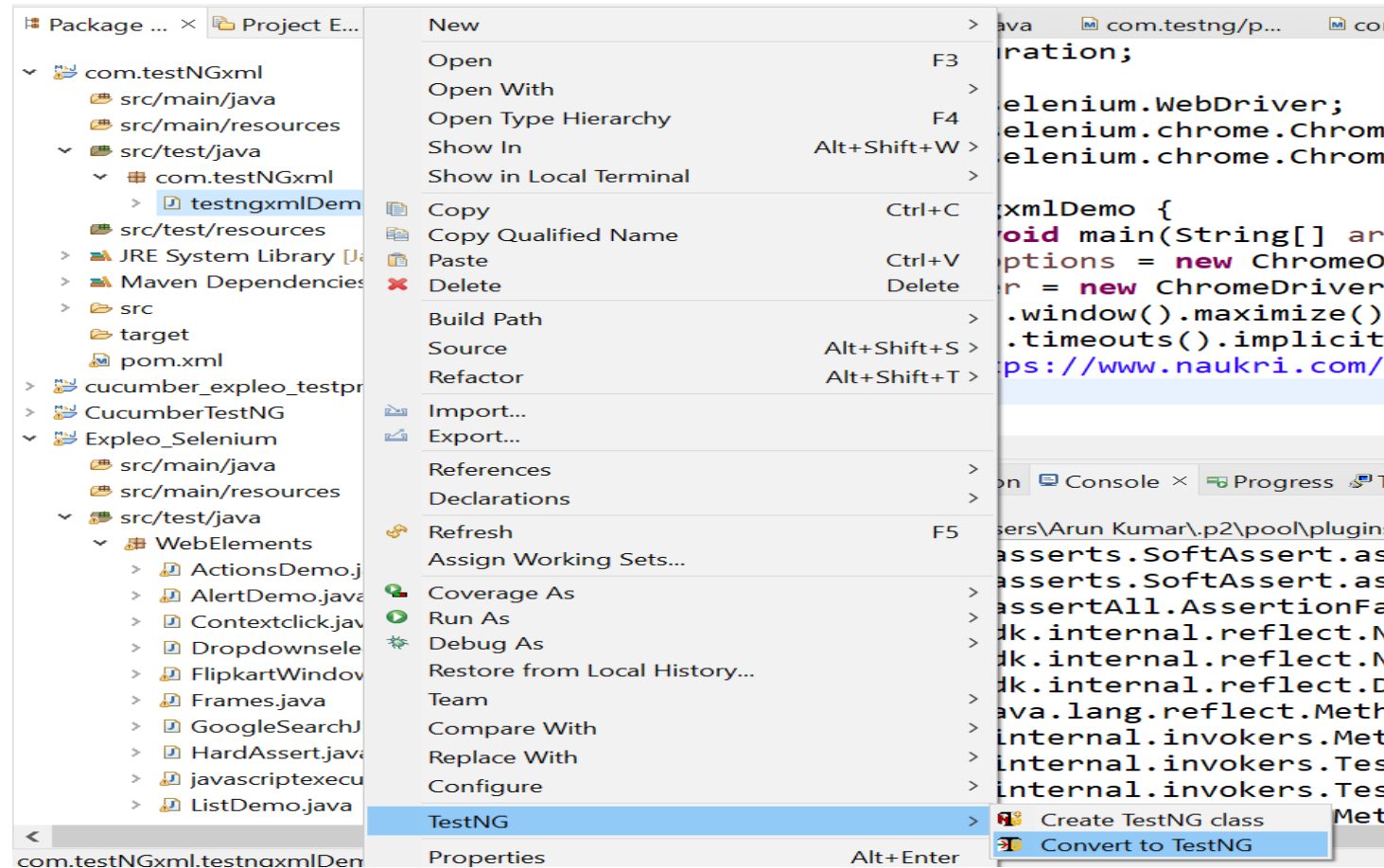
Step 1 – In the below project, there is **no testng.xml** present.



testNG.xml

Run using testNG.xml and testNG class

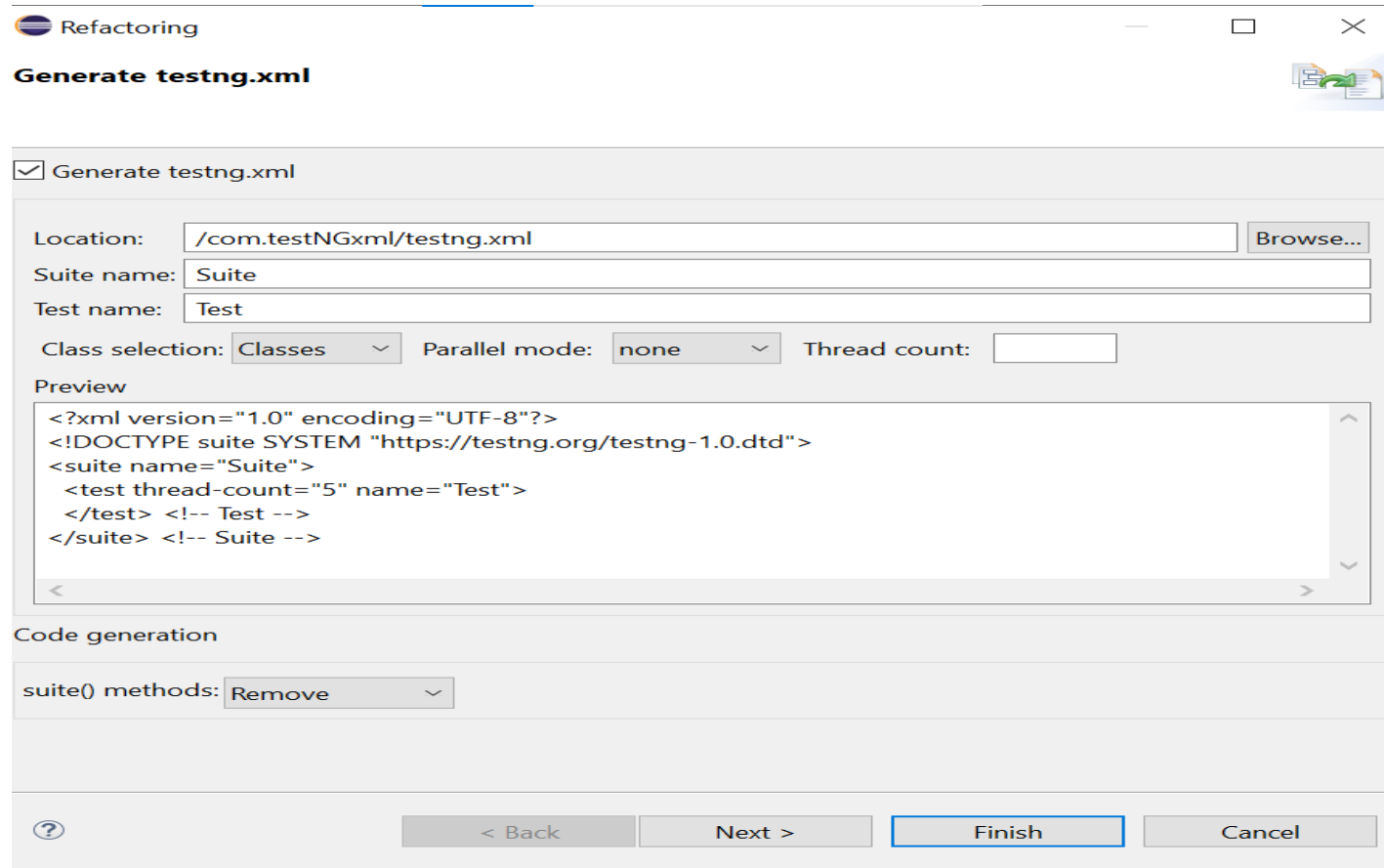
Step 2 – Right click on class name “API_Test.java” and navigate “TestNG→Convert to TestNG”.



testNG.xml

Run using testNG.xml and testNG class

Step 3 – Now a new window for testng.xml will be generated as shown below. Click the **Finish** Button.



Refactoring

Generate testng.xml

☒ Generate testng.xml

Location:

Suite name:

Test name:

Class selection: Parallel mode: Thread count:

Preview

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE suite SYSTEM "https://testng.org/testng-1.0.dtd">
<suite name="Suite">
  <test thread-count="5" name="Test">
    </test> <!-- Test -->
  </suite> <!-- Suite -->

```

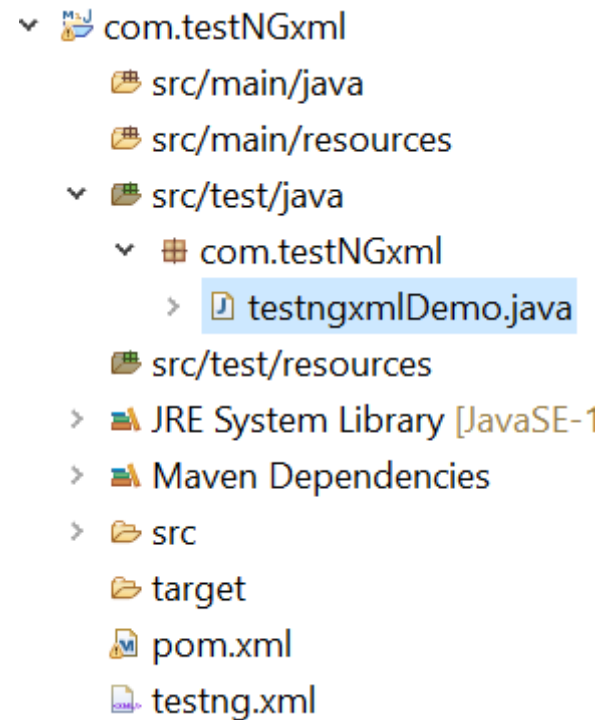
Code generation

suite() methods:

testNG.xml

Run using testNG.xml and testNG class

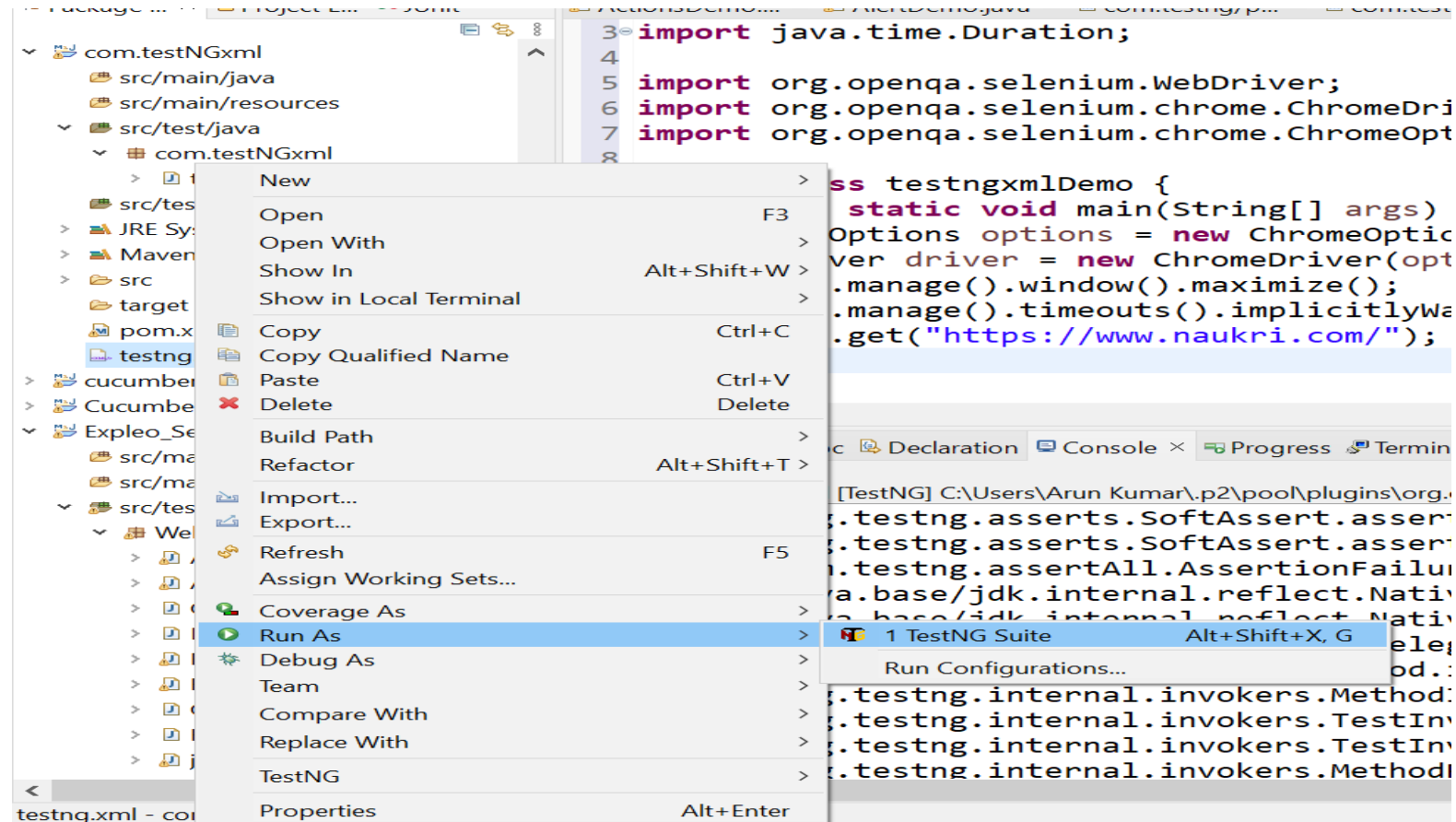
Step 4 – A xml file named “**testng.xml**” will be generated in project hierarchy.



testNG.xml

Run using testNG.xml and testNG class

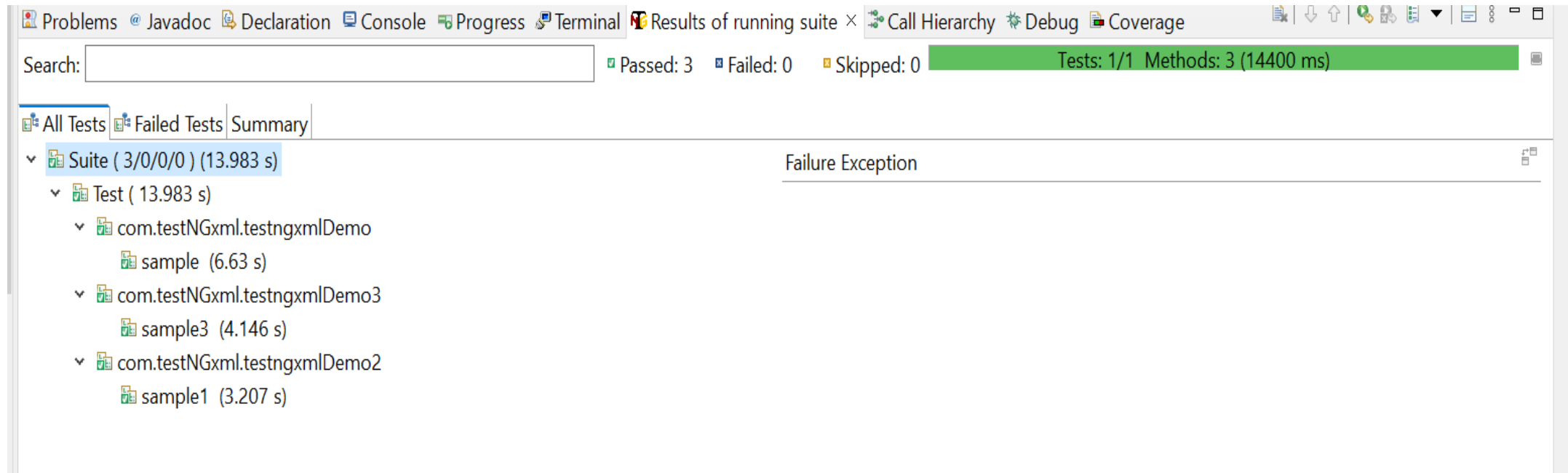
Step 5 – Right click on this testng.xml and select Run As→ Testng Suite.



testNG.xml

Run using testNG.xml and testNG class

Step 6 – You will view the result in two different tabs: **Console** and “**Results of running suite**”. Below is the image of Rest of running suite tab.



Run testNG tests for Command Line



testNG.xml

Run testNG tests for Command Line

Prerequisite

- 1.Selenium
 - 2.TestNG
 - 3.Maven
 - 4.Java 7 and above
 - 5.Maven Compiler Plugin
 - 6.Maven Surefire Plugin
- Imagine we need to run the TestNG Tests in CI/CD pipelines like Jenkins or GitLab, then we can't right-click and select TestNG Suite or tesng.xml to run the tests. In such situations, the tests can be executed ***through the command line.***

testNG.xml

Run testNG tests for Command Line

- We need to add plugins to pom.xml to compile the test code and then run the tests. To know more about Maven Surefire Plugin for TestNG, refer to this URL.
- **<https://maven.apache.org/surefire/maven-surefire-plugin/examples/testng.html>**

testNG.xml

Run testNG tests for Command Line

<build>

<plugins>

<plugin>

<groupId>org.apache.maven.plugins</groupId>

<artifactId>maven-compiler-plugin</artifactId>

<version>3.11.0</version>

<configuration>

<source>1.8</source>

<target>1.8</target>

</configuration>

</plugin>

<plugin>

<groupId>org.apache.maven.plugins</groupId>

<artifactId>maven-surefire-plugin</artifactId>

<configuration>

<suiteXmlFiles>

<suiteXmlFile>testng.xml</suiteXmlFile>

</suiteXmlFiles>

</configuration>

</plugin>

</plugins>

139 </build>

testNG.xml

Run testNG tests for Command Line

- It is needed to add testng.xml to suiteXmlFile to start the execution of the tests.

```
<suiteXmlFiles>  
    <suiteXmlFile>testng.xml</suiteXmlFile>  
</suiteXmlFiles>
```

testNG.xml

Run testNG tests for Command Line

- The below commands are used to execute the 'testng.xml' file from the command line.
- First, we need to go the place where the pom.xml of the project is placed.
- Then use the mvn compile test to compile the code and execute the TestNG tests..

```
C:\Windows\system32>cd C:\Users\Arun Kumar\eclipse-workspace\com.testNGxml
```

```
C:\Users\Arun Kumar\eclipse-workspace\com.testNGxml>mvn compile test
```

testNG.xml

Run testNG tests for Command Line

- After executing the above command, it should execute the tests that we have specified in testng.xml file. Below is the screenshot after the execution of the tests.

```
C:\Users\Arun Kumar\eclipse-workspace\com.testNGxml>mvn compile test
[INFO] Scanning for projects...
[WARNING]
[WARNING] Some problems were encountered while building the effective model for testNGxml:com.testNGxml:jar:0.0.1-SNAPSHOT
[WARNING] 'build.plugins.plugin.version' for org.apache.maven.plugins:maven-surefire-plugin is missing. @ line 29, column 9
[WARNING]
[WARNING] It is highly recommended to fix these problems because they threaten the stability of your build.
[WARNING]
[WARNING] For this reason, future Maven versions might no longer support building such malformed projects.
[WARNING]
[INFO]
[INFO] -----< testNGxml:com.testNGxml >-----
[INFO] Building com.testNGxml 0.0.1-SNAPSHOT
[INFO] from pom.xml
[INFO] -----[ jar ]-----
[INFO]
[INFO] --- resources:3.3.1:resources (default-resources) @ com.testNGxml ---
[WARNING] Using platform encoding (Cp1252 actually) to copy filtered resources, i.e. build is platform dependent!
[INFO] Copying 0 resource from src\main\resources to target\classes
[INFO]
[INFO] --- compiler:3.11.0:compile (default-compile) @ com.testNGxml ---
[INFO] Nothing to compile - all classes are up to date
[INFO]
[INFO] --- resources:3.3.1:resources (default-resources) @ com.testNGxml ---
[WARNING] Using platform encoding (Cp1252 actually) to copy filtered resources, i.e. build is platform dependent!
[INFO] Copying 0 resource from src\main\resources to target\classes
[INFO]
[INFO] --- compiler:3.11.0:compile (default-compile) @ com.testNGxml ---
[INFO] Nothing to compile - all classes are up to date
[INFO]
[INFO] --- resources:3.3.1:testResources (default-testResources) @ com.testNGxml ---
[WARNING] Using platform encoding (Cp1252 actually) to copy filtered resources, i.e. build is platform dependent!
[INFO] Copying 0 resource from src\test\resources to target\test-classes
[INFO]
[INFO] --- compiler:3.11.0:testCompile (default-testCompile) @ com.testNGxml ---
[INFO] Changes detected - recompiling the module! :input tree
[WARNING] File encoding has not been set, using platform encoding Cp1252, i.e. build is platform dependent!
[INFO] Compiling 3 source files with javac [debug target 1.8] to target\test-classes
```

testNG.xml

Run testNG tests for Command Line

- After executing the above command, it should execute the tests that we have specified in testng.xml file. Below is the screenshot after the execution of the tests.

```
INFO: Driver path: C:\Users\Arun Kumar\.cache\selenium\chromedriver\win64\118.0.5993.70\chromedriver.exe
Oct 23, 2023 12:23:37 PM org.openqa.selenium.manager.SeleniumManager lambda$runCommand$1
INFO: Browser path: C:\Users\Arun Kumar\.cache\selenium\chrome\win64\118.0.5993.70\chrome.exe
Oct 23, 2023 12:23:38 PM org.openqa.selenium.devtools.CdpVersionFinder findNearestMatch
WARNING: Unable to find an exact match for CDP version 118, so returning the closest version found: 116
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 15.51 s -- in TestSuite
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 21.131 s
[INFO] Finished at: 2023-10-23T12:23:40+05:30
[INFO] -----

C:\Users\Arun Kumar\eclipse-workspace\com.testNGxml>
```

testNG.xml

Run testNG tests for Command Line

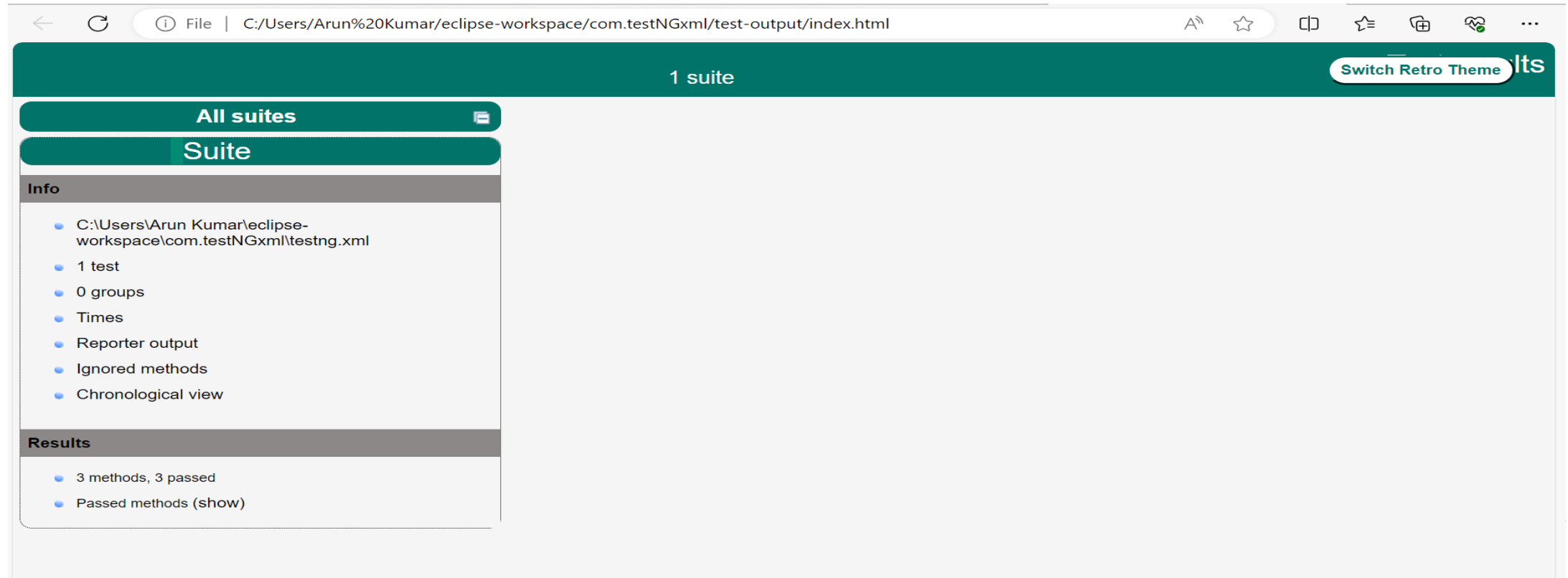
- This execution generates various TestNG Reports.
- We are concerned about **emailable-report.html** and **index.html**.

<

testNG.xml

Run testNG tests for Command Line

- This execution generates various TestNG Reports.
- We are concerned about **emailable-report.html** and **index.html**.



Parameterization using testng.xml



Parameterization

- Similar to parameters in any other programming language, they are declared to pass some values onto the function.
- One of the important features of TestNG is the ability to pass different test data to a test case as arguments, which is called parametrization
- A simple reason to use parameters is that they let us run a function many times with different values or to run different functions with the same values.
- Parameters pass the values in the runtime.

Parameterization using testing.xml

- Parameterization using testng.xml means declaring values (parameters) in your suite or test XML file and injecting them into TestNG test methods or configuration methods using the `@Parameters` annotation.
- It's useful for environment/browser configuration, timeouts, base URLs, and small sets of values that control test behavior.
- Use testng.xml parameters when you want:
 - To run the same tests against different environments (QA, Staging).
 - To vary browser or base URL without changing code.
 - To control time-outs, feature toggles, or small config values at runtime. To have CI/CD pipelines provide values per job (via separate testng.xml files or overriding).

Where to declare parameters (scope/override rules)

- You can declare parameters at two levels:

1. Suite-level — available to all <test> blocks unless overridden.

```
<suite name="Suite">
  <parameter name="baseUrl" value="https://demoblaze.com"/>
  ...
</suite>
```

Where to declare parameters (scope/override rules)

- You can declare parameters at two levels:

2. Test-level — overrides suite-level value for that <test>.

```
<test name="ChromeTest">  
  <parameter name="browser" value="chrome"/>  
  ...  
</test>
```

Override rule: Test-level parameter with **same name overrides suite-level** for that specific <test>.

TestNG parameters

- If we need to pass some simple values such as String or Integer types to the test methods at runtime, there is something called `@Parameter` where parameter values through TestNG XML configuration files pass to test.
- **Syntax:**
 - i) `@Parameters("value")`
 - ii) `@Parameters {"a", "b"}`

where value (single value) that pass to the function, a and b are the values that pass to the function.

Parameterization using testng.xml

An example XML file

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE suite SYSTEM "https://testng.org/testng-1.0.dtd">
<suite name="TestNG Parameters Suite">
  <test name="Params">
    <parameter name="val1" value="2" />
    <parameter name="val2" value="3" />
    <classes>
      <class name="Params" />
    </classes>
  </test>
</suite>
```

name: Name of the variable that you declared in the test case file like val1 and val2

value: The value of the variable you want to insert.

Sample testNG Parameterization Program1

```
package com.testng;
import org.testng.annotations.Test;
public class Params
{
    @Test
    @Parameters ({"val1", "val2"})
    public void Sum(int v1, int v2) {
        int finalsum = v1 + v2;
        System.out.println("The final sum of the given values is " + finalsum);
    }

    @Test
    @Parameters ({"val1", "val2"})
    public void Sum1(int v1, int v2) {
        int finalsum = v1 - v2;
        System.out.println("The final difference of the given values is " + finalsum);
    }
}
```

Sample testNG Parameterization Program2

```
public class TestNGParameterizationDemo {
    public WebDriver driver;
    By userName = By.name("username");
    By passWord = By.name("password");

    By loginBtn = By.xpath("//*[@class='oxd-form']/div[3]/button");
    By loginTitle = By.xpath("//*[@class='oxd-topbar-header-breadcrumb']/h6");
    By errorMessage = By.xpath("//*[@class='orangehrm-login-error']/div[1]/div/p");

    @BeforeMethod
    @Parameters("browser")
    public void parameterizedTest(String browser) {
        if (browser.equalsIgnoreCase("firefox")) {

            FirefoxOptions options=new FirefoxOptions();
            options.addArguments("--start-maximized");
            driver=new FirefoxDriver(options);
            System.out.println("Browser Started :" + browser);
        }
    }
}
```


Sample testNG Parameterization Program2

```
else if (browser.equalsIgnoreCase("chrome")) {  
    ChromeOptions options=new ChromeOptions();  
    options.addArguments("--start-maximized");  
    driver=new ChromeDriver(options);  
    System.out.println("Browser Started :" + browser);  
}  
  
driver.manage().timeouts().implicitlyWait(10, TimeUnit.SECONDS);  
driver.get("https://opensource-demo.orangehrmlive.com/");  
}
```

Sample testNG Parameterization Program2

```
@Test
public void validCredentials() {

    driver.findElement(userName).sendKeys("Admin");
    driver.findElement(passWord).sendKeys("admin123");
    driver.findElement(loginBtn).click();
    String newPageText = driver.findElement(loginTitle).getText();
    System.out.println("newPageText :" + newPageText);
    assertEquals(newPageText,"Dashboard");
}
```

Sample testNG Parameterization Program2

```
@Test
public void invalidCredentials() {

    driver.findElement(userName).sendKeys("1234");
    driver.findElement(passWord).sendKeys("admin3456");
    driver.findElement(loginBtn).click();
    String actualErrorMessage = driver.findElement(errorMessage).getText();
    System.out.println("Actual ErrorMessage :" + actualErrorMessage);
    assertEquals(actualErrorMessage,"Invalid credentials");

}

@AfterMethod
public void closeBrowser() {
    driver.quit();
}
}
```

Parameterization using testng.xml

testNG.xml file

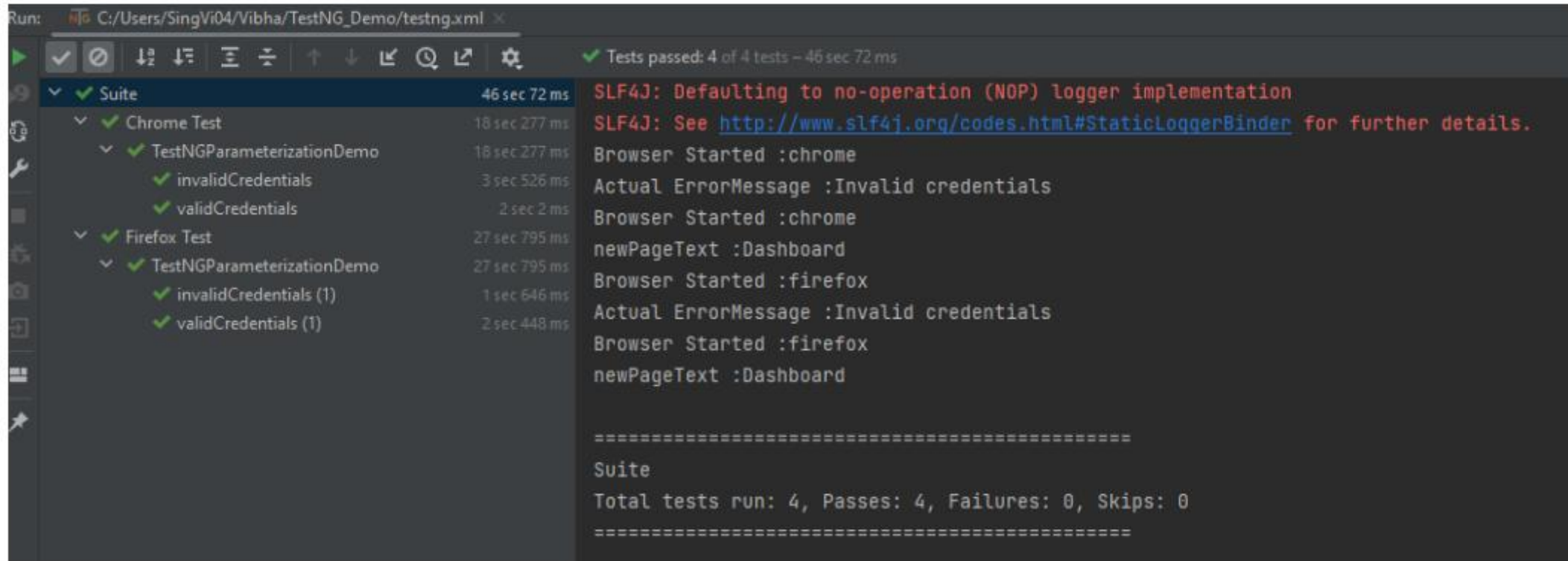
- TestNG.xml looks like this, as shown below.
- Here, the parameter name is the browser name value for the browser is “Chrome”. So, this “Chrome” value is passed to Test as a parameter, and as a result, a Google Chrome browser opens. Similarly, the same tests are run using Firefox, as it is mentioned in the testng.xml.

```
1  <?xml version="1.0" encoding="UTF-8"?>
2  <!DOCTYPE suite SYSTEM "https://testng.org/testng-1.0.dtd">
3
4  <suite name="Suite ">
5
6      <test name="Chrome Test">
7          <parameter name="browser" value="chrome" />
8          <classes>
9              <class name="TestNGParameterizationDemo" />
10          </classes>
11      </test> <!-- Test -->
12
13      <test name="Firefox Test">
14          <parameter name="browser" value="firefox" />
15          <classes>
16              <class name="TestNGParameterizationDemo" />
17          </classes>
18      </test> <!-- Test -->
19  </suite> <!-- Suite -->
```

Parameterization using testng.xml

testNG.xml file

The output of the above program is



Run: C:/Users/SingVi04/Vibha/TestNG_Demo/testng.xml

Tests passed: 4 of 4 tests – 46 sec 72 ms

Test Suite	Time
✓ Suite	46 sec 72 ms
✓ Chrome Test	18 sec 277 ms
✓ TestNGParameterizationDemo	18 sec 277 ms
✓ invalidCredentials	3 sec 526 ms
✓ validCredentials	2 sec 2 ms
✓ Firefox Test	27 sec 795 ms
✓ TestNGParameterizationDemo	27 sec 795 ms
✓ invalidCredentials (1)	1 sec 646 ms
✓ validCredentials (1)	2 sec 448 ms

```

SLF4J: Defaulting to no-operation (NOP) logger implementation
SLF4J: See http://www.slf4j.org/codes.html#StaticLoggerBinder for further details.
Browser Started :chrome
Actual ErrorMessage :Invalid credentials
Browser Started :chrome
newPageText :Dashboard
Browser Started :firefox
Actual ErrorMessage :Invalid credentials
Browser Started :firefox
newPageText :Dashboard

=====
Suite
Total tests run: 4, Passes: 4, Failures: 0, Skips: 0
=====

```

Parallel Tests in Selenium with TestNG



TestNG Suite

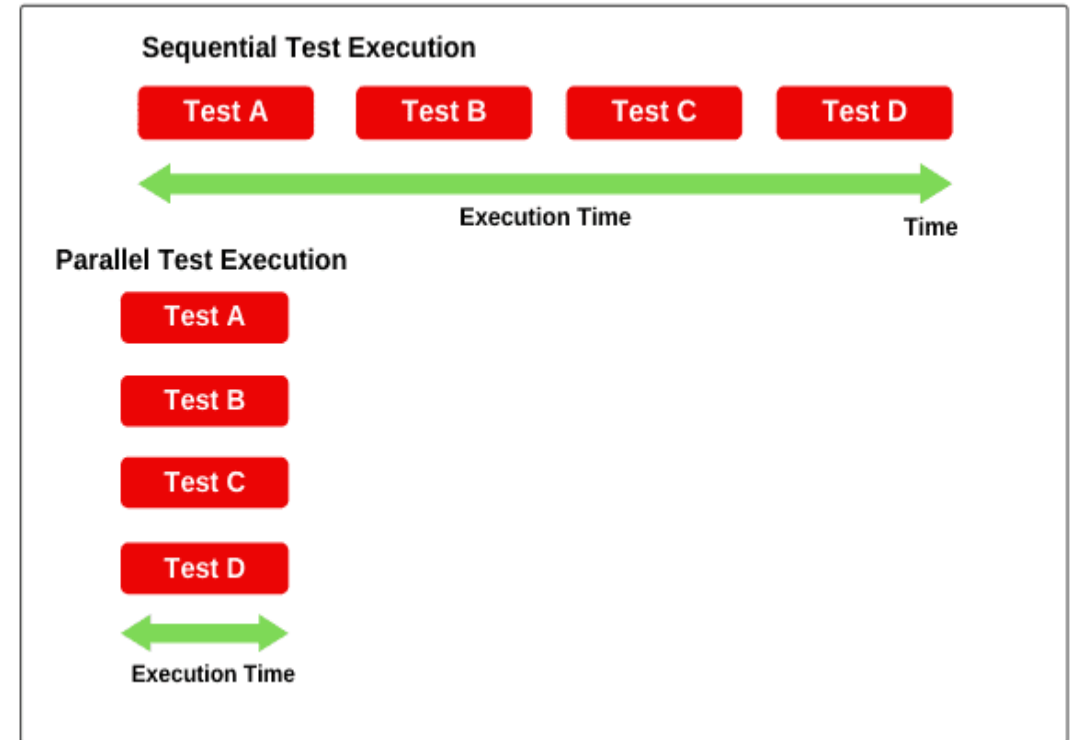
- A test suite is a collection of test cases intended to test a behavior or a set of behaviors of software program
- In TestNG, we cannot define a suite in testing source code, but it is represented by one XML file, as suite is the feature of execution
- It is defined by the `<suite>` tag.

```
testng.xml
1 <?xml version="1.0" encoding="UTF-8"?>
2 <!DOCTYPE suite SYSTEM "http://testng.org/testng-1.0.dtd">
3 <suite name="Testing Google Apps">
4   <test name="Regression">
5     <classes>
6       <class name="Googletest.GmailTest"/>
7       <class name="Googletest.MapsTest"/>
8       <class name="Googletest.ImagesTest"/>
9     </classes>
10  </test> <!-- Test -->
11 </suite> <!-- Suite -->
12
```


TestNG Suite and Parallel Execution

TestNG Parallel Execution

- Parallel execution, as the name suggests, is a process of running the test case parallelly rather than one after the other.
- In parallel testing, the program's multiple parts (or modules) execute together, saving the testers a lot of time and effort.
- The operating systems functionalities do this, but as a user, we need to trigger the parallel execution through TestNG



TestNG Parallel Execution (Contd.)

Advantages of Parallel Testing:

- 1) Reduces Time
- 2) Allow Multi-Threaded Tests
- 3) Wider compatibility
- 4) Optimizes Continuous Integratiion and Delivery
- 5) Distributed Tests
- 6) Better test Coverage

Parallel Tests in Selenium with TestNG

- Parallelism or multi-threading in software terms is defined as the ability of the software, operating system, or program to execute multiple parts or sub-components of another program simultaneously.
- Parallel testing helps to reduce execution time and efforts and results in faster time to delivery.
- In a scenario where we have two versions of software available, and we need to check its stability and compatibility, we can run the two versions simultaneously and find issues at a much faster rate.

How to run Parallel Tests with Selenium?

- TestNG provides multiple ways to execute tests in separate threads. In testng.xml, if we set 'parallel' attribute on the tag to 'methods', testNG will run all the '@Test' methods in tag in a separate thread.
- Let us create a class with multiple tests. In the below Example, four test cases or methods . We want to run these methods parallelly. To achieve this, we need to add the below command in testng.xml

testNG.xml

How to run Parallel Tests with Selenium?

- Now, let us create a testng.xml. Right click on the project and select **TestNG -> Convert to TestNG**.
- The attribute thread-count allows you to specify how many threads should be allocated for this execution.
- **parallel = “methods”** means that the methods will run parallel
- The parallel attribute can be extended for multiple values, as below:
- **Methods:** Helps run methods in separate threads
- **Tests:** Help to run all methods belonging to the same tag in the same thread, means tests will run sequentially
- **Classes:** Helps to run all methods belonging to a class in a single thread
- **Instances:** Helps run all methods in the same instance in the same thread

How to run Parallel Tests with Selenium?



```

Package ... × Project E... JUnit
Appium_Sample
com.cucumberframework
▼ com.parallelTestNGRun
    src/main/java
    src/main/resources
    ▼ src/test/java
        ▼ com.parallelTestNGRun
            > parallelTest.java
    src/test/resources
    > JRE System Library [JavaSE-1.8]
    > Maven Dependencies
    > src
    > target
    pom.xml
    testng.xml

com.testng/p... assertAll.java com.testNGxm... testngxmlDem... testng.xml *parallelTes... pa

https://testng.org/testng-1.0.dtd (doctype)
1 <?xml version="1.0" encoding="UTF-8"?>
2 <!DOCTYPE suite SYSTEM "https://testng.org/testng-1.0.dtd">
3 <suite name="Suite">
4 <test name="Parallel Tests" parallel = "methods" thread-count="4">
5 <classes>
6 <class name="com.parallelTestNGRun.parallelTest"/>
7 </classes>
8 </test> <!-- Test -->
9 </suite> <!-- Suite -->
10
  
```

Data-Driven Testing and Apache POI



Introduction

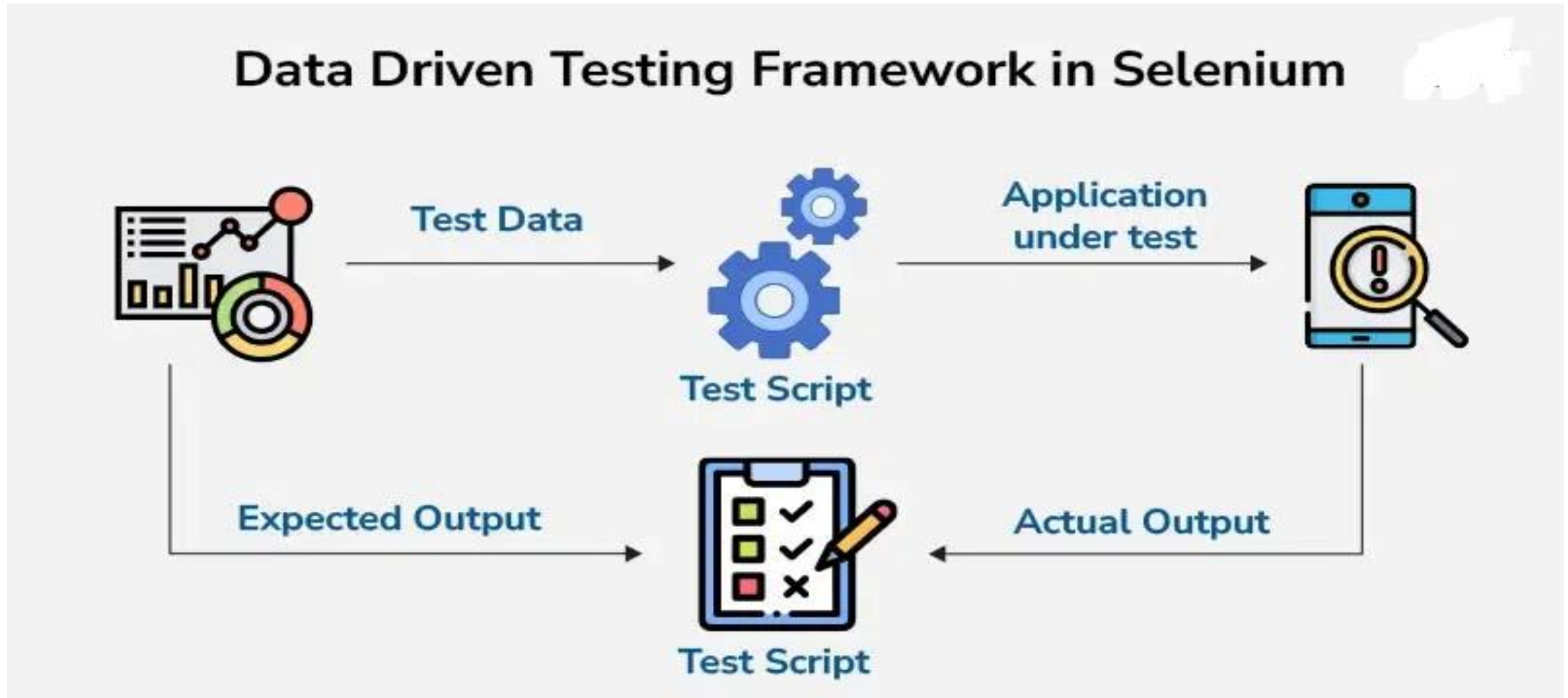
- As new functionalities (or features) are added to a web product, test cases (or test suites) must be implemented to test the new set of features. Before adding a new test case, you should have relevant answers to the following questions:
- Do we really need to develop a new test case for verifying the corresponding functionality?
- Is it feasible to modify an existing test case to suit the testing needs of the feature under test?
- Is it possible to change the existing test implementation so that the said 'product features' can be tested against different input combinations (or data sets)?
- Is it required to add new test implementation when there are minimal changes in the business rules?
- Is there a more optimal way to separate the test data from the test implementation?

The answers to the questions mentioned above lie with Data Driven Testing in Selenium WebDriver

Data Driven Testing

- The Selenium Data-Driven Test Framework Design pattern is developed for test scripts to read data from some external sources, such as Excel files, CSV files, databases, or XML files.
- The test scripts should not have hard-coded test data but rather feed in from outside sources that can later be modified with little tussle without changing the test scripts themselves.
- This not only eases the maintainability of tests but, in many cases, also makes the test process much easier to scale because a new data set is only added.
- The data feed or data source not only contains the input values used for Selenium automation testing but can also be used for storing the expected test result and the output test result.
- This can be useful in comparing the test execution result and storing the same for referring to later stages.

Data Driven Testing



Data Driven Testing

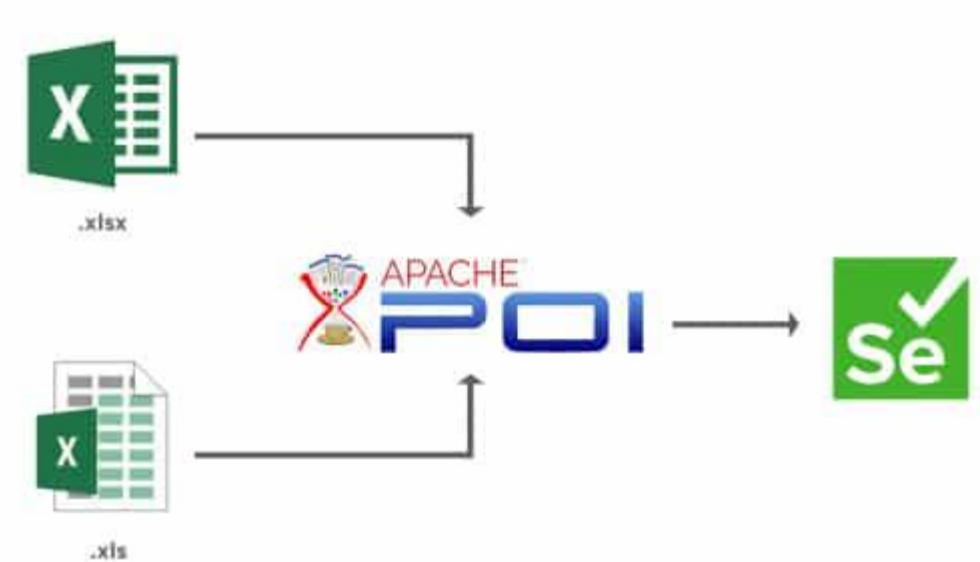
- This approach greatly improves maintainability of test scripts.
- For example, if the data is changed, such as adding or updating test cases, then the only edit that would be needed is in the data file; nothing is modified inside the test scripts.
- This way, through separation of data from test logic, it also means the scaling of the test process can be achieved more easily.
- Adding new test cases or data sets can be done just by adding a new entry in the data source and without writing any code.
- This does not only speed up the testing process but also adds consistency and coverage as a result of each running test logic for all data sets.

Advantages of Data Driven Testing

- **Reusability:** By reusing test scripts to test different sets of data, it reduces the number of test scripts that would need to be written.
- **Maintainability:** Since the test data are maintained outside the test script per se, it is far easier to do so.
- **Scalability:** Here, new test cases to be added are appended to the external file.
- **Effectiveness:** It reduces the number of scripts to be written and, in the long run, cuts down execution time and effort.

Apache POI

- POI stands For "Poor Obfuscation Implementation".
- Apache POI is an API provided by Apache foundation which is a collection of different java libraries.
- These libraries gives the facility to read, write and manipulate different Microsoft files such as excel sheet, power-point, and word files.
- It's first version release on 30 December 2001.
- It is the most commonly used API for realizing data driven tests in Selenium.



Handling Excel Files using Apache POI

- The Apache POI consists of classes and methods that simplify processing MS Excel sheets in Java.
 - Therefore, it is important to be well-versed with the common terminologies used when working with MS Excel.
- 1. Workbook** – A Workbook in Microsoft Excel is primarily used to create and maintain the Spreadsheet (or Sheet). It can contain one or more spreadsheets.
 - 2. Sheet** – A Sheet in a workbook is a page made up of Rows and Columns.
 - 3. Row** – A Row in a Sheet runs horizontally and is represented as a collection of Cells.
 - 4. Cell** – A Cell in a Sheet is represented by a Row and Column combination. Data entered by the user is stored in a cell. For example – Cell (0, 1) indicates the data is stored in Row: 0 and Column: 1.
- XSSF implementation in Apache POI lets you perform operations on MS Excel Files in the .xlsx format.

Handling .xls Files using Apache POI

- For reading .xls files, **HSSF implementation** is provided by the Apache POI library. The methods used for performing operations on the .xls files are available in the classes that belong to the org.apache.poi.hssf.usermodel package.
- Here are the important classes & methods in the org.apache.poi.hssf.usermodel package that helps in performing CRUD operations on .xls files:
 1. **HSSFWorkbook** – Represents a Workbook in the .xls format
 2. **HSSFSheet** – Represents a Sheet in the .xls workbook
 3. **HSSFRow** – Represents a Row in the .xls file
 4. **HSSFCell** – Represents a Cell in the Row of the .xls file

Handling .xlsx Files using Apache POI

- For reading .xlsx files, **XSSF implementation** is provided by the Apache POI library and the org.apache.poi.xssf.usermodel package contains classes and methods that let you achieve the same.
- Here are the important classes & methods in the org.apache.poi.xssf.usermodel package that helps in performing CRUD operations on .xlsx files:
 1. **XSSFWorkbook** – Represents a Workbook in the .xlsx format
 2. **XSSFSheet** – Represents a Sheet in the .xlsx workbook
 3. **XSSFRow** – Represents a Row in the .xlsx file
 4. **XSSFCell** – Represents a Cell in the Row of the .xlsx file

Using Flat files in DDT(Java)

Step 1: Create csv file (e.g LoginData.csv)

```
user1,pass1  
user2,pass2
```


Step 2: Create csv file reader (Utility class to read data)

```
package com.utilities;
import java.io.BufferedReader;
import java.io.FileReader;
import java.util.ArrayList;
import java.util.List;

public class CSVUtils {
    public static List<String[]> readCSV(String filePath) {
        List<String[]> data = new ArrayList<>();
        String line;

        try (BufferedReader br = new BufferedReader(new FileReader(filePath))) {
            while ((line = br.readLine()) != null) {
                String[] values = line.split(",");
                data.add(values);
            }
        } catch (Exception e) {
            e.printStackTrace();
        }
        return data;
    }
}
```

Step 3: Create test file with DDT (Main test class)

```
package com.test;

import java.time.Duration;
import java.util.List;
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.support.ui.Wait;
import org.openqa.selenium.support.ui.WebDriverWait;
import org.openqa.selenium.support.ui.ExpectedConditions;

import com.utilities.CSVUtils;

public class DataDrivenDemo {

    public static void main(String[] args) {
        // TODO Auto-generated method stub
        System.out.println("Working Directory = " + System.getProperty("user.dir"));
        List<String[]> credentials = CSVUtils.readCSV("C:\\Java-Selenium-
Automation\\Selenium_Basics\\src\\test\\resources\\LoginData.csv");
```

Step 3: Create test file with DDT (Main test class)

```
for (String[] login : credentials) {  
    String username = login[0];  
    String password = login[1];  
  
    System.out.println("Testing: " + username + " / " + password);  
  
    WebDriver driver = new ChromeDriver();  
    Wait<WebDriver> wait = new WebDriverWait(driver, Duration.ofSeconds(5));  
    driver.manage().window().maximize();  
    //driver.manage().timeouts().implicitlyWait(Duration.ofSeconds(5));  
    driver.get("https://www.demoblaze.com");  
    WebElement login1 = driver.findElement(By.id("login2"));  
    login1.click();  
  
    WebElement uname =  
        wait.until(ExpectedConditions.visibilityOfElementLocated(By.id("loginusername")));  
    uname.sendKeys(username);  
  
    WebElement passwd =  
        wait.until(ExpectedConditions.visibilityOfElementLocated(By.id("loginpassword")));  
    passwd.sendKeys(password);  
}
```

Step 3: Create test file with DDT (Main test class)

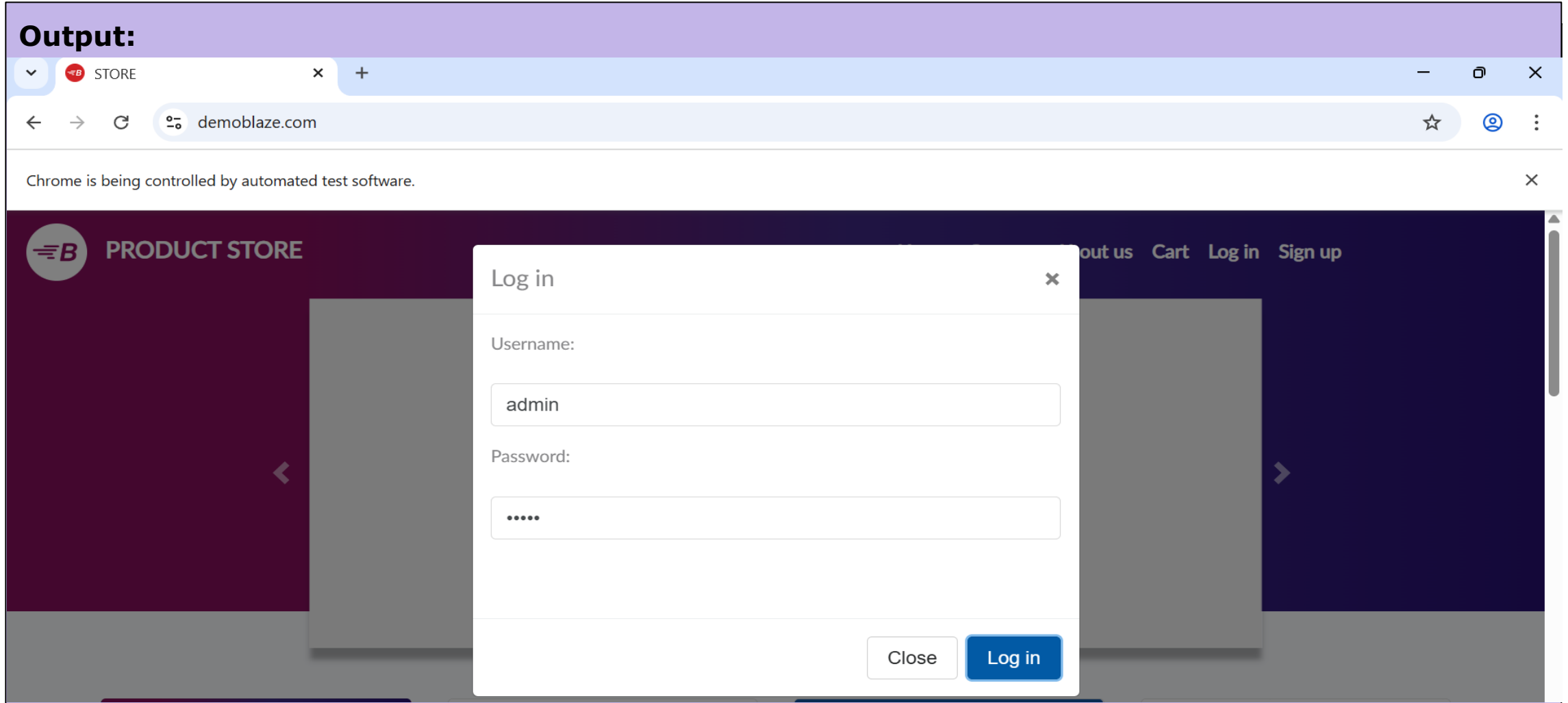
```
WebElement loginbtn = driver.findElement(By.xpath("//button[normalize-space()='Log in']"));
    loginbtn.click();

    driver.close();
}
}
```

Why do we need selenium waits?

An Example Scenario

Output:

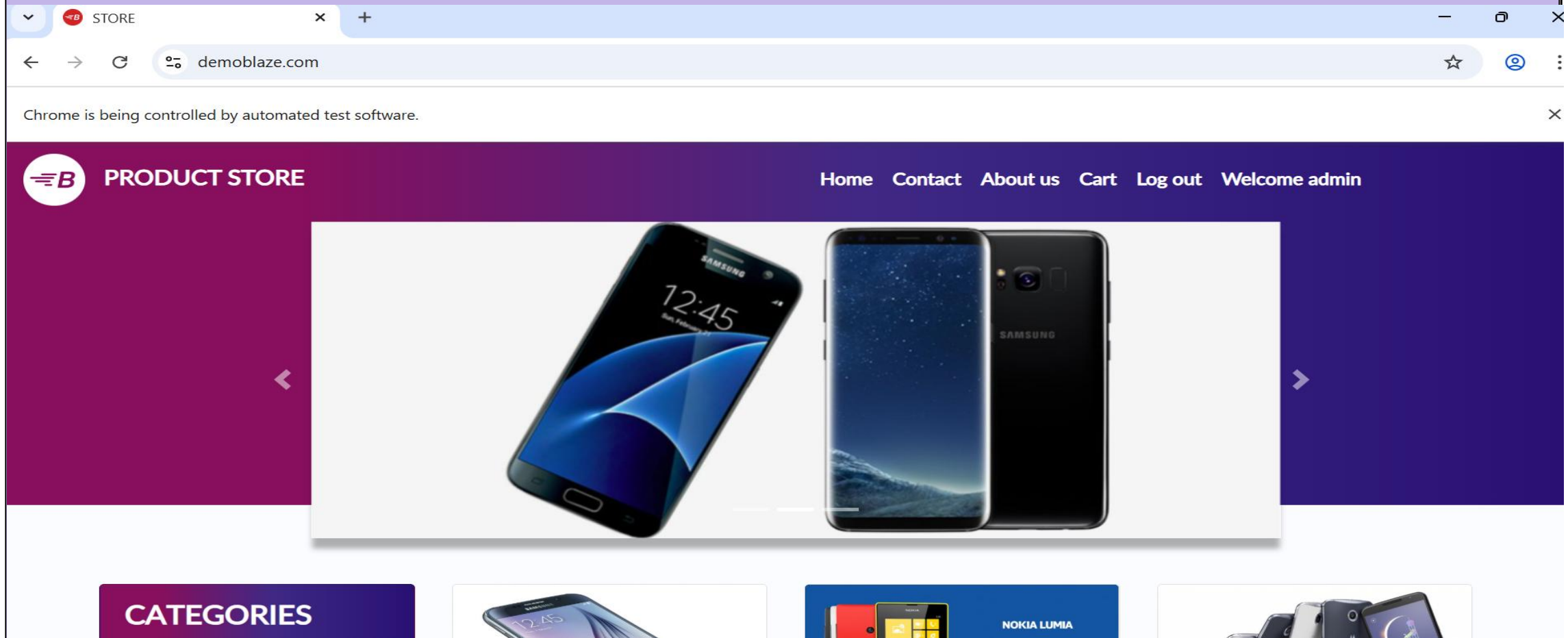


The screenshot shows a web browser window with the address bar displaying "demoblaze.com". The page content includes a "PRODUCT STORE" header and a "Log in" modal dialog. The modal has fields for "Username:" (containing "admin") and "Password:" (masked with dots). At the bottom of the modal are "Close" and "Log in" buttons. A notification bar at the top of the browser window states "Chrome is being controlled by automated test software."

Why do we need selenium waits?

An Example Scenario

Output:



Using Excel in DDT with Apache POI (Java)

Step 1: Create Excel file (e.g LoginData.xlsx)

Username	Password
user1	pass1
user2	pass2

Using Excel in DDT with Apache POI (Java)

Step 2: Add Apache POI Dependency (Maven)

```
<dependencies>  
  <dependency>  
    <groupId>org.apache.poi</groupId>  
    <artifactId>poi-ooxml</artifactId>  
    <version>5.2.3</version>  
  </dependency>  
</dependencies>
```


Step 3: Create Utility to Read Excel File

```
import org.apache.poi.ss.usermodel.*;
import java.io.File;
import java.io.FileInputStream;
public class ExcelUtils {
    @DataProvider(name = "excelData", parallel=true)
    public static String[][] readExcel(String filePath, String sheetName) {
        String[][] data = null;
        try {
            FileInputStream fis = new FileInputStream(new File(filePath));
            Workbook workbook = WorkbookFactory.create(fis);
            Sheet sheet = workbook.getSheet(sheetName);
            int rowCount = sheet.getPhysicalNumberOfRows();
            int colCount = sheet.getRow(0).getLastCellNum();
            data = new String[rowCount - 1][colCount];
        }
    }
}
```

Step 3: Create Utility to Read Excel File

```
        for (int i = 1; i < rowCount; i++) {  
            Row row = sheet.getRow(i);  
            for (int j = 0; j < colCount; j++) {  
                Cell cell = row.getCell(j);  
                data[i - 1][j] = cell.toString();  
            }  
        }  
        workbook.close();  
        fis.close();  
    } catch (Exception e) {  
        e.printStackTrace();  
    }  
    return data;  
}  
}
```

Step 4: Use Data in Test

```
public class LoginTest {  
  
    public static void main(String[] args) {  
        String[][] credentials = ExcelUtils.readExcel("LoginData.xlsx", "Sheet1");  
  
        for (String[] login : credentials) {  
            String username = login[0];  
            String password = login[1];  
  
            System.out.println("Testing with: " + username + " / " + password);  
  
            // Replace this with your actual Selenium test method  
            performLogin(username, password);  
        }  
    }  
}
```

Step 4: Use Data in Test

```
public static void performLogin(String username, String password) {  
    // Simulated login test  
    if (username.equals("user1") && password.equals("pass1")) {  
        System.out.println("Login successful for " + username);  
    } else {  
        System.out.println("Login failed for " + username);  
    }  
}
```

Quiz



1) What is the key component that separates logic from input values in data-driven frameworks?

a) Object Repository

b) Test Runner

c) External Data file

d) Script file

Answer : Option c)

Quiz



2) Why is reading test data from an external file preferred in large-scale automation?

a) It ensures faster browser launch

b) It avoids hardcoding and increases test flexibility

c) It supports only CSV files

d) It adds locators automatically

Answer : Option b)

Quiz



3) Why might a data-driven test suite produce inconsistent results with large datasets?

a) Data formatting issues or slow rendering

b) Memory leaks

c) Locator strategy issues

d) Application size

Answer : Option a)

Data Providers in TestNG



Data Provider

- A Data Provider (@DataProvider) is a TestNG feature that allows you to run a test method multiple times with different sets of data — i.e., data-driven testing.
- It helps avoid writing repetitive test cases for different inputs.
- A data-driven framework stores the test data in a table or spreadsheet format. It allows automation engineers to use a single test script to execute all test data in the table.
- In this framework, input values are read from data files and are stored in a variable in test scripts.
- Data-Driven testing enables the creation of both positive and negative test cases into a single test.
- In this test automation framework, input data can be stored in a single or multiple data sources like XLSX, XML, CSV, and databases.

Data Provider

Without DataProvider

One test = one set of input

Repetition of test methods

Hard-coded values

With DataProvider

One test = multiple sets of inputs

Centralized data handling

Dynamic, reusable data

Ideal for: Login credentials, checkout variations, form validation, negative testing, etc.

Data Provider - Syntax & Structure

```
@DataProvider(name = "dataSetName")
public Object[][] methodName() {
    return new Object[][] {
        {"data1", "data2"},
        {"data3", "data4"}
    };
}

@Test(dataProvider = "dataSetName")
public void testMethod(String value1, String value2) {
    // use the data here
}
```

- The method must return a 2D Object array (Object[][]).
- Each inner array represents one set of input data for one test execution.
- You can use primitive types, objects, or custom classes inside the array..

Data Provider

- A Data Provider is a method on the class that returns an array of array of objects. This method is annotated with `@DataProvider`
- A `@Test` method specifies its Data Provider with the `dataProvider` attribute. This name must correspond to a method on the same class annotated with `@DataProvider(name="...")` with a matching name.
- TestNG dataprovider returns a 2d list of objects.. An array of array of objects (`Object[][]`) where the first dimension's size is the number of times the test method will be invoked and the second dimension size contains an array of objects that must be compatible with the parameter types of the test method.

Data Provider

- DataProviders are not declared on top of the functions like TestNG parameters but have a method of their own, which in regular speaking terms called a dataprovider method. For example, dpMethod here.
- The dataprovider name calls the dataprovider method, and if there is no name specified by the tester, then the dataprovider method is the default name used in the receiving @Test case.
- Data providers can run in parallel with the attribute parallel.

Data Provider – Example1

```
import org.openqa.selenium.By;
import org.openqa.selenium.Keys;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.chrome.ChromeDriver;
import org.testng.annotations.AfterMethod;
import org.testng.annotations.BeforeMethod;
import org.testng.annotations.DataProvider;
import org.testng.annotations.Test;
public class dataProviderDemo {
    WebDriver driver;
    @DataProvider(name = "testData")
    public Object[][] dataProvFunc() {
        return new Object[][] { { "Selenium" }, { "TestNG" } };
    }
    @BeforeMethod
    public void setUp() {
        System.out.println("Start the test");
        driver = new ChromeDriver();
        driver.get("https://www.bing.com/");
        driver.manage().window().maximize();
    }
}
```

Data Provider – Example1

```
// Passing the dataProvider to the test method through @Test annotation
@Test(dataProvider = "testData")
public void search(String keyWord) {
    WebElement txtBox = driver.findElement(By.id("sb_form_q"));
    txtBox.sendKeys(keyWord);
    System.out.println("Keyword entered is : " + keyWord);
    txtBox.sendKeys(Keys.ENTER);
    System.out.println("Search result is displayed.");
}
@AfterMethod
public void tearDown() {
    driver.quit();
    System.out.println("End the test");
}
}
```

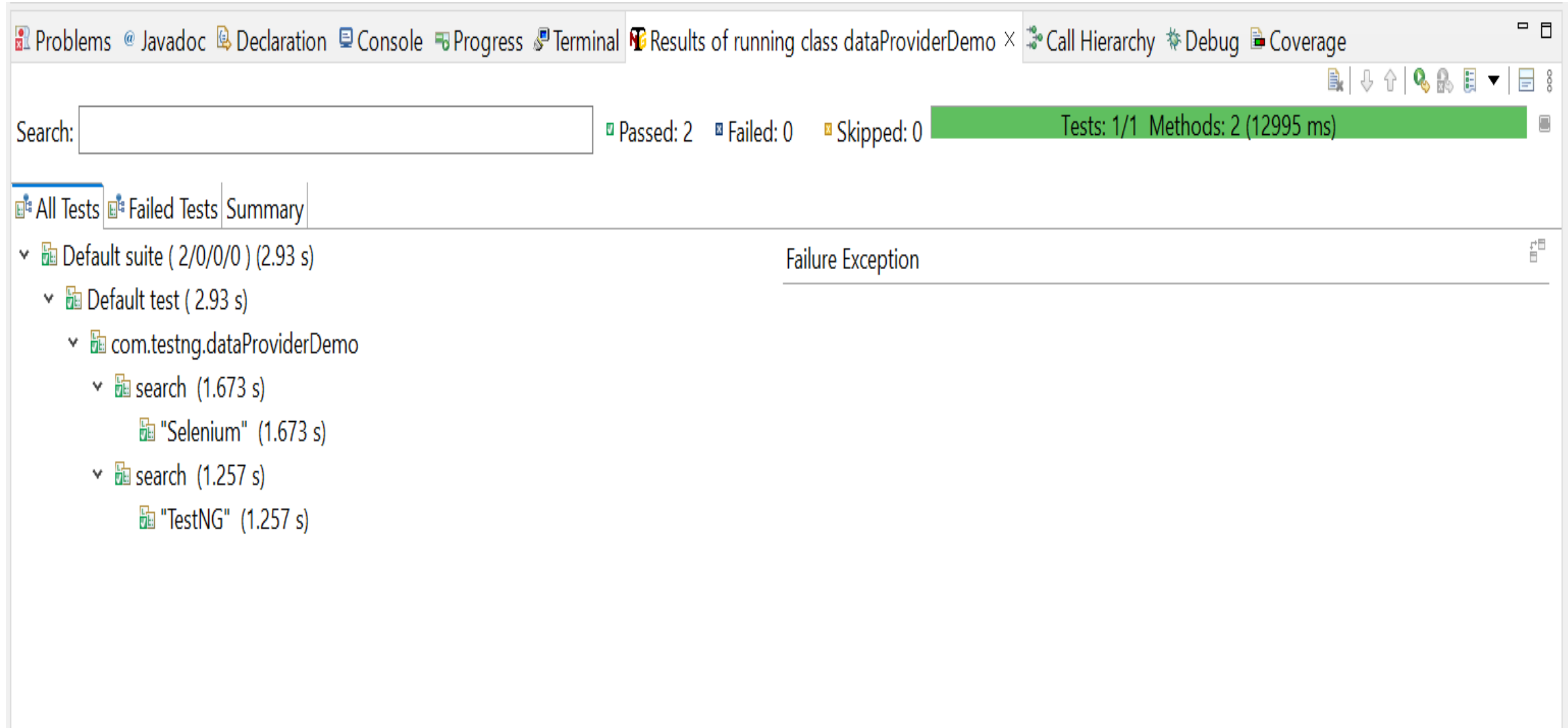
Data Provider – Example1 Output

```
[RemoteTestNG] detected TestNG version 7.4.0
Start the test
Starting ChromeDriver 93.0.4577.15 (660fc11082ba57405eca2e8c49c3e1af756fbfae-refs/branch-heads/4577@{#203}) on port 12560
Only local connections are allowed.
Please see https://chromedriver.chromium.org/security-considerations for suggestions on keeping ChromeDriver safe.
ChromeDriver was started successfully.
Oct 10, 2021 10:27:45 PM org.openqa.selenium.remote.ProtocolHandshake createSession
INFO: Detected dialect: W3C
Keyword entered is : Selenium
Search result is displayed.
End the test
Start the test
Starting ChromeDriver 93.0.4577.15 (660fc11082ba57405eca2e8c49c3e1af756fbfae-refs/branch-heads/4577@{#203}) on port 5324
Only local connections are allowed.
Please see https://chromedriver.chromium.org/security-considerations for suggestions on keeping ChromeDriver safe.
ChromeDriver was started successfully.
Oct 10, 2021 10:27:49 PM org.openqa.selenium.remote.ProtocolHandshake createSession
INFO: Detected dialect: W3C
Keyword entered is : TestNG
Search result is displayed.
End the test
PASSED: search("TestNG")
PASSED: search("Selenium")

=====
      Default test
      Tests run: 1, Failures: 0, Skips: 0
=====

=====
Default suite
Total tests run: 2, Passes: 2, Failures: 0, Skips: 0
=====
```


Data Provider – Example1 Output



The screenshot displays the TestNG results window in an IDE. The top toolbar includes tabs for Problems, Javadoc, Declaration, Console, Progress, Terminal, and Results of running class dataProviderDemo. The Results tab is active, showing a search bar and summary statistics: Passed: 2, Failed: 0, Skipped: 0, and a green bar indicating Tests: 1/1, Methods: 2 (12995 ms).

Below the summary, the test hierarchy is expanded under 'All Tests':

- Default suite (2/0/0/0) (2.93 s)
 - Default test (2.93 s)
 - com.testng.dataProviderDemo
 - search (1.673 s)
 - "Selenium" (1.673 s)
 - search (1.257 s)
 - "TestNG" (1.257 s)

The 'Failure Exception' column is visible but empty, indicating all tests passed.

DataProvider – Example2 Create separate class for DataProvider

- It is messy to have supporting methods like DataProvider and test code in one class.
- It is always preferred to declare the test case in one class and define TestNG parameters like DataProviders in another class.
- By default, the data provider will be looked for in the current test class or one of its base classes.
- If you want to put your data provider in a different class, it needs to be a static method or a class with a non-arg constructor, and you specify the class where it can be found in the dataProviderClass attribute.

//DataProvider Class

```
import org.testng.annotations.DataProvider;
public class DPDemo {
    @DataProvider(name = "testData")
    public Object[][] dataProvFunc() {
    return new Object[][] {
        { "Selenium" }, { "TestNG" }, { "Automation" } };
    }
}
```

DataProvider – Example2

```
public class dataProviderDemo2 {  
    public WebDriver driver;  
    @BeforeMethod  
    public void setUp() {  
        System.out.println("Start the test");  
        driver = new ChromeDriver();  
        driver.get("https://www.bing.com/");  
        driver.manage().window().maximize();  
    }  
    // Passing the dataProvider to the test method through @Test annotation  
    @Test(dataProvider = "testData", dataProviderClass = DPDemo.class)  
    public void search(String keyWord) {  
        WebElement txtBox = driver.findElement(By.id("sb_form_q"));  
        txtBox.sendKeys(keyWord);  
        System.out.println("Keyword entered is : " + keyWord);  
        txtBox.sendKeys(Keys.ENTER);  
        System.out.println("Search result is displayed.");  
    }  
    @AfterMethod  
    public void tearDown() {  
        driver.quit();  
        System.out.println("End the test");  
    }  
}
```

DataProvider – Example2 Output

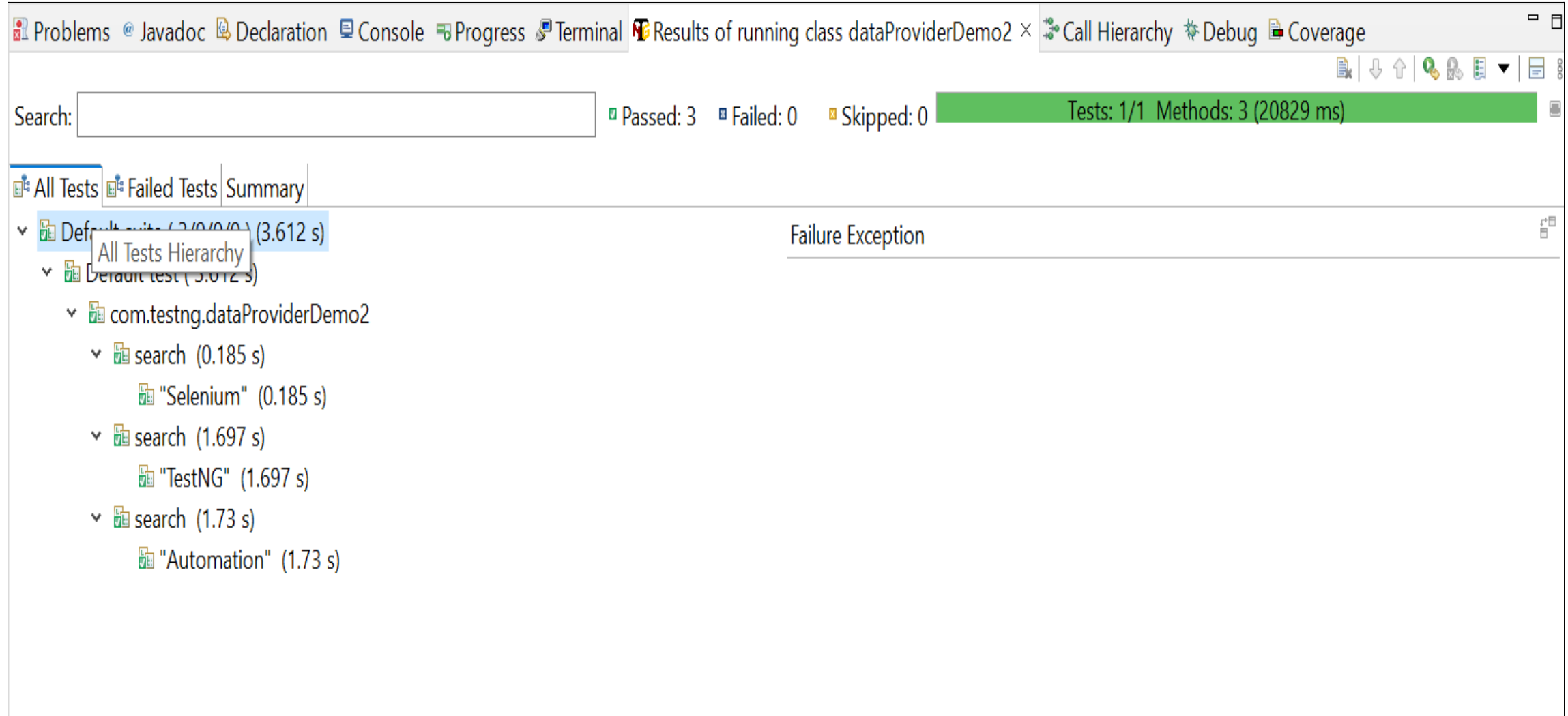
```
[RemoteTestNG] detected TestNG version 7.4.0
Start the test
Starting ChromeDriver 93.0.4577.15 (660fc11082ba57405eca2e8c49c3e1af756fbfae-refs/bran
Only local connections are allowed.
Please see https://chromedriver.chromium.org/security-considerations for suggestions o
ChromeDriver was started successfully.
Oct 11, 2021 3:49:35 PM org.openqa.selenium.remote.ProtocolHandshake createSession
INFO: Detected dialect: W3C
Keyword entered is : Selenium
Search result is displayed.
End the test
Start the test
Starting ChromeDriver 93.0.4577.15 (660fc11082ba57405eca2e8c49c3e1af756fbfae-refs/bran
Only local connections are allowed.
Please see https://chromedriver.chromium.org/security-considerations for suggestions o
ChromeDriver was started successfully.
Oct 11, 2021 3:49:39 PM org.openqa.selenium.remote.ProtocolHandshake createSession
INFO: Detected dialect: W3C
Keyword entered is : TestNG
Search result is displayed.
End the test
Start the test
Starting ChromeDriver 93.0.4577.15 (660fc11082ba57405eca2e8c49c3e1af756fbfae-refs/bran
Only local connections are allowed.
Please see https://chromedriver.chromium.org/security-considerations for suggestions o
ChromeDriver was started successfully.
Oct 11, 2021 3:49:42 PM org.openqa.selenium.remote.ProtocolHandshake createSession
INFO: Detected dialect: W3C
Keyword entered is : Automation
Search result is displayed.
End the test
PASSED: search("TestNG")
PASSED: search("Automation")
PASSED: search("Selenium")

=====
      Default test
      Tests run: 1, Failures: 0, Skips: 0
=====

=====
Default suite
Total tests run: 3, Passes: 3, Failures: 0, Skips: 0
```

Data Providers in TestNG

DataProvider – Example2 Output



Problems Javadoc Declaration Console Progress Terminal Results of running class dataProviderDemo2 × Call Hierarchy Debug Coverage

Search:

Passed: 3 Failed: 0 Skipped: 0 Tests: 1/1 Methods: 3 (20829 ms)

All Tests Failed Tests Summary

▼ Default suite (3.612 s) (3.612 s)

▼ Default test (3.612 s)

▼ com.testng.dataProviderDemo2

▼ search (0.185 s)

 "Selenium" (0.185 s)

▼ search (1.697 s)

 "TestNG" (1.697 s)

▼ search (1.73 s)

 "Automation" (1.73 s)

Failure Exception

Data Providers in TestNG

Use of Excel in DataProvider for TestNG. (.xlsx)

- You can create an Excel – File and placed it on the Desktop or any location.

OR

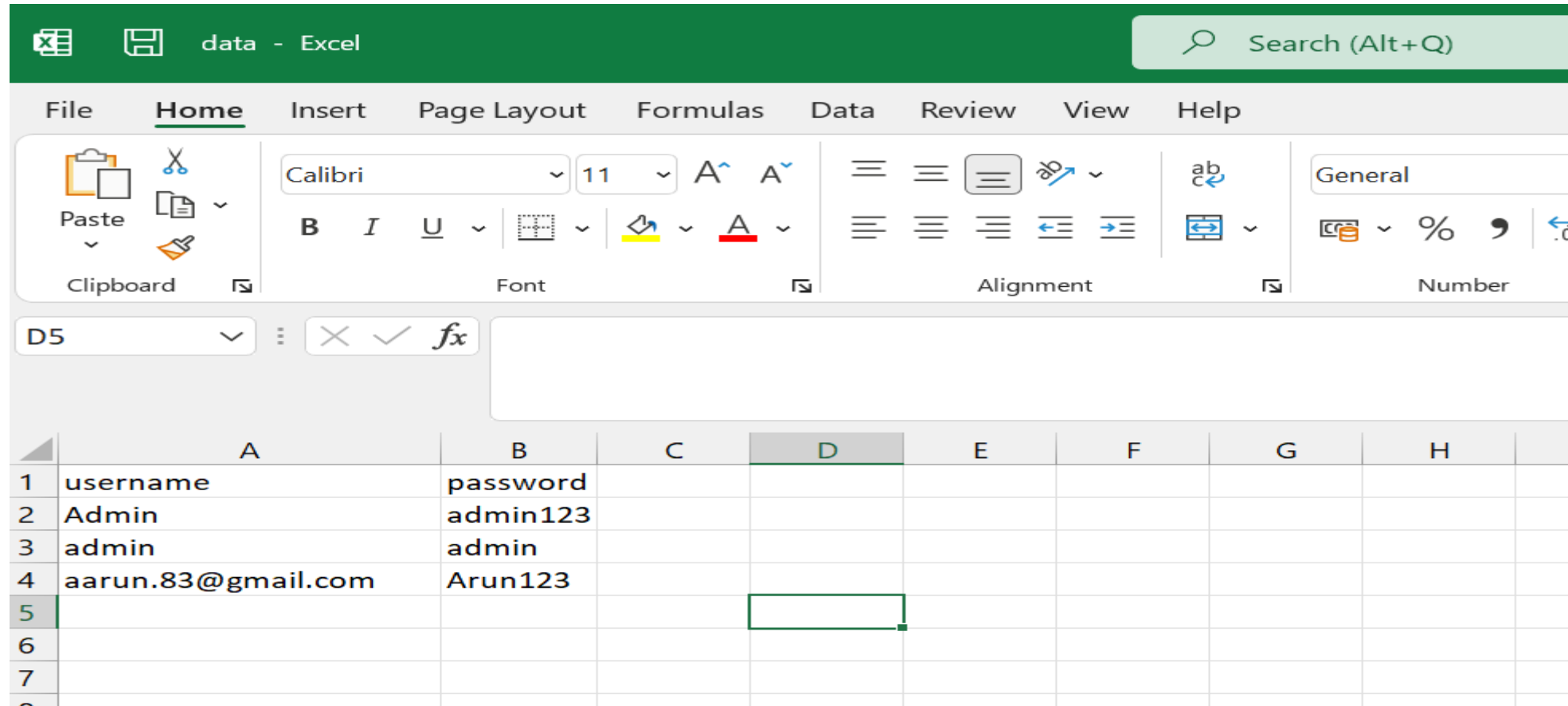
- You can create a Test Data folder in your project and place the Excel file within it.
- So, my datasheet looks like the below:

```
▼ 📁 src/main/java
  ▼ 📁 com.testng
    > 📄 dataDrivenTestingDemo.java
    > 📄 extentReport.java
    > 📄 ParallelTest.java
  📁 src/main/resources
  > 📁 src/test/java
  ▼ 📁 src/test/resources
    📄 data.xlsx
```

Data Providers in TestNG

Use of Excel in DataProvider for TestNG. (.xlsx)

- Sample excel file with data



	A	B	C	D	E	F	G	H
1	username	password						
2	Admin	admin123						
3	admin	admin						
4	aarun.83@gmail.com	Arun123						
5								
6								
7								
8								

DataProvider – Example3 – Using .xlsx File

```
import java.io.File;
import java.io.FileInputStream;
import java.io.FileNotFoundException;
import java.io.IOException;
import org.apache.poi.ss.usermodel.Cell;
import org.apache.poi.ss.usermodel.CellType;
import org.apache.poi.xssf.usermodel.XSSFCell;
import org.apache.poi.xssf.usermodel.XSSFRow;
import org.apache.poi.xssf.usermodel.XSSFSheet;
import org.apache.poi.xssf.usermodel.XSSFWorkbook;
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;
import org.testng.Assert;
import org.testng.annotations.DataProvider;
import org.testng.annotations.Test;
```


DataProvider – Example3

```
public class dataDrivenTestingDemo {  
    @Test(dataProvider="supplier")  
    public void loginTest(String email,String password) {  
        WebDriver driver = new ChromeDriver();  
        driver.manage().window().maximize();  
        driver.get("http://tutorialsninja.com/demo/");  
        driver.findElement(By.xpath("//span[text()='My Account']")).click();  
        driver.findElement(By.linkText("Login")).click();  
        driver.findElement(By.id("input-email")).sendKeys(email);  
        driver.findElement(By.id("input-password")).sendKeys(password);  
        driver.findElement(By.xpath("//input[@value='Login']")).click();  
        Assert.assertTrue(driver.getTitle().equals("My Account"));  
        driver.quit();  
    }  
    @DataProvider(name="supplier")  
    public Object[][] dataSupplier() {  
        String excelFilePath = System.getProperty("user.dir")+"\\src\\test\\resources\\data.xlsx";  
        File excelFile = new File(excelFilePath);  
        FileInputStream fis = null;  
        try {  
            fis = new FileInputStream(excelFile);  
        } catch (FileNotFoundException e) {  
            e.printStackTrace();  
        }  
    }  
}
```

DataProvider – Example3

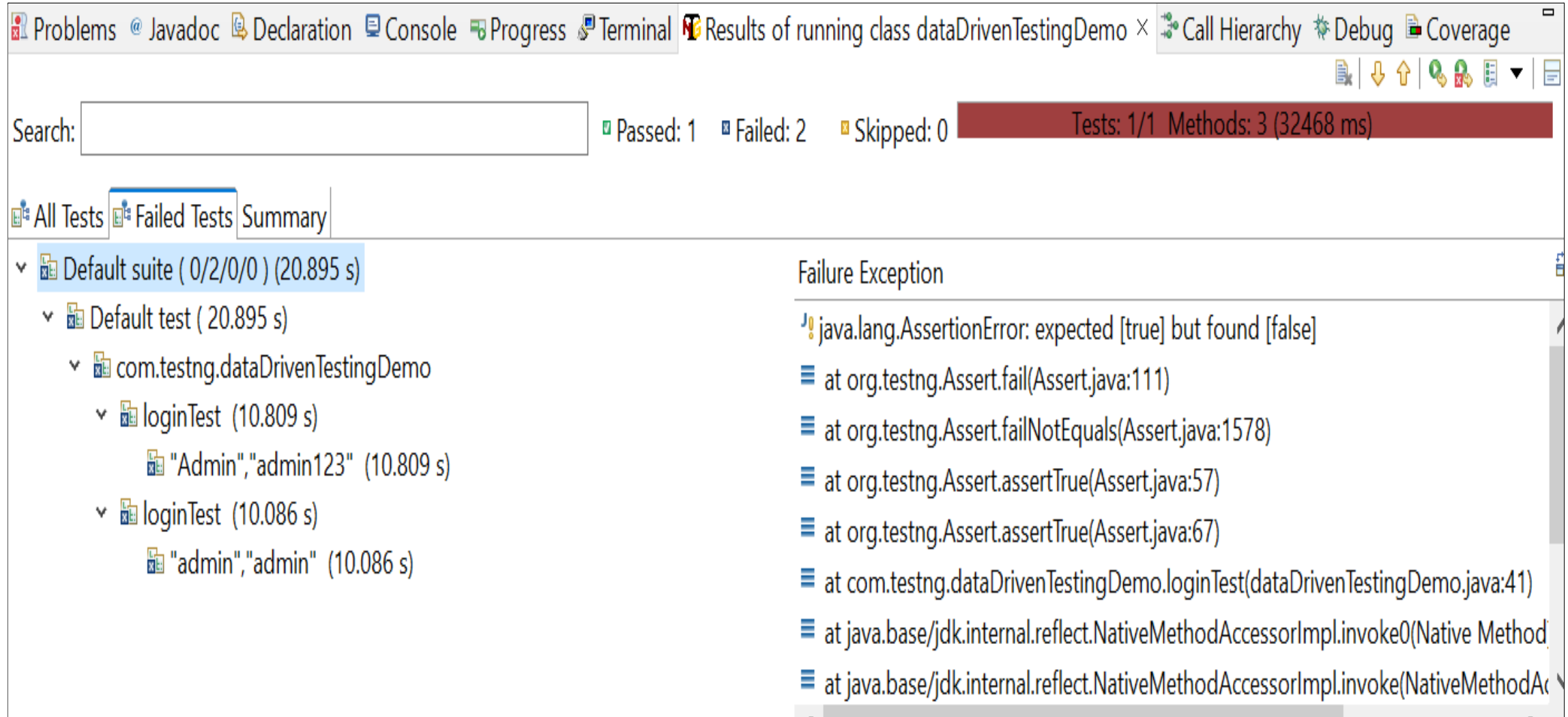
```
XSSFWorkbook workbook = null;  
try { workbook = new XSSFWorkbook(fis); }  
catch (IOException e) { e.printStackTrace(); }  
XSSFSheet sheet = workbook.getSheet("Sheet1");  
int rowCount = sheet.getPhysicalNumberOfRows();  
System.out.println(rowCount);  
int colCount = sheet.getRow(0).getLastCellNum();  
System.out.println(colCount);  
Object[][] data = new Object[rowCount-1][colCount];  
for(int r=0;r<rowCount-1;r++) {  
    XSSFRow row = sheet.getRow(r+1);  
    for(int c=0;c<colCount;c++) {  
        XSSFCell cell = row.getCell(c);  
        CellType cellType = cell.getCellType();  
        switch(cellType) {  
            case STRING:  
                data[r][c] = cell.getStringCellValue();  
                break;  
            case NUMERIC:  
                data[r][c] = (long) cell.getNumericCellValue();  
                System.out.println(data[r][c]);  
                data[r][c] = String.valueOf(data[r][c]);  
                System.out.println(data[r][c]);  
                break; } } }  
    return data;  
}
```

```
[RemoteTestNG] detected TestNG version 7.8.0
SLF4J: No SLF4J providers were found.
SLF4J: Defaulting to no-operation (NOP) logger implementation
SLF4J: See http://www.slf4j.org/codes.html#noProviders for further details.
ERROR StatusLogger Log4j2 could not find a logging implementation. Please add log4j-core to the classpath. Using SimpleLogger to log to the console
4
2
Oct 27, 2023 4:05:16 PM org.openqa.selenium.manager.SeleniumManager lambda$runCommand$1
INFO: Driver path: C:\Users\Arun Kumar\AppData\Roaming\npm\chromedriver.cmd
Oct 27, 2023 4:05:16 PM org.openqa.selenium.manager.SeleniumManager lambda$runCommand$1
INFO: Browser path: C:\Program Files\Google\Chrome\Application\chrome.exe
Oct 27, 2023 4:05:16 PM org.openqa.selenium.remote.RemoteWebDriver findTestNGTest
PASSED: com.testng.dataDrivenTestingDemo.loginTest("aarun.83@gmail.com", "Arun123")
FAILED: com.testng.dataDrivenTestingDemo.loginTest("Admin", "admin123")
java.lang.AssertionError: expected [true] but found [false]
    at org.testng.Assert.fail(Assert.java:111)

FAILED: com.testng.dataDrivenTestingDemo.loginTest("admin", "admin")
java.lang.AssertionError: expected [true] but found [false]
    at org.testng.Assert.fail(Assert.java:111)
    at org.testng.Assert.failNotEquals(Assert.java:1538)
```

Data Providers in TestNG

DataProvider – Example3 Output



Problems Javadoc Declaration Console Progress Terminal Results of running class dataDrivenTestingDemo × Call Hierarchy Debug Coverage

Search: Passed: 1 Failed: 2 Skipped: 0 Tests: 1/1 Methods: 3 (32468 ms)

All Tests Failed Tests Summary

- ▼ Default suite (0/2/0/0) (20.895 s)
 - ▼ Default test (20.895 s)
 - ▼ com.testng.dataDrivenTestingDemo
 - ▼ loginTest (10.809 s)
 - "Admin","admin123" (10.809 s)
 - ▼ loginTest (10.086 s)
 - "admin","admin" (10.086 s)

Failure Exception

```

! java.lang.AssertionError: expected [true] but found [false]
  at org.testng.Assert.fail(Assert.java:111)
  at org.testng.Assert.failNotEquals(Assert.java:1578)
  at org.testng.Assert.assertTrue(Assert.java:57)
  at org.testng.Assert.assertTrue(Assert.java:67)
  at com.testng.dataDrivenTestingDemo.loginTest(dataDrivenTestingDemo.java:41)
  at java.base/jdk.internal.reflect.NativeMethodAccessorImpl.invoke0(Native Method)
  at java.base/jdk.internal.reflect.NativeMethodAccessorImpl.invoke(NativeMethodAc
  
```

Rerun Failed Test Cases



Rerun Failed Test Cases

testng-failed.xml

- When you execute a TestNG suite (e.g., testng.xml), TestNG automatically generates an extra XML file in your project's output folder named:

test-output/testng-failed.xml

- It contains only the failed test methods from your last suite execution.
- You don't have to rerun the entire test suite — you can rerun only the failed ones using this file.

Rerun Failed Test Cases

testng-failed.xml

- When you run your suite (say testng.xml) from:
 1. Eclipse (Run As → TestNG Suite)
 2. Maven (mvn test)
 3. Jenkins job (with TestNG runner)
- TestNG creates the test-output/ directory. Inside it, you'll find:
- The testng-failed.xml is auto-generated if at least one test failed in your run.

```
test-output/  
├─ index.html  
├─ testng-results.xml  
├─ testng-failed.xml  
└─ <other reports...>
```

Rerun Failed Test Cases

Structure of testng-failed.xml

```
<!DOCTYPE suite SYSTEM "https://testng.org/testng-1.0.dtd">
<suite name="Failed suite [MySuite]" parallel="false">
  <test name="Failed test [SmokeTest_Chrome]" verbose="2">
    <classes>
      <class name="tests.LoginTests">
        <methods>
          <include name="verifyInvalidLogin" />
        </methods>
      </class>
      <class name="tests.ProductTests">
        <methods>
          <include name="verifyAddToCart" />
        </methods>
      </class>
    </classes>
  </test>
</suite>
```


Rerun Failed Test Cases

How to re-run failed tests

A. From Eclipse (Run As → TestNG Suite)

1. After a suite run, go to: **test-output** → **testng-failed.xml**
2. Right-click → Run As → TestNG Suite.

B. From Command Line using Maven

```
mvn test -Dsurefire.suiteXmlFiles=test-output/testng-failed.xml
```

How to re-run failed tests

B. From Command Line using Maven

```
<build>
  <plugins>
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-surefire-plugin</artifactId>
      <version>3.2.3</version>
      <configuration>
        <testFailureIgnore>>false</testFailureIgnore>
        <rerunFailingTestsCount>2</rerunFailingTestsCount>
      </configuration>
    </plugin>
  </plugins>
</build>
```

TestNG Reports



TestNG Reports

- When you execute a TestNG test suite, TestNG automatically generates detailed HTML and XML reports summarizing your test execution results.
- These reports show:
 - Total tests run
 - Passed, failed, skipped tests
 - Execution time
 - Suite hierarchy (tests, classes, methods)
 - Stack trace of failures
- After every run, reports are automatically created inside: **test-output/**

Types of Reports Generated by TestNG

- When a suite finishes, you'll see files like these:

File	Description
index.html	Main HTML dashboard (most commonly opened)
emailable-report.html	Summary report that can be shared via email
testng-results.xml	XML version of report — used by CI/CD or Allure
testng-failed.xml	Contains only failed test cases for rerun
junitreports/	Optional — generated when configured for CI tools
old/	Backup of reports from previous runs

Example Folder Structure

```
test-output/  
|  
├─ index.html  
├─ emailable-report.html  
├─ testng-results.xml  
├─ testng-failed.xml  
└─ Default suite/  
    ├─ test.html  
    ├─ classes.html  
    └─ groups.html
```


Integration with Selenium for end-to-end automation



Selenium and TestNG

- Using TestNG's testing framework capabilities to design, manage, execute, and report Selenium-based UI automation scenarios — end-to-end.
- Essentially, Selenium drives the browser actions, and TestNG provides the test organization, parameterization, grouping, execution control, and reporting.

Selenium and TestNG

Aspect	Purpose
Framework Integration	Run Selenium tests using TestNG annotations, groups, and XMLs
Data Flow	Use DataProviders and parameters to feed test data dynamically
Assertions	Validate UI behavior (titles, messages, cart items, etc.)
Reports	Generate TestNG or Extent/Allure reports
Reruns	Automatically retry failed Selenium tests
End-to-End Flow	Execute full user journeys on Demoblaze (Login → Add to Cart → Logout)

TestNG Listeners



Introduction

- ***TestNG listeners are the piece of code that listens to the events occurring in the TestNG.***
- For example, we want to print the exception error onto the reports only if the test fails. Here, we can apply a TestNG listener that will listen to the event of "***failing of test case***" and when it does, it will log the error.
- The listener is defined as an interface that modifies the default TestNG's behaviour. There are several interfaces that allow you to modify TestNG's behaviour that are called "TestNG Listeners". It allows customizing TestNG reports or logs.

TestNG Listeners

Types Of Listeners In TestNG

1. ITestListener
 2. IReporter
 3. ISuiteListener
 4. IInvokedMethodListener
 5. IHookable
 6. IConfigurationListener
 7. IConfigurableListener
 8. IAnnotationTransformer
 9. IExecution
 10. IMethodInterceptor
- When you implement one of these interfaces, you can let TestNG know about it in either of the following ways:
 - 1. Using in your testng.xml file.**
 - 2. Using the @Listeners annotation on any of your test classes.**

ITestListener In TestNG

- The ITestListener **implements** since it is an interface, and the class in which we implement the listener overrides the ITestListener methods.
- ITestListener listens to specific events (depending on its methods) and executes the code written inside the method.
- With ITestListener in TestNG, we can also log the events onto the reports using the Selenium web driver. The ITestListener contains the following methods:

1. onStart: This method invokes when the test class is instantiated and before executing any test method.

Syntax: void onStart(ITestContext context);

ITestListener In TestNG

2. onFinish: This method invokes when all the test methods have run, and calling of all of their configuration methods happens.

Syntax: void onFinish(ITestContext context);

3. onStart: This method invokes every time a test method is called and executed.

Syntax: void onStart(ITestResult result);

4. onSuccess: This method is invoked every time a test case passes (succeeds).

Syntax: void onSuccess(ITestResult result);

5. onFailure: This method invokes every time a test case fails.

Syntax: void onFailure(ITestResult result);

ITestListener In TestNG

6. **onTestSkipped**: This method invokes every time a test skips.

Syntax: void onTestSkipped (ITestResult result);

7. **onTestFailedButWithinSuccessPercentage**: This method invokes when the test method fails as a whole but has passed a certain success percentage, which is defined by the user.

Syntax: void onTestFailedButSuccessPercentage (ITestResult result);

TestNG Listeners

Steps to use listener - ITestListener

Step 1) Create a class “ListenerDemo” that implements ‘ITestListener’. Add methods like onTestFailure, onTestSkipped, onTestStart, onTestSuccess to this class

Step 2) Create another class “ListenerTestCases” for the login process automation. Selenium will execute these ‘TestCases’ to login automatically.

Step 3) Next, implement this listener in our regular project class i.e. “ListenerTestCases “. There are two different ways to connect to the class and interface.

3.1 The first way is to use Listeners annotation (@Listeners) as shown below:

```
1 | @Listeners(com.selenium.testng.TestNGDemo.ListenerDemo.class)
```

3.2 The second way is to Using XML listener section

```
<listeners>    <listener class-name="com.pomwithPageFactory.ListenerDemo"/> </listeners>
```


Program (Contd.)

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE suite SYSTEM "http://testng.org/testng-1.0.dtd" >
<suite name="Test Suite2" verbose="2">
<listeners>
<listener class-name="com.pomwithPageFactory.ListenerDemo"></listener>
</listeners>
<test name="ITestListenerTest">
<classes>
<class name="com.pomwithPageFactory.DashboardTests">
</class>
</classes>
</test>
</suite>
```

TestNG Listeners

Steps to use listener - ITestListener

Step 4): Execute the “ListenerTestCases” class. Methods in class “TestPass ” are called automatically according to the behaviour of methods annotated as @Test.

Step 5): Verify the Output that logs display at the console.

Note: When to choose which?

- **Use testng.xml → when you want listener applied to multiple classes/suite.**
- **Use @Listeners → when you want it only for specific test class**

ITestListener Example Step1: Create ITestListener Class

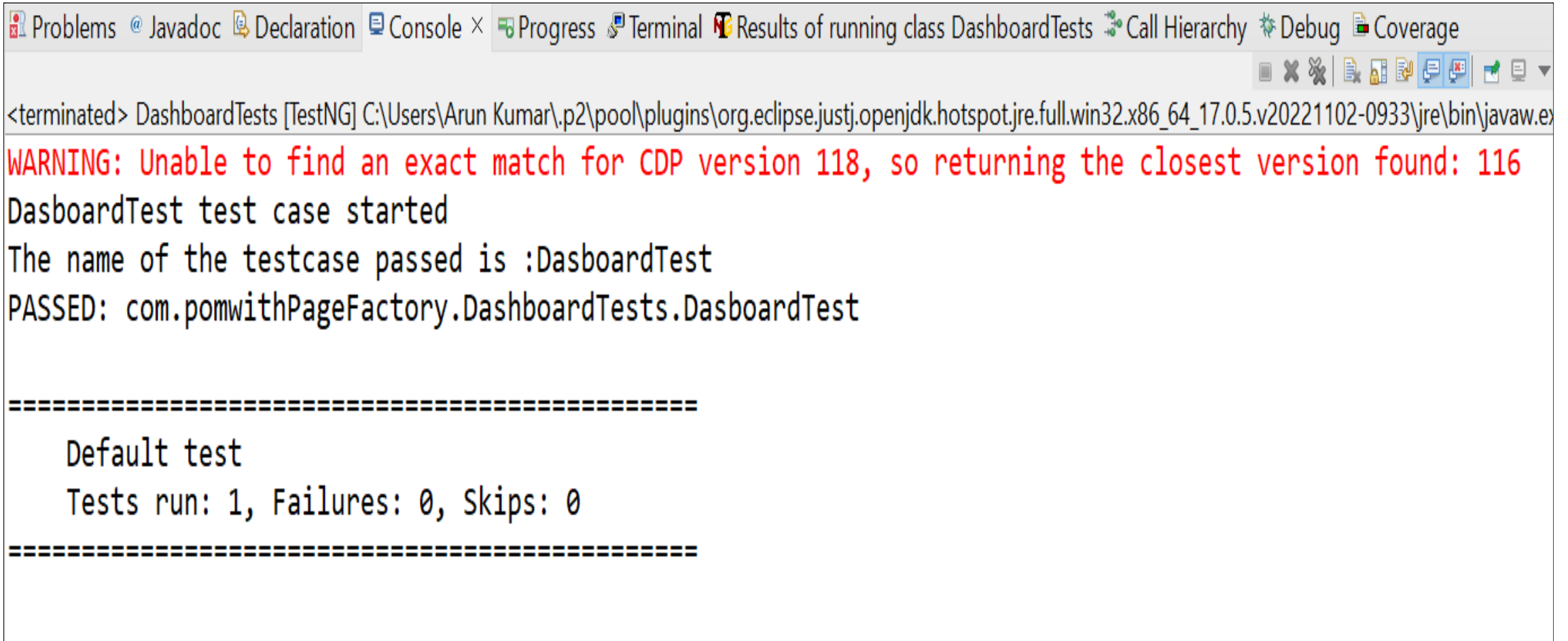
```
import org.testng.ITestListener;
import org.testng.ITestResult;
public class ListenerDemo implements ITestListener {
// When Test case get failed, this method is called.
public void onTestFailure(ITestResult Result) {
System.out.println("The name of the testcase failed is :" + Result.getName());
}
// When Test case get Skipped, this method is called.
public void onTestSkipped(ITestResult Result) {
System.out.println("The name of the testcase Skipped is :" + Result.getName());
}
// When Test case get Started, this method is called.
public void onTestStart(ITestResult Result) {
System.out.println(Result.getName() + " test case started");
}
// When Test case get passed, this method is called.
public void onTestSuccess(ITestResult Result) {
System.out.println("The name of the testcase passed is :" + Result.getName());
}
}
```

ITestListeners Example: Connect ITestListener Class to the Regular class with test Methods.

```
@Listeners(com.pomwithPageFactory.ListenerDemo.class)
public class DashboardTests extends BaseTest {
    LoginPage objLogin;
    DashboardPage objDashboardPage;
    @Test(priority = 1)
    public void DashboardTest() {
        objLogin = new LoginPage(driver);
        // login to application
        objLogin.login("Admin", "admin123");
        // go the dashboard page
        objDashboardPage = new DashboardPage(driver);
        // Verify dashboard page
        Assert.assertTrue(objDashboardPage.getHomePageText().contains("Dashboard"));
    }
}
```

TestNG Listeners

ITestListeners Example: Output



The screenshot shows the Eclipse IDE's console window. The title bar includes tabs for Problems, Javadoc, Declaration, Console (active), Progress, Terminal, Results of running class DashboardTests, Call Hierarchy, Debug, and Coverage. The console output is as follows:

```
<terminated> DashboardTests [TestNG] C:\Users\Arun Kumar\.p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64_17.0.5.v20221102-0933\jre\bin\javaw.exe
WARNING: Unable to find an exact match for CDP version 118, so returning the closest version found: 116
DashboardTest test case started
The name of the testcase passed is :DashboardTest
PASSED: com.pomwithPageFactory.DashboardTests.DashboardTest

=====
    Default test
    Tests run: 1, Failures: 0, Skips: 0
=====
```

ITestListener Example: Screenshot on Test Failure

```
import org.testng.ITestListener;
import org.testng.ITestResult;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.TakesScreenshot;
import org.openqa.selenium.OutputType;
import java.io.File;
import org.apache.commons.io.FileUtils;

public class ListenerDemo implements ITestListener {

    @Override
    public void onTestFailure(ITestResult result) {
        System.out.println("Test Failed: " + result.getName());

        Object currentClass = result.getInstance();
        WebDriver driver = ((BaseTest) currentClass).driver;

        takeScreenshot(driver, result.getName());
    }
}
```

IEventListener Example: Screenshot on Test Failure

```
public void takeScreenshot(WebDriver driver, String testName) {  
    try {  
        File src = ((TakesScreenshot) driver).getScreenshotAs(OutputType.FILE);  
        String path = System.getProperty("user.dir") + "/screenshots/" + testName + "_" +  
            System.currentTimeMillis() + ".png";  
        FileUtils.copyFile(src, new File(path));  
        System.out.println("Screenshot saved at: " + path);  
    } catch (Exception e) {  
        e.printStackTrace();  
    }  
}
```

Quiz



1) Which of the following is a TestNG report?

a) Testcase report

b) Test suite report

c) Emailable report

d) Failure report

Answer : Option c)

Quiz



2) Is the statement true: TestNG uses the method of alphabetical execution to execute its test method.

a) True

b) False

c) Both a and b

d) Not a nor b

Answer : Option a)

Quiz



3) Which tag is used to run tests in a group in TestNG?

a) execute

b) Run

c) add

d) include

Answer : Option b)

Quiz



4) Parallel testing can be applied at

a) Methods

b) Tests

c) Classes

d) All the above

Answer : Option d)

THANK YOU